FORMATION

Jenkins

Intégration continue – de la théorie à la chaîne de fabrication logicielle

Partie 1 · L'Intégration Continue | Partie 2 · Utilisation de Jenkins | Partie 3 · Construire un projet Java avec Maven

Utopios – MA-IT Consulting · Support v1.6

Objectifs de la formation

À l'issue de la formation, vous saurez

- Expliquer les principes et les bonnes pratiques de l'intégration continue
- Situer Jenkins dans la chaîne de fabrication logicielle
- Installer, configurer et administrer un serveur Jenkins
- Créer des jobs Freestyle et des Pipelines (`Jenkinsfile`)
- Construire, tester, analyser et déployer un projet Java/Maven
- Gérer les notifications, les rapports et les habilitations

Prérequis

- Bases de Java et de Maven
- Notions de Git (clone, commit, push, branch)
- Ligne de commande Linux / Docker (recommandé)

Environnement de TP

- Jenkins LTS (JDK 21) lancé en Docker
- Git + dépôt GitHub/GitLab
- JDK 21, Maven 3.9
- SonarQube Community (Docker), Nexus Repository (Docker)

Programme

1 – L'Intégration Continue

- Définition, principes
- Notions de génie logiciel
- Best practices d'intégration continue
- La chaîne de fabrication logicielle

2 – Utilisation de Jenkins

- Concepts, définitions
- Jenkins comme serveur de build
- Archétypes de projet, déclencheurs, résultat du build, workspace
- Visite guidée de l'interface, Jenkins dans l'IDE
- Installation et démarrage, configuration générale, plugins

3 – Construire un projet Java avec Maven

- Rappels Maven, création d'un job, accès aux sources
- Paramétrage de Maven, rapports de tests unitaires
- Envoi de mails de notification
- Déploiement automatique
- Rapports d'analyse qualité
- Habilitations

Fil rouge : **library-api**, une API REST Spring Boot (Java 21, Maven) de gestion de bibliothèque, hébergée sur Git, construite et déployée par Jenkins.

01

L'Intégration Continue

Définition et principes · Notions de génie logiciel · Best practices · La chaîne de fabrication logicielle

Intégration continue : définition

> « L'intégration continue est une pratique de développement où les membres d'une équipe intègrent leur travail fréquemment – généralement chaque personne au moins une fois par jour. Chaque intégration est vérifiée par un build automatisé (tests inclus) pour détecter les erreurs d'intégration le plus tôt possible. »
>
> Martin Fowler, *Continuous Integration* (version de 2006 ; article révisé en 2024)

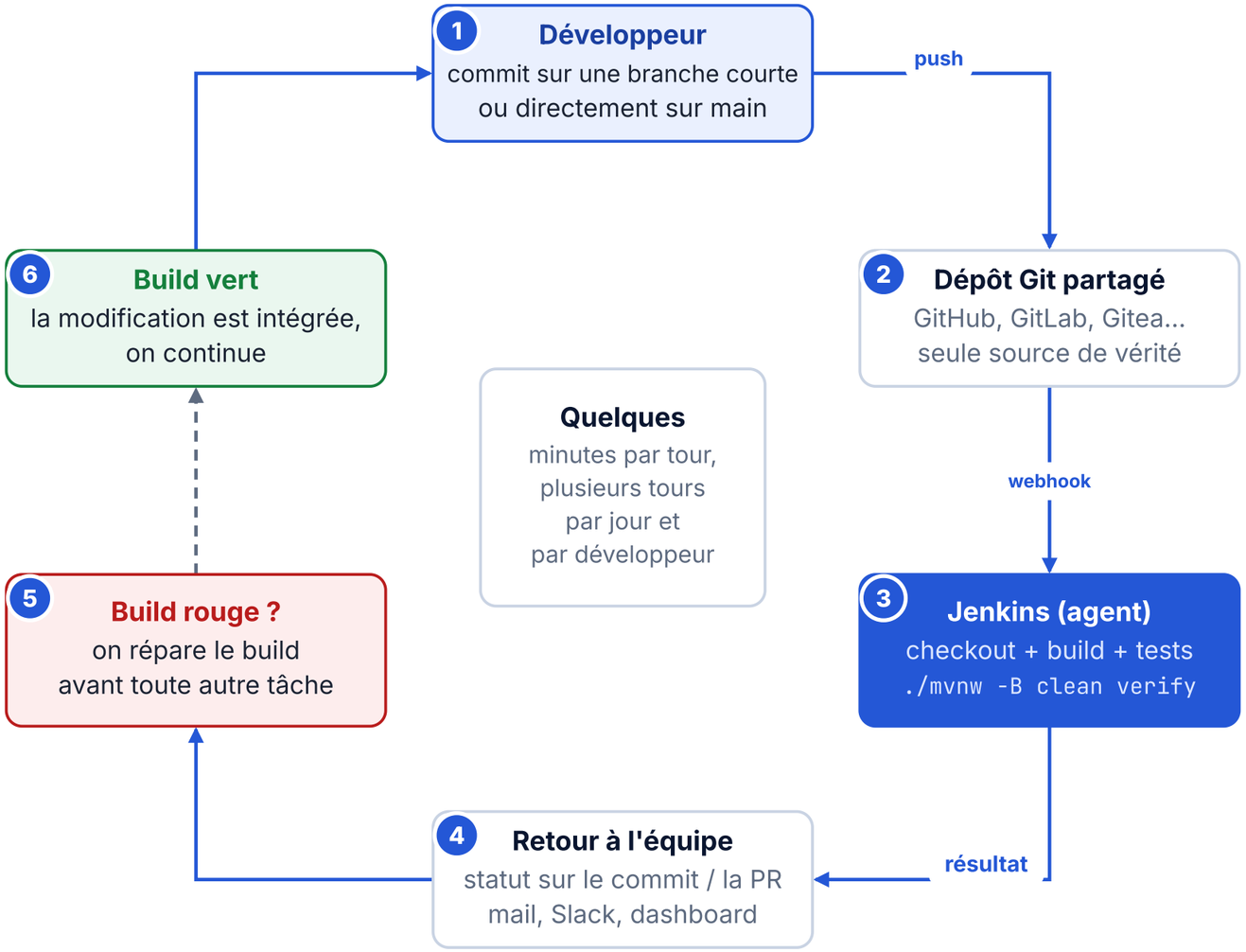
Les 3 mots-clés

- **Intégrer** : fusionner son code avec celui des autres dans une branche partagée (`main`)
- **Continue** : plusieurs fois par jour, pas une fois par sprint
- **Vérifiée** : chaque intégration déclenche un build + des tests automatiques

Ce que l'IC n'est pas

- Un simple serveur qui compile la nuit (*nightly build*)
- Un outil : Jenkins est un *moyen*, l'IC est une *pratique*
- Une garantie de qualité sans tests : sans tests, un build vert ne prouve que la compilation

Schéma : la boucle de l'intégration continue



Comment le lire

1. Le développeur intègre **souvent** un petit changement : commit sur une branche de quelques heures, ou sur `main`.
2. Le `push` dépose le changement dans le dépôt partagé : c'est là que l'intégration a lieu.
3. Le dépôt prévient Jenkins (webhook). Jenkins construit et teste **la version intégrée**, sur un agent propre, pas sur le poste du développeur.
4. Le résultat revient à l'équipe en quelques minutes : statut sur le commit ou la PR, notification.
5. Rouge : la priorité de l'équipe est de remettre le build au vert (corriger ou annuler le commit).
6. Vert : le changement est intégré ; le tour suivant repart d'une base qui fonctionne.

À retenir

La valeur de l'IC vient de la **fréquence** des tours et de la **vitesse** du retour : un tour par jour et un build d'une heure, c'est déjà trop lent.

Pourquoi l'intégration continue ?

Le problème : « l'enfer de l'intégration »

- Chaque développeur travaille des semaines sur sa branche
- À la fusion : conflits massifs, régressions, comportements imprévus
- Plus le délai entre deux intégrations est long, plus le coût de la fusion croît (de façon non linéaire)

Le coût d'un bug selon sa détection

Détecté…	Ce qu'il coûte à corriger
À l'écriture (IDE, test unitaire)	Le contexte est en tête : quelques minutes
Au build d'intégration (CI)	Commit identifié, auteur prévenu : vite corrigé
En recette / QA	Diagnostic, nouveau build, nouvelle livraison en recette
En production	Incident, impact utilisateurs, correctif en urgence

Les ratios souvent cités (« ×100 en production ») ne reposent sur aucune étude vérifiable : on retient la tendance, pas le chiffre.

Les bénéfices attendus

- **Feedback rapide** : un commit cassé est connu en quelques minutes
- **Risque réduit** : des intégrations petites et fréquentes
- **Code toujours livrable** : `main` est en permanence dans un état sain
- **Transparence** : l'état du projet est visible de tous (tableau de bord)
- **Automatisation** : moins de tâches manuelles, moins d'erreurs humaines
- **Base de la livraison continue** : sans CI, pas de CD

Mesure

Les métriques DORA (*deployment frequency*, *change lead time*, *change fail rate*, *failed deployment recovery time*, *deployment rework rate*) : la CI fait partie des capacités qui les améliorent.

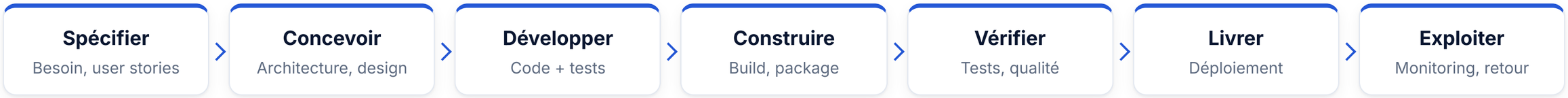
Les principes de l'intégration continue

Les pratiques décrites par Martin Fowler (liste de 2006 ; la version 2024 remplace le point 9 par « masquer le travail en cours », feature flags) :

1. **Un dépôt de sources unique** – tout ce qui est nécessaire au build est versionné (code, scripts, config, schémas de BDD)
2. **Automatiser le build** – une seule commande construit tout (`mvn verify`)
3. **Rendre le build auto-testant** – les tests font partie du build
4. **Tout le monde commit sur la branche principale chaque jour**
5. **Chaque commit déclenche un build** sur une machine d'intégration (pas « chez moi ça marche »)
6. **Réparer immédiatement un build cassé** – priorité absolue de l'équipe
7. **Garder le build rapide** – objectif : < 10 minutes
8. **Tester dans un clone de l'environnement de production**
9. **Rendre le dernier exécutable facilement accessible** – artefacts publiés
10. **Tout le monde voit ce qui se passe** – dashboard, notifications
11. **Automatiser le déploiement** – la porte d'entrée vers la livraison continue

Jenkins outille les points 5, 9, 10 et 11, et aide au point 7 (parallélisme, agents). Les points 2 et 3 relèvent de l'outil de build (Maven) ; les autres, de la discipline d'équipe.

Notions de génie logiciel : le cycle de vie



Modèles historiques

- **Cycle en V / cascade** : phases séquentielles, intégration tardive (« big bang »)
- **Itératif / Agile** : cycles courts, intégration permanente, feedback continu
- **DevOps** : la boucle se referme – Dev et Ops partagent les outils et la responsabilité de la mise en production

Où se place la CI ?

- Elle automatise **Construire** et **Vérifier** à chaque changement
- Elle alimente **Livrer** (CD) avec des artefacts validés
- Elle impose des exigences aux phases amont : code testable, build scriptable, dépendances déclarées

Shift-left
Déplacer la vérification le plus tôt possible dans le cycle : compiler, tester, analyser dès le commit.

Génie logiciel : gestion de version et stratégies de branches

Prérequis indispensable à la CI

- Un SCM (*Source Control Management*) : Git, très largement majoritaire (GitHub, GitLab, Bitbucket, Gitea)
- Commits atomiques, messages explicites
- Le dépôt est la **seule source de vérité** : rien de nécessaire au build n'est hors Git

Stratégies courantes

Stratégie	Principe	Compatibilité CI
Trunk-based	Tous sur `main`, branches < 1-2 jours, feature flags	Excellente
GitHub Flow	Branche courte + Pull Request + merge	Très bonne
GitFlow	`develop`, `release/*`, `hotfix/*`, `feature/*`	Lourde, intégration retardée

Impact sur Jenkins

- Un job **Multibranch Pipeline** construit automatiquement chaque branche et chaque PR
- Le `Jenkinsfile` est versionné avec le code : la définition du pipeline suit l'évolution du projet
- Avec une règle de protection de branche (*required status check*), une PR ne peut être fusionnée que si son build est vert

Règle d'or

Une branche qui vit plus de quelques jours sans être intégrée n'est plus de l'intégration *continue*.

Schéma : trois stratégies de branches vues dans Git

Trunk-based

branches de quelques heures, intégrées dans la journée

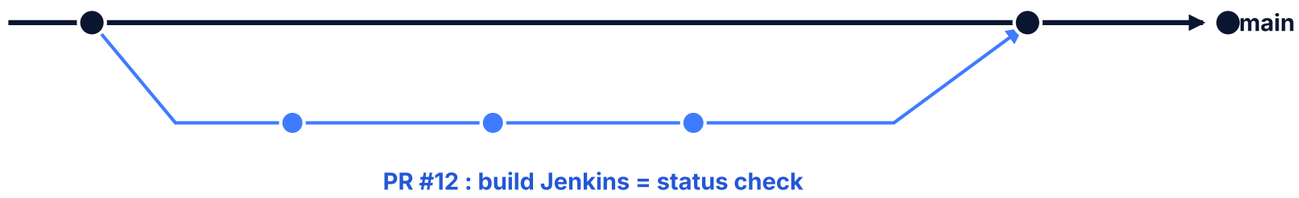
1



GitHub Flow

une branche + une Pull Request ; le build Jenkins conditionne le merge

2

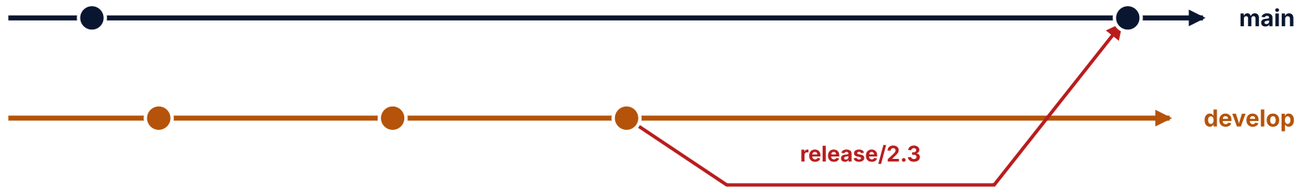


✔ vert : merge autorisé · ✘ rouge : bloqué

GitFlow

develop, release/*, hotfix/* : l'intégration attend la release

3



Comment le lire

1. **Trunk-based** : tout le monde intègre sur `main` plusieurs fois par jour ; les rares branches vivent quelques heures. Jenkins construit `main` en continu.
2. **GitHub Flow** : une branche par changement, fusionnée par une Pull Request. Le build Jenkins de la PR est un *status check* : avec une règle de protection de branche, une PR rouge ne peut pas être fusionnée.
3. **GitFlow** : le travail s'accumule sur `develop` et n'arrive sur `main` qu'à la release : les conflits et les régressions sont découverts tard.
4. Dans les trois cas, un **Multibranch Pipeline** construit chaque branche et chaque PR avec le Jenkinsfile qu'elle contient.

À retenir

Plus une branche vit longtemps sans être fusionnée, moins on fait d'intégration continue.

Génie logiciel : build automatisé et reproductible

Build automatisé

- Une commande unique, sans intervention humaine, depuis un checkout propre
- Outils de build : **Maven**, Gradle (Java) ; npm/pnpm (JS) ; Make/CMake (C/C++) ; dotnet (C#) ; pip/poetry (Python)
- Le build produit un **artefact** : JAR/WAR, image Docker, paquet npm, binaire…

Build reproductible

- Mêmes sources + mêmes dépendances + même outil ⇒ même résultat
- Versions de dépendances **figées** (pas de `LATEST` ni de plages ouvertes)
- Outils et JDK versionnés et déclarés (Maven Wrapper `./mvnw`, `.tool-versions`, image Docker)
- Pas de dépendance à l'état de la machine (fichiers locaux, variables non déclarées)

Pourquoi c'est critique pour Jenkins

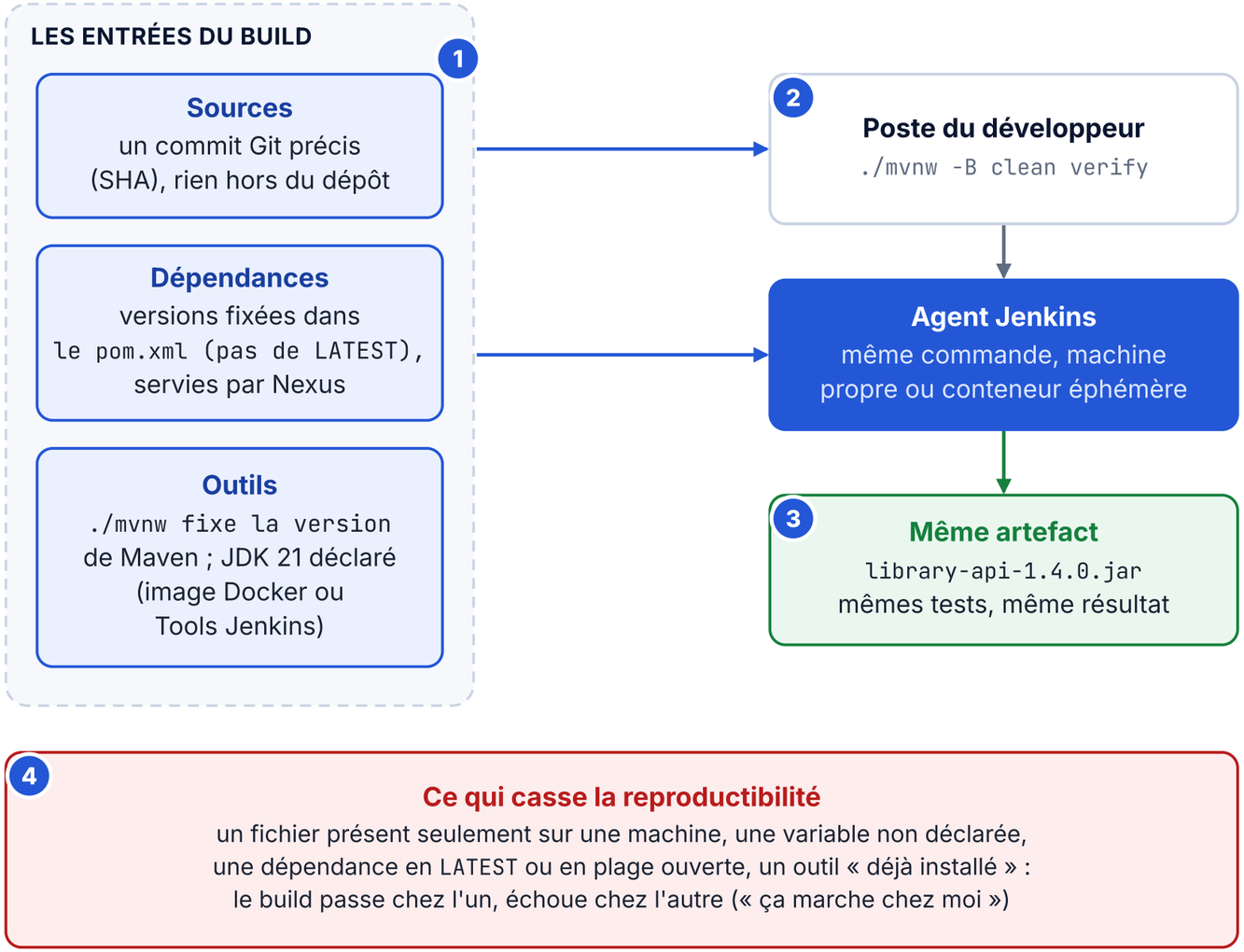
- Jenkins exécute le build sur un **agent**, pas sur le poste du développeur
- Un build non reproductible donne des résultats aléatoires : « ça marche chez moi »
- Les agents doivent être **interchangeables** (idéalement éphémères : conteneurs)

```
# Build complet du fil rouge, sans Jenkins
git clone https://git.example.com/utopios/library-api.git
cd library-api
./mvnw -B clean verify
ls target/*.jar
```

Test simple

Si un nouvel arrivant ne peut pas exécuter le build avec une seule commande sur une machine vierge, le build n'est pas prêt pour la CI.

Schéma : un build reproductible, ce sont des entrées maîtrisées



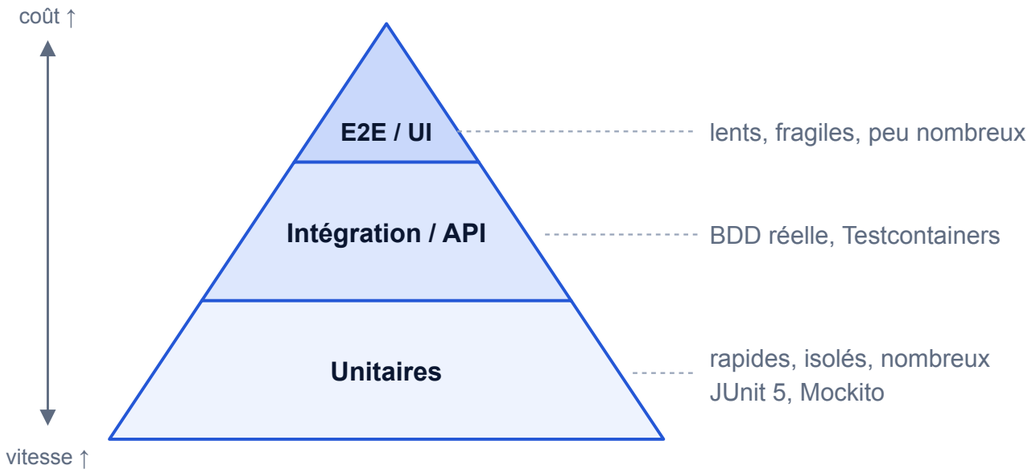
Comment le lire

1. Trois familles d'entrées décident du résultat : le **code** (un commit), les **dépendances** (versions figées) et les **outils** (Maven et JDK versionnés).
2. La même commande tourne sur le poste et sur l'agent : Jenkins n'invente rien, il rejoue ce que le développeur sait déjà lancer.
3. Mêmes entrées ⇒ même artefact et mêmes tests : un build rouge dans Jenkins signale un vrai problème, pas une différence de machine.
4. Toute entrée cachée (fichier local, variable, version flottante) rend le résultat aléatoire : on la rend explicite et versionnée.

À retenir

Test simple : un nouvel arrivant lance le build sur une machine vierge avec **une seule commande**.

Génie logiciel : les tests automatisés



La pyramide des tests

- **Unitaires** : une classe/méthode, sans I/O – exécutés par Maven `surefire` en phase `test`
- **Intégration** : plusieurs composants, base de données, HTTP – `failsafe` en phase `integration-test`
- **End-to-end** : parcours utilisateur complet (Selenium, Playwright, Cypress)

Exigences pour la CI

- **Déterministes** : pas de tests flaky (aléatoires) – ils détruisent la confiance dans le build
- **Rapides** : le feedback doit arriver en minutes
- **Rapports standard** : format XML JUnit, lisible par Jenkins
- Résultat binaire : un test rouge = build UNSTABLE ou FAILURE

Génie logiciel : qualité du code et analyse statique

Qualité interne vs externe

- **Externe** : le logiciel fait ce qu'on attend (tests fonctionnels)
- **Interne** : le code est maintenable, lisible, sûr (analyse statique, revue)

Ce que l'analyse statique détecte sans exécuter le code

- Bugs probables (NPE, ressources non fermées, comparaisons incorrectes : SpotBugs, PMD)
- Vulnérabilités (injection SQL, secrets en dur, crypto faible)
- Code smells, duplication, complexité cyclomatique
- Non-respect des conventions (Checkstyle, PMD)
- Dépendances vulnérables (OWASP Dependency-Check, Trivy)

Indicateurs usuels

Indicateur	Outil	Seuil typique
Couverture de tests	JaCoCo (mesure) + SonarQube (seuil)	≥ 80 % sur nouveau code
Bugs / vulnérabilités	SonarQube	0 bloquant / critique
Duplication	SonarQube	< 3 %
Dette technique	SonarQube	Note A ou B
CVE dans les dépendances	Dependency-Check	Aucune CVSS ≥ 7

Quality Gate

Un ensemble de seuils dont le non-respect fait échouer le build. C'est la CI qui l'applique, pas la bonne volonté.

Best practices d'intégration continue (1/2)

Committer souvent, en petits lots

Plusieurs commits par jour sur la branche d'intégration. Un petit changement cassé se diagnostique en minutes ; un gros, en heures.

Builder localement avant de pousser

`./mvnw verify` avant `git push`. La CI confirme, elle ne découvre pas.

Build rapide (< 10 min)

Paralléliser les tests, mettre en cache les dépendances (`~/.m2`), séparer un pipeline rapide (unitaires) d'un pipeline long (intégration, E2E) déclenché ensuite.

Ne jamais committer sur un build cassé

Empiler des commits sur un `main` rouge rend le diagnostic impossible et bloque toute l'équipe.

Réparer un build cassé immédiatement

Celui qui casse répare (ou revert) dans les 10 minutes. Le build cassé est l'urgence n°1 de l'équipe.

Un seul chemin de build

Le même `Jenkinsfile` / la même commande pour tous les environnements ; pas de build « spécial prod » fait à la main.

Best practices d'intégration continue (2/2)

Pipeline as Code

La définition du pipeline (`Jenkinsfile`) est versionnée avec le code source, revue en PR, historisée.

Agents éphémères et identiques

Construire dans des conteneurs Docker ou des pods Kubernetes jetables : pas de dérive de configuration entre agents.

Construire une seule fois, promouvoir ensuite

L'artefact testé est celui qui sera déployé en recette puis en production (même SHA, même numéro de version). On ne rebuild pas par environnement.

Secrets hors du code

Mots de passe, tokens et clés dans le **Credentials store** de Jenkins (ou Vault), jamais dans le dépôt ni dans les logs.

Rendre l'état visible

Notifications ciblées (mail, Slack/Teams), dashboards, badges de statut dans le README. Alerter sur les changements d'état, pas sur chaque build vert.

Mesurer et améliorer

Durée de build, taux d'échec, temps de réparation (MTTR). Un build qui ralentit de semaine en semaine finit par être ignoré.

Anti-patterns fréquents

Le build de nuit uniquement
Le feedback arrive le lendemain matin ; 20 commits sont suspects à la fois.

Les tests qu'on désactive « temporairement »
`@Disabled` qui reste des mois. Un test ignoré est un test supprimé.

Le build rouge « connu »
Tout le monde sait que « c'est normal, c'est le test X ». Le signal n'a plus de valeur.

Les jobs configurés à la souris
Des dizaines de jobs Freestyle, non versionnés, modifiés à la main, impossibles à reproduire après un crash.

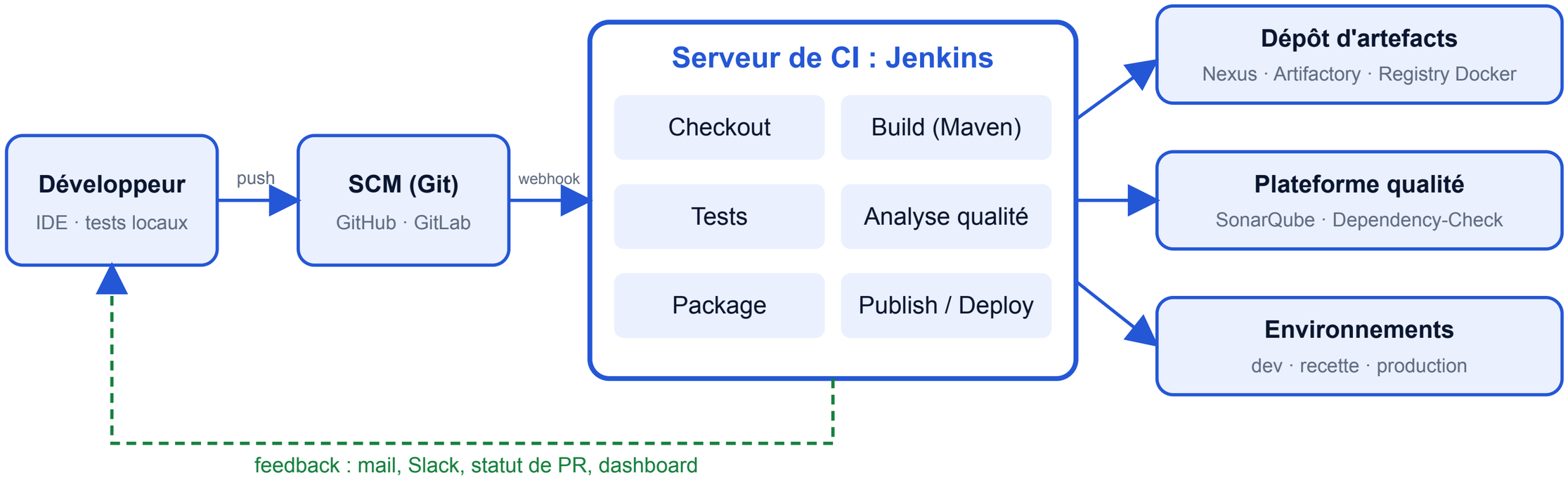
Le serveur Jenkins « animal de compagnie »
Un unique controller qui construit tout lui-même, avec 150 plugins, jamais mis à jour, sans sauvegarde.

Les branches longues
Une branche `feature/refonte` de 3 mois. L'intégration finale est un projet en soi.

Les secrets dans le Jenkinsfile
`sh "docker login -u admin -p S3cret"` – visible dans Git et dans la console.

Le build qui dépend du réseau externe sans cache
Chaque build retélécharge 300 Mo de dépendances ; une panne de Maven Central bloque l'équipe.

La chaîne de fabrication logicielle : vue d'ensemble



Les étapes de la chaîne de fabrication

Étape	Rôle	Outils (écosystème Java)	Dans Jenkins
Checkout	Récupérer les sources à une révision précise (commit SHA)	Git	Git plugin, `checkout scm`
Build	Compiler, résoudre les dépendances	Maven, Gradle, JDK	Tool Maven, `sh 'mvn …'`
Tests	Unitaires puis intégration ; produire des rapports XML	JUnit 5, Mockito, Testcontainers, Surefire, Failsafe	`junit` step
Analyse	Qualité, couverture, sécurité, dépendances	SonarQube, JaCoCo, SpotBugs, OWASP DC, Trivy	Plugins Sonar, Warnings NG, Coverage
Package	Produire l'artefact livrable, versionné	Maven (jar/war), Jib / Docker (image)	`archiveArtifacts`
Publish	Stocker l'artefact dans un dépôt immuable	Nexus, Artifactory, GitLab Registry, Harbor	`mvn deploy`, `docker push`
Deploy	Installer l'artefact dans un environnement	SSH, Ansible, Docker, Helm/Kubernetes, Terraform	`sh`, plugins SSH/K8s, `input`
Feedback	Informer les acteurs de l'état de la chaîne	SMTP, Slack, Teams, statut GitHub/GitLab	`post { }`, Email Ext

Chaque étape est une **porte** : elle ne s'exécute que si la précédente a réussi. C'est le principe du *pipeline*.

CI, Continuous Delivery, Continuous Deployment



Intégration continue

Chaque commit est construit et testé automatiquement. Objectif : un `main` toujours sain. **Sortie :** un artefact validé.

Livraison continue

Chaque artefact validé est **déployable en production** à tout moment, via un pipeline automatisé jusqu'à la recette. La mise en production reste une **décision humaine** (bouton, `input`).

Déploiement continu

Tout commit qui passe le pipeline est déployé en production **sans intervention**. Exige une couverture de tests très élevée, des feature flags, du monitoring et un rollback automatisé.

Jenkins couvre les trois niveaux : la différence tient au périmètre du pipeline et à la présence (ou non) d'une étape d'approbation manuelle.

L'écosystème des outils de CI

Outil	Modèle	Points forts
Jenkins	Open source, auto-hébergé	Universel, plus de 2 000 plugins, Pipeline as Code, total contrôle, on-premise / air-gapped
GitLab CI	Intégré à GitLab	YAML simple, runners, lié au dépôt
GitHub Actions	Intégré à GitHub, SaaS	Marketplace d'actions, gratuit pour l'open source
Azure DevOps	SaaS / on-prem Microsoft	Écosystème Microsoft, boards + repos + pipelines
Tekton / Argo Workflows	Kubernetes natif	Pipelines déclaratifs en CRD, cloud-native
CircleCI, Travis CI	SaaS	Simplicité (orbs chez CircleCI)
Drone, Woodpecker	Open source, auto-hébergé	Pipelines YAML exécutés en conteneurs

Quand choisir Jenkins ?

- Besoin d'un serveur **indépendant du fournisseur Git**
- Contraintes d'hébergement : on-premise, réseau isolé, souveraineté
- Besoins d'intégration variés (legacy, outils propriétaires, matériel)
- Équipe capable d'administrer la plateforme (mises à jour, plugins, agents)

Points de vigilance

- Maintenance des plugins et de leurs dépendances
- Nécessité de discipliner l'usage (Pipeline as Code, JCasC)
- Courbe d'apprentissage de Groovy pour les pipelines avancés

02

Utilisation de Jenkins

Concepts et définitions · Serveur de build · Archétypes de projet · Déclencheurs · Résultat du build · Workspace · Interface · IDE · Installation · Configuration · Plugins

Qu'est-ce que Jenkins ?

Carte d'identité

- **Serveur d'automatisation open source** (licence MIT), écrit en Java
- Né en 2004 sous le nom **Hudson** (Kohsuke Kawaguchi, Sun Microsystems) ; en 2011, après un conflit avec Oracle (qui avait racheté Sun) sur la marque, la communauté poursuit le projet sous le nom Jenkins (Hudson continue à part, puis est abandonné)
- Projet hébergé par la **Continuous Delivery Foundation** (Linux Foundation)
- Deux lignes de versions : **LTS** (une nouvelle base toutes les 12 semaines, correctifs entre) et **weekly**
- Extensible par **plugins** (plus de 2 000 dans l'Update Center)

Ce que Jenkins fait

- Surveille des sources (Git) et des événements (webhooks, horloge)
- Exécute des **jobs** : séquences d'étapes (shell, Maven, scripts, plugins) sur des agents
- Collecte résultats, rapports, artefacts ; historise et notifie
- Orchestre des pipelines complets : CI, CD, tâches d'exploitation

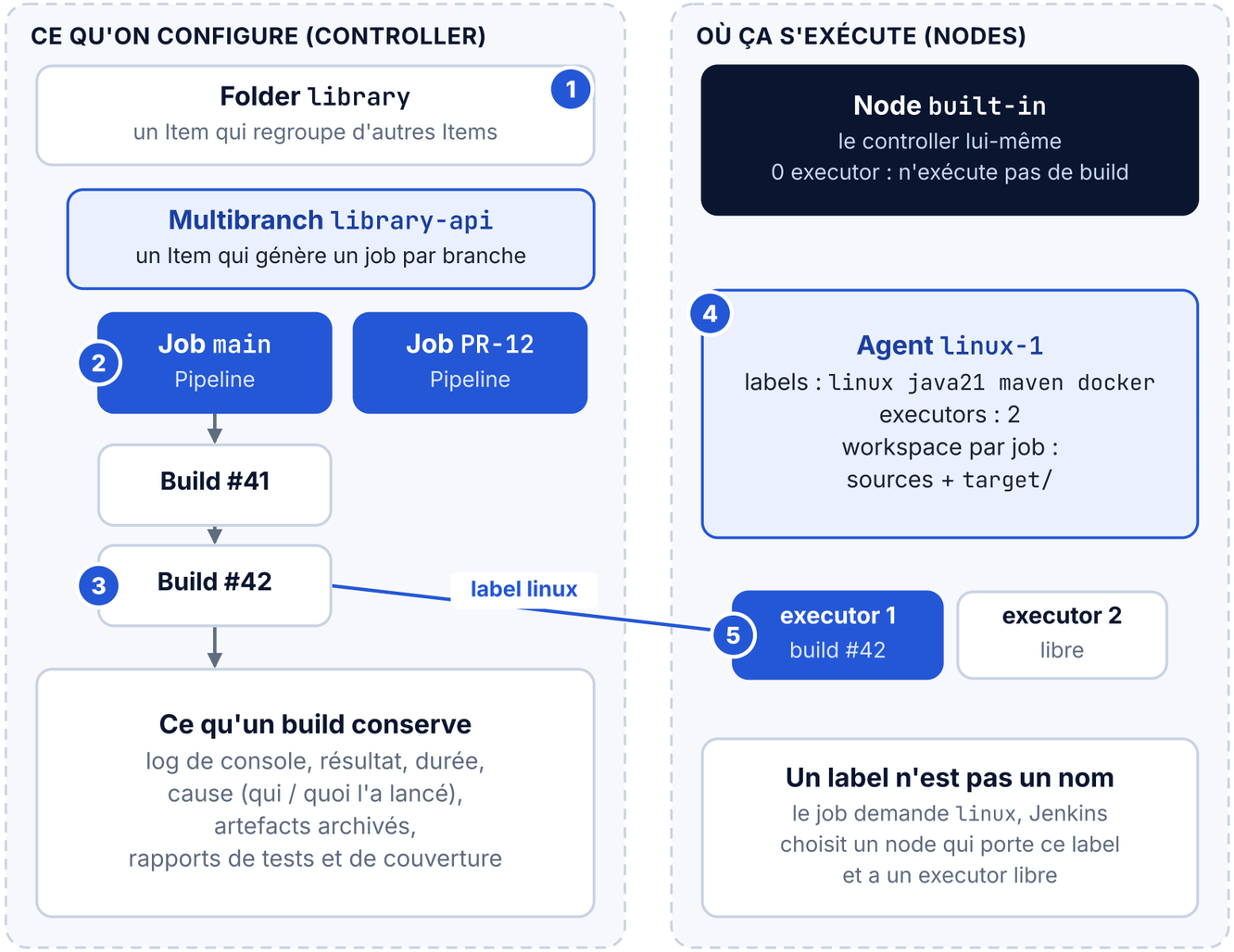
Ce que Jenkins ne fait pas

- Il ne compile pas, ne teste pas lui-même : il **appelle** Maven, Gradle, npm, Docker…
- Il ne stocke pas les artefacts à long terme : c'est le rôle de Nexus/Artifactory
- Il n'est pas un SCM : Git reste la source de vérité

Concepts et définitions

Terme	Définition
Controller (ex-master)	Le processus Jenkins central : interface web, configuration, ordonnancement des builds, stockage de l'historique dans `JENKINS_HOME`.
Node	Toute machine capable d'exécuter des jobs : le controller lui-même (le node `built-in`) et les agents.
Agent	Node distinct du controller (VM, conteneur, pod) qui s'y connecte et exécute les builds qu'il lui confie.
Executor	Un « slot » d'exécution sur un node. Un node avec 4 executors peut exécuter 4 builds en parallèle.
Label	Étiquette attachée à un node (`linux`, `docker`, `java21`) ; un job demande un label, Jenkins choisit un node qui le porte.
Item	Tout ce qu'on crée via *New Item* : un job, ou un conteneur qui ne lance pas de build lui-même et regroupe ou génère des jobs (Folder, Multibranch Pipeline, Organization Folder).
Job (ex-*Project*)	Description configurée d'un travail que Jenkins exécute : Freestyle, Pipeline… « Project » est l'ancien terme, encore visible dans l'UI (*Freestyle project*).
Build	Une exécution numérotée d'un job (#1, #2, …) avec son log, son résultat, ses artefacts. Dans l'API Java, l'objet s'appelle `Run` ; en Pipeline, `currentBuild` en donne une vue (`RunWrapper`).
Workspace	Répertoire de travail sur le node (en pratique un agent) où les sources sont extraites et le build exécuté.
Artifact	Fichier produit par un build et archivé sur le controller avec `archiveArtifacts` (JAR, WAR, rapport…). Le log de console est conservé à part.
Trigger	Événement qui déclenche un build : commit, webhook, horloge, build amont, appel manuel.
Plugin	Extension packagée (`.hpi`) qui ajoute des steps, des intégrations, des types de jobs.
Credential	Secret stocké chiffré dans Jenkins (mot de passe, token, clé SSH, certificat) et injecté dans les builds sans apparaître en clair.

Schéma : comment les concepts s'emboîtent



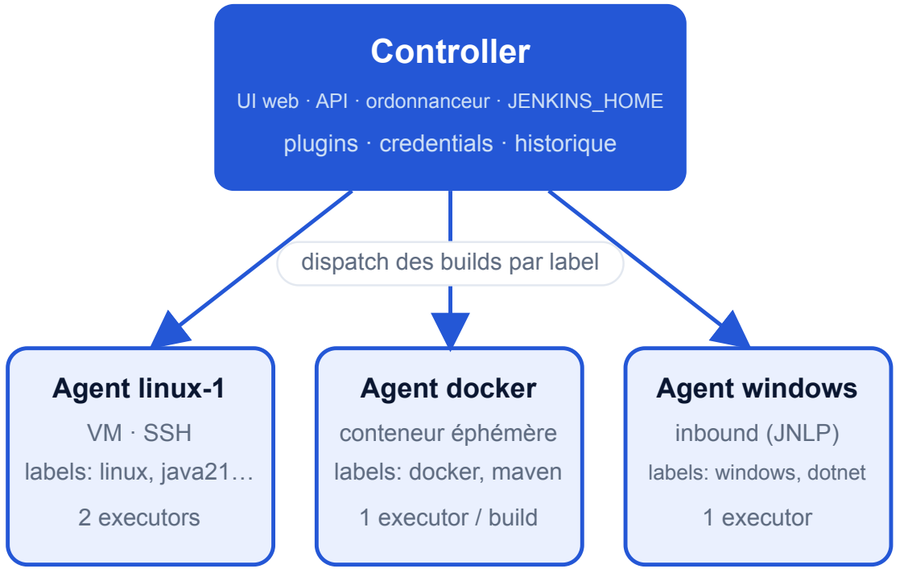
Comment le lire

1. Un **Item** est tout ce qu'on crée par *New Item*. Un Folder ou un Multibranch Pipeline sont des Items qui contiennent ou génèrent des jobs, sans builder eux-mêmes.
2. Un **job** décrit un travail : ici un Pipeline par branche (`main`, `PR-12`).
3. Chaque exécution d'un job est un **build** numéroté, qui garde son log, son résultat et ses rapports sur le controller.
4. Les **nodes** exécutent : le controller (node `built-in`, sans executor en production) et les **agents**, décrits par leurs labels.
5. Le build #42 demande le label `linux` : il occupe un **executor** libre de `linux-1` et travaille dans le **workspace** du job sur cet agent.

À retenir

Le controller garde la **configuration et l'historique** ; les agents font le **travail**.

Architecture : controller et agents



Principe

- Le **controller** ne devrait **pas exécuter de builds** (0 executor sur le node built-in) : sécurité et performance
- Les **agents** portent les outils (JDK, Maven, Docker) et exécutent les steps
- L'agent exécute le jar Remoting : `remoting.jar` copié par le controller en SSH, `agent.jar` téléchargé par l'agent en inbound (même programme). Le canal est chiffré en SSH et en inbound TCP (TLS) ; en WebSocket, seulement si Jenkins est servi en HTTPS

Modes de connexion

- **SSH** : le controller se connecte à l'agent (le plus courant sous Linux)
- **Inbound** (JNLP/WebSocket) : l'agent se connecte au controller (Windows, agents derrière NAT)
- **Cloud** : agents créés à la demande – Docker, Kubernetes, EC2, Azure VM – puis détruits

Scalabilité
Ajouter des agents augmente la capacité de build sans toucher au controller.

Jenkins comme serveur de build

Cycle de vie d'un build

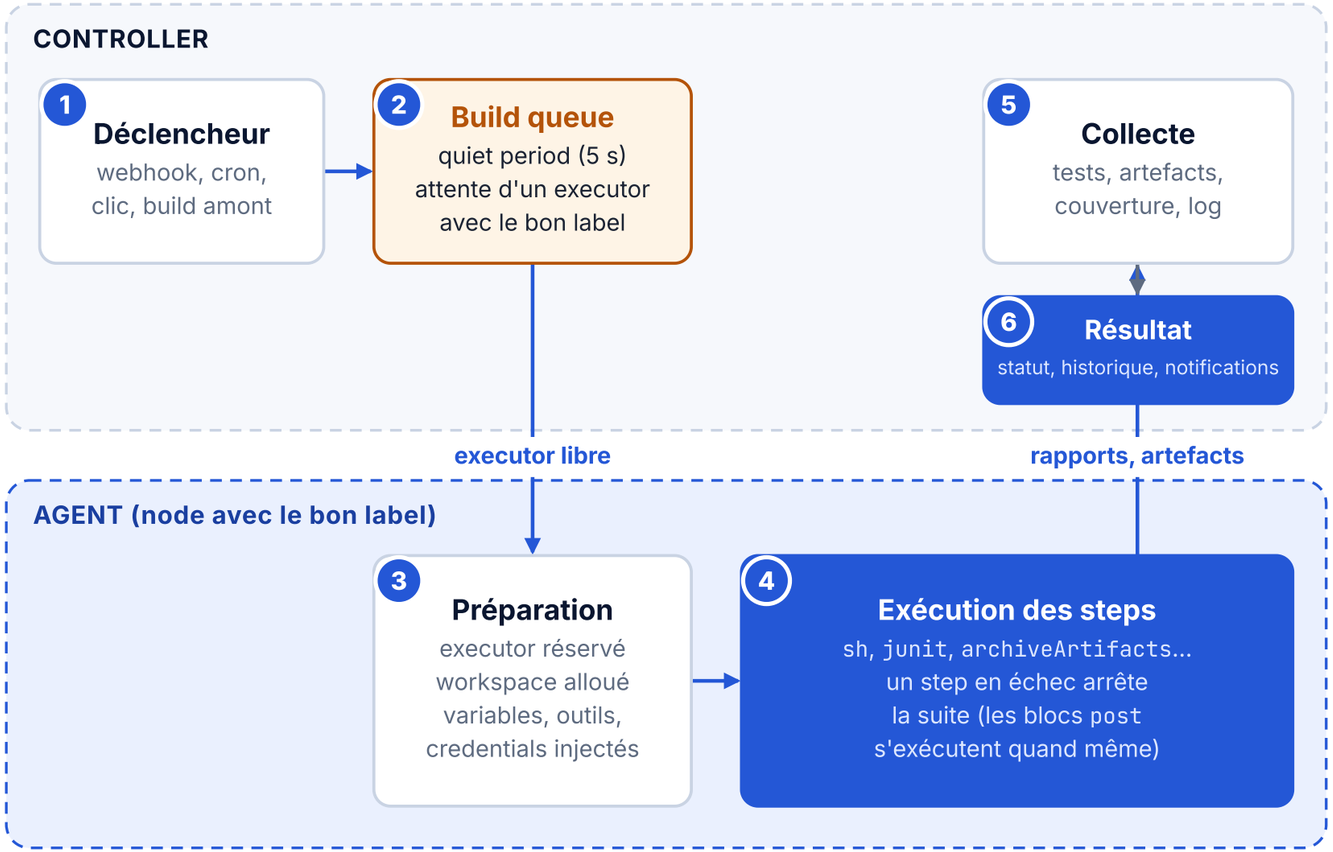
1. **Déclenchement** : un trigger place le build dans la **build queue**
2. **Ordonnancement** : Jenkins attend un executor libre portant le bon label (*quiet period* de 5 s par défaut pour regrouper les commits)
3. **Préparation** : allocation d'un workspace sur l'agent, injection des variables d'environnement et des credentials
4. **Exécution** : les steps s'enchaînent ; un step en échec stoppe la suite (sauf `post`)
5. **Collecte** : rapports de tests, artefacts, couverture sont rapatriés sur le controller
6. **Résultat** : statut calculé, historique mis à jour, notifications envoyées

Variables d'environnement fournies

Variable	Contenu
`BUILD_NUMBER`	Numéro du build (ex. `42`)
`BUILD_ID`	Identique à `BUILD_NUMBER`
`BUILD_TAG`	`jenkins-<job>-<n>` : unique sur le controller
`JOB_NAME`	Nom du job (ex. `library-api/main`)
`BUILD_URL`, `JOB_URL`	URLs vers le build / le job
`WORKSPACE`	Chemin du workspace sur l'agent
`GIT_COMMIT`, `GIT_BRANCH`	Révision et branche extraites
`NODE_NAME`, `EXECUTOR_NUMBER`	Où s'exécute le build
`JENKINS_URL`	URL racine du controller

Liste complète : `<JENKINS_URL>/env-vars.html`

Schéma : le parcours d'un build, du déclencheur au résultat



Pendant l'attente en queue, le build n'occupe aucun executor ; une fois parti, il garde son executor jusqu'à la fin (sauf un stage sans agent dans un pipeline `agent none`).

Comment le lire

1. Un **déclencheur** crée une demande de build : push (webhook), horloge, clic, fin d'un job amont.
2. La demande entre dans la **build queue**. Jenkins attend la quiet period, puis un executor libre sur un node qui porte le label demandé.
3. Sur l'agent, Jenkins **prépare** : executor réservé, workspace, variables (`BUILD_NUMBER`, `WORKSPACE`…), outils et credentials.
4. Les **steps** s'exécutent dans l'ordre. Le premier step en erreur arrête la suite ; les blocs `post` tournent malgré tout.
5. Les rapports et artefacts sont **rapatriés sur le controller**, qui les conserve avec le build.
6. Le controller calcule le **statut**, met à jour l'historique et notifie.

À retenir

Un build « bloqué » est souvent un build **en queue** : lire le message de la queue (label absent, agents hors ligne, executors occupés).

Archétypes de projet (types d'items)

Type	Usage	Quand l'utiliser
Freestyle project	Job configuré via l'interface : SCM, triggers, build steps (shell, Maven…), post-build actions. Historique, simple.	Tâches simples, découverte, scripts d'exploitation. À éviter pour la CI d'applications.
Pipeline	Job défini par un `Jenkinsfile` (Groovy DSL), saisi dans l'UI ou lu depuis le SCM. Stages, parallélisme, reprise, visualisation.	Le standard pour toute CI/CD. Un pipeline par dépôt.
Multibranch Pipeline	Scanne un dépôt, crée automatiquement un sous-job par branche et par Pull Request contenant un `Jenkinsfile` ; supprime les jobs des branches effacées.	Projets avec branches et PR : le choix par défaut avec Git.
Organization Folder	Scanne une organisation GitHub / un groupe GitLab et crée un Multibranch Pipeline par dépôt.	Dizaines de dépôts avec la même convention.
Folder	Conteneur hiérarchique : regroupe des jobs, porte ses propres credentials et habilitations.	Organisation par équipe / produit ; isolation des droits.
Multi-configuration (Matrix)	Exécute le même job pour une combinaison d'axes (OS × JDK × navigateur).	Legacy ; préférer `matrix` dans un Pipeline déclaratif.
External Job	Suivi d'un processus exécuté hors Jenkins.	Rare.

Freestyle vs Pipeline

Freestyle project

- Configuration **cliquée**, stockée dans `config.xml` sur le controller
- Build linéaire : liste de *build steps* puis de *post-build actions*
- Pas de stages, pas de parallélisme natif, pas de reprise après redémarrage
- Enchaînements via « Build other projects » : fragile, peu lisible
- Non versionné, non revu, difficile à dupliquer

Verdict
Utile pour comprendre les notions de base et pour de petits scripts d'exploitation. Pas pour une chaîne CI/CD d'application.

Pipeline (Jenkinsfile)

- Définition en **code** (Groovy DSL), versionnée avec le projet
- **Stages** nommés, visualisés ; **parallélisme** ; **matrix** ; **agents** par stage
- **Durable** : survit à un redémarrage du controller
- Gestion des conditions (`when`), des entrées (`input`), des erreurs (`post`)
- Factorisation via **Shared Libraries**
- Même pipeline pour toutes les branches via Multibranch

Verdict
Le standard. Dans cette formation, chaque fonctionnalité est montrée en Freestyle (pour comprendre l'UI) puis en Pipeline (pour la pratique réelle).

Qu'est-ce qu'un Jenkinsfile ?

- Un **fichier texte** nommé `Jenkinsfile`, placé à la **racine du dépôt**, à côté du `pom.xml`
- Il décrit **toute la chaîne** du projet : où construire, quelles étapes, quoi publier, qui prévenir
- Écrit dans un **DSL** (langage dédié) bâti sur **Groovy** : `pipeline`, `stage`, `sh`… sont des mots du DSL
- Versionné, relu et branché **comme le code** : une branche peut faire évoluer son pipeline sans gêner les autres
- Le job Jenkins ne contient plus que **l'adresse du dépôt** et le chemin du fichier (*Script Path*)

Pipeline as Code

La configuration de la CI quitte l'interface (et le `config.xml` du controller) pour rejoindre le dépôt : on la retrouve, la compare et la restaure avec Git.

Deux façons de le fournir

Definition du job	Où vit le script	Usage
Pipeline script	Saisi dans l'éditeur du job, stocké dans son `config.xml`	Essais, mise au point (démo 9)
Pipeline script from SCM	Fichier du dépôt, relu à chaque build	La pratique normale
Multibranch Pipeline	Fichier de **chaque branche**	Projets Git avec branches et PR

Ce qu'il n'est pas

- Pas un script shell : les commandes système passent par le step `sh` (ou `bat`)
- Pas un remplaçant de Maven ou Gradle : il **orchestre** l'outil de build, il ne le refait pas

Comment Jenkins exécute un Jenkinsfile

1. **Déclenchement** : clic, webhook, cron… Le job sait où lire le fichier.
2. **Lecture** : le controller récupère le `Jenkinsfile` du commit à construire (*Obtained Jenkinsfile from git …*).
3. **Validation** : la syntaxe déclarative est vérifiée **avant** tout stage. Une faute = échec immédiat, avec la ligne fautive.
4. **Exécution** : le **controller** déroule le script ; chaque step est envoyé là où il doit s'exécuter.
5. **Traces** : chaque stage et chaque step sont enregistrés (Pipeline Overview, Stages, console).

Deux machines, deux rôles

Le **Groovy** du Jenkinsfile (variables, `if`, boucles) s'exécute sur le **controller**. Les **steps** qui touchent aux fichiers ou lancent des commandes (`sh`, `readFile`, `junit`) s'exécutent sur l'**agent**, dans le workspace.

Conséquences pratiques

Constat	Pourquoi
Le build reprend après un redémarrage du controller	L'état du script est **sérialisé** à chaque step (moteur CPS) : pipeline *durable*
`new File('pom.xml')` ne trouve pas le fichier	Ce Groovy lit le disque du **controller** ; utiliser le step `readFile`, qui lit le workspace
Une boucle Groovy lourde ralentit tout Jenkins	Elle consomme le controller, partagé par tous les builds : le calcul va dans `sh`
Certaines méthodes Groovy sont refusées	Le script tourne dans un **bac à sable** (Script Security)

Règle : peu de Groovy, beaucoup de steps. La logique de build reste dans Maven / Gradle.

Pipeline : déclaratif vs scripté

Declarative Pipeline

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn -B clean package'
      }
    }
  }
}
```

- Structure imposée : `pipeline → agent → stages → stage → steps`
- Validation syntaxique avant exécution, messages d'erreur clairs
- Sections dédiées : `environment`, `options`, `parameters`, `triggers`, `tools`, `post`, `when`
- **Recommandé** : lisible, maintenable, suffisant dans la grande majorité des cas

Scripted Pipeline

```
node {
  stage('Build') {
    checkout scm
    sh 'mvn -B clean package'
  }
}
```

- Groovy quasi libre : boucles, conditions, fonctions, try/catch
- Plus puissant, mais plus difficile à lire et à sécuriser (sandbox Groovy)
- Utile pour des cas dynamiques (génération de stages) ou dans les Shared Libraries

Mixte

Un bloc `script { }` dans un pipeline déclaratif permet d'insérer du Groovy scripté ponctuellement.

Anatomie d'un Jenkinsfile déclaratif

```
pipeline {
  agent { label 'linux' }            // où exécuter (any, none, label, docker, kubernetes)

  options {
    timeout(time: 20, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
    disableConcurrentBuilds()
  }

  environment {
    APP_NAME = 'library-api'
    MAVEN_OPTS = '-Dmaven.repo.local=/var/cache/m2'
  }

  tools { jdk 'jdk21'; maven 'maven-3.9' }

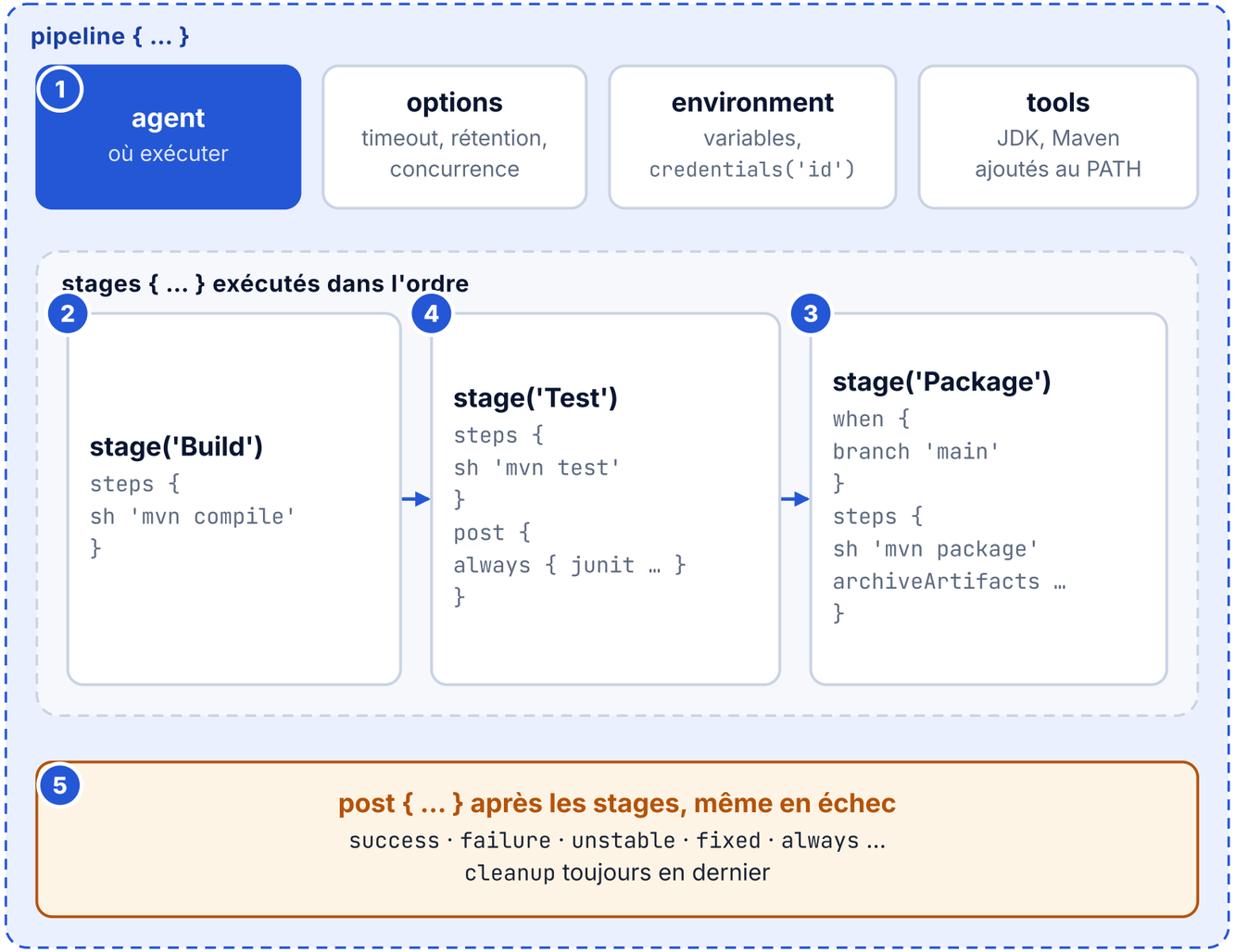
  stages {
    // pas de stage Checkout : un Jenkinsfile lu depuis le SCM est extrait automatiquement
    stage('Build')    { steps { sh 'mvn -B clean compile' } }
    stage('Test') {
      steps { sh 'mvn -B test' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Package') {
      when { branch 'main' }           // BRANCH_NAME : job Multibranch uniquement
      steps {
        sh 'mvn -B -DskipTests package'
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
      }
    }
  }

  post {
    success { echo "Build ${env.BUILD_NUMBER} OK" }
    failure { mail to: 'team@example.com', subject: "KO ${env.JOB_NAME}", body: env.BUILD_URL }
    always  { cleanWs() }
  }
}
```

Section	Rôle
`agent`	Sur quel node/conteneur tourne le pipeline (ou chaque stage)
`options`	Timeout, rétention, concurrence, timestamps, retry
`environment`	Variables globales ou par stage ; `credentials('id')` injecte un secret
`tools`	Outils déclarés dans *Manage Jenkins → Tools*, ajoutés au `PATH`
`parameters`	Paramètres saisis au lancement (string, choice, boolean…)
`triggers`	Déclencheurs : `cron`, `pollSCM`, `upstream`
`stages` / `stage`	Étapes visualisées ; `parallel` et `matrix` possibles
`steps`	Actions : `sh`, `bat`, `junit`, `archiveArtifacts`, `withCredentials`…
`when`	Condition d'exécution d'un stage (branche, paramètre, expression)
`post`	`always`, `success`, `failure`, `unstable`, `changed`, `cleanup`

Référence : `<JENKINS_URL>/pipeline-syntax/` (Snippet Generator et Declarative Directive Generator).

Schéma : la structure d'un Jenkinsfile déclaratif



Comment le lire

1. Les **directives globales** s'appliquent à tout le pipeline : `agent`, `options`, `environment`, `tools` (et `parameters`, `triggers`).
2. `stages` contient les étapes, exécutées **dans l'ordre** ; chacune contient ses `steps`.
3. `when` est évalué à l'entrée du stage (après l'allocation de son agent, sauf `beforeAgent true`) : s'il est faux, le stage est sauté (« skipped »).
4. Un stage peut avoir son propre `post` : ici, les rapports JUnit sont publiés même si les tests échouent.
5. Le `post` global s'exécute après les stages, quel que soit le résultat ; chaque condition (`success`, `failure`…) ne s'exécute que si elle correspond, `cleanup` en dernier.

À retenir

Un stage peut redéfinir `agent`, `environment`, `tools` ou `options` : la directive du stage l'emporte pour ce stage.

Jenkinsfile : où placer quoi

Élément	Sous `pipeline { }`	Sous `stage('…') { }`	Dans `steps { }`
`agent`	oui (obligatoire)	oui	non
`environment`, `tools`, `options`, `post`	oui	oui	non
`parameters`, `triggers`, `libraries`	oui	**non**	non
`when`, `input` (directive)	non	oui	non
`stages`	oui (obligatoire)	oui (stages imbriqués)	non
`parallel`, `matrix`	non	oui (à la place de `steps`)	non
Steps : `sh`, `junit`, `echo` …	non	non	**oui** (et dans `post`)
`script { }` (Groovy libre)	non	non	**oui**

Un seul bloc de chaque directive par niveau : deux `options` sous `pipeline` sont refusés.

Ce que répond Jenkins quand on se trompe

```
stage('Build') { sh 'mvn verify' }
→ Unknown stage section "sh". Starting with
  version 0.5, steps in a stage must be in
  a ‘steps’ block.

stage('Build') { parameters { … } … }
→ Unknown stage section "parameters".

options { timestamps() }
options { timeout(20) }
→ Multiple occurrences of the options section
```

Repère

Directive = structure ou comportement (va sous `pipeline` ou sous un `stage`). **Step** = action (va dans `steps` ou dans une condition de `post`). Messages relevés sur Jenkins 2.580.2.

La directive `options`

Sous `pipeline` : tout le job

Option	Effet
`buildDiscarder(logRotator(…))`	Rétention : `numToKeepStr: '10'` garde les 10 derniers builds (disque du controller)
`timeout(time: 20, unit: 'MINUTES')`	Interrompre le build au-delà de 20 min (ABORTED)
`disableConcurrentBuilds()`	Un seul build à la fois pour ce job (ou cette branche)
`timestamps()`	Heure devant chaque ligne de console
`retry(2)`	Relancer tout le pipeline en cas d'échec
`skipDefaultCheckout()`	Pas de clonage automatique au début
`skipStagesAfterUnstable()`	Arrêter dès que le build devient UNSTABLE
`quietPeriod(30)`	Attendre 30 s avant de démarrer (regroupe les commits)
`disableResume()`	Ne pas reprendre le build après un redémarrage
`parallelsAlwaysFailFast()`	Un échec dans un `parallel` arrête les autres branches

Sous un `stage` : ce stage seulement

```
stage('Tests d\'intégration') {
  options {
    timeout(time: 10, unit: 'MINUTES')
    retry(2)
  }
  steps { sh 'mvn -B verify' }
}
```

Options valides au niveau stage (liste renvoyée par Jenkins) : `timeout`, `retry`, `timestamps`, `skipDefaultCheckout`, `lock`, `catchError`, `warnError`, `withCredentials`, `withEnv`… mais **pas** `buildDiscarder` ni `disableConcurrentBuilds`, qui concernent le job entier.

Minimum conseillé

`buildDiscarder` + `timeout` + `timestamps` sur tout pipeline : disque maîtrisé, aucun build bloqué indéfiniment, logs datés.

Liste complète : *Pipeline Syntax → Declarative Directive Generator → options*. L'unité par défaut de `timeout` est `MINUTES` : le générateur écrit `timeout(20)`.

Variables, paramètres et secrets

```
pipeline {
  agent { label 'linux' }
  parameters {
    choice(name: 'CIBLE', choices: ['recette', 'production'])
    booleanParam(name: 'RUN_IT', defaultValue: true)
    string(name: 'VERSION_MINEURE', defaultValue: '1.4')
  }
  environment {
    APP_VERSION = "${params.VERSION_MINEURE}.${BUILD_NUMBER}"
    NEXUS       = credentials('nexus-deploy')  // NEXUS, NEXUS_USR, NEXUS_PSW
  }
  stages {
    stage('Build') {
      steps {
        echo "Version ${env.APP_VERSION} pour ${params.CIBLE}"
        sh 'mvn -B -Drevision=$APP_VERSION -s settings.xml deploy'
      }
    }
  }
}
```

Source	Accès
Variables de Jenkins	`env.BUILD_NUMBER`, `env.BRANCH_NAME`, `env.BUILD_URL`, `env.WORKSPACE`
`environment { }`	`env.APP_VERSION` en Groovy, `$APP_VERSION` dans `sh`
`parameters { }`	`params.CIBLE` (types : `string`, `text`, `booleanParam`, `choice`, `password`)
`credentials('id')`	Secret masqué (`****`) ; un *Username with password* donne aussi `_USR` et `_PSW`

Guillemets

`'…'` : texte brut, **le shell** lit `$VAR`. `"…${x}"` : **Groovy** remplace avant d'exécuter. Un secret se met toujours entre guillemets simples, sinon : *A secret was passed to "sh" using Groovy String interpolation, which is insecure.*

Les paramètres n'apparaissent dans **Build with Parameters** qu'après un premier build : Jenkins les découvre en exécutant le fichier.

Contrôler le flux : `when`, `post`, `parallel`

`when` : faut-il exécuter ce stage ?

- `branch 'main'`, `buildingTag()`, `tag 'v*'`
- `changeRequest()` : une Pull Request
- `changeset '**/*.java'` : fichiers modifiés
- `environment name: 'CIBLE', value: 'production'`
- `expression { params.RUN_IT }` : du Groovy
- Combinaisons : `allOf`, `anyOf`, `not`
- `beforeAgent true` : évaluer avant d'allouer un agent

Faux : le stage est grisé, *skipped due to when conditional*.

`post` : après le stage ou le pipeline

Condition	Quand
`always`	toujours
`success` / `failure` / `unstable` / `aborted`	selon le résultat
`unsuccessful`	tout sauf succès
`changed`	résultat différent du build précédent
`fixed` / `regression`	retour au vert / passage au rouge
`cleanup`	en tout dernier

`parallel` : en même temps

```
stage('Qualité') {
  failFast true
  parallel {
    stage('Tests') {
      steps { sh 'mvn -B test' }
    }
    stage('Sonar') {
      agent { label 'sonar' }
      steps { sh 'mvn sonar:sonar' }
    }
  }
}
```

`matrix` décline un même stage sur des axes (JDK 17 / 21, OS…).

Le Groovy utile dans un Jenkinsfile

Groovy est un langage de la JVM, proche de Java mais plus concis. Un Jenkinsfile déclaratif en contient peu ; on le rencontre dans `script { }`, `expression { }`, les chaînes et les Shared Libraries.

Notion	Exemple
Chaînes	`'brut'`, `"Build ${env.BUILD_NUMBER}"`, `'''multi-lignes'''`
Variable	`def cible = 'recette'`
Liste, map	`def jdks = ['17', '21']` · `def ports = [recette: 8090, prod: 8091]`
Condition	`if (env.BRANCH_NAME == 'main') { … } else { … }`
Boucle, closure	`jdks.each { v -> echo "JDK ${v}" }`
Appel nommé	`sh script: 'ls', returnStdout: true` = `sh([script: 'ls', returnStdout: true])`

Les steps du DSL sont des méthodes Groovy : `sh 'ls'` s'écrit aussi `sh('ls')`.

Dans un pipeline déclaratif : `script { }`

```
stage('Infos') {
  steps {
    script {
      def commit = sh(script: 'git rev-parse --short HEAD',
                      returnStdout: true).trim()
      currentBuild.displayName = "#${env.BUILD_NUMBER} ${commit}"
      if (currentBuild.previousBuild?.result == 'FAILURE') {
        echo 'Le build précédent avait échoué'
      }
    }
  }
}
```

- `sh(returnStdout: true)` récupère la sortie ; `returnStatus: true` le code retour, sans échouer
- `currentBuild` : résultat, nom affiché, description, builds précédents
- `?.` évite une erreur si l'objet est `null`

Groovy dans Jenkins : bac à sable et bonnes limites

Le bac à sable (Script Security)

- Un Jenkinsfile s'exécute **en sandbox** : seules les méthodes jugées sûres sont permises
- Sinon : `RejectedAccessException: Scripts not permitted to use method …`
- Un administrateur peut approuver la signature dans *Manage Jenkins* → *In-process Script Approval* : à faire rarement, après lecture
- Les **Shared Libraries** globales de confiance tournent hors sandbox : c'est là que va le code avancé (partie 2)

Le moteur CPS

- Pour être durable, Jenkins transforme le Groovy et **sérialise** son état entre les steps
- Un objet non sérialisable gardé dans une variable fait échouer le build (`NotSerializableException`)
- `@NonCPS` marque une méthode utilitaire exécutée d'un bloc, sans step à l'intérieur

Déclaratif d'abord, scripté si nécessaire

Besoin	Solution
Enchaîner build, tests, publication	Déclaratif seul
Une valeur calculée, un nom de build	`script { }` de quelques lignes
Stages générés dynamiquement	Scripted Pipeline ou `matrix`
Même logique dans plusieurs dépôts	**Shared Library** (`vars/*.groovy`)
Calcul, parsing de fichiers lourds	Script dans le dépôt, lancé par `sh`

Signal d'alerte

Un `script { }` de plus de 15 lignes, ou dupliqué dans plusieurs Jenkinsfiles : le sortir dans une Shared Library ou dans un script du projet.

Construire un Jenkinsfile, pas à pas

1. **Partir du minimum** : `pipeline`, `agent`, un stage qui lance l'outil de build (`sh 'mvn -B verify'`). Le faire passer.
2. **Ajouter une brique à la fois** : rapports (`junit`), artefacts, puis conditions, notifications, publication.
3. **Ne pas écrire la syntaxe de mémoire** : *Pipeline Syntax* → *Snippet Generator* pour les steps, *Declarative Directive Generator* pour les directives.
4. **Valider avant de pousser** : `POST /pipeline-model-converter/validate` ou l'extension de l'IDE.
5. **Mettre au point sans commit** : **Replay** rejoue un build avec un script modifié.
6. **Commiter** le fichier ; en Multibranch, chaque branche teste sa version.

Bonnes pratiques

- La logique de build reste dans **Maven / Gradle** : le Jenkinsfile orchestre
- `-B` (batch) pour Maven, `--no-daemon` pour Gradle : sorties lisibles, pas de processus résiduel
- Toujours un `timeout` et un `buildDiscarder`
- Rapports dans un `post { always { } }` **du stage** : publiés même en échec
- Secrets par `credentials()` ou `withCredentials`, jamais en clair
- Noms d'outils, d'agents et de credentials stables (`jdk21` , `linux` , `gitea-token`) : le même fichier marche sur toutes les instances
- Un pipeline par dépôt ; un job de déploiement paramétré à part

Référence : `jenkins.io/doc/book/pipeline/syntax`.

Déclencheurs de build (triggers)

Déclencheur	Fonctionnement	Usage recommandé
Manuel (« Build Now »)	Clic dans l'UI, appel API REST (`POST /job/x/build`), CLI	Tests, jobs d'exploitation, builds paramétrés
Webhook SCM	GitHub/GitLab envoie une requête HTTP au controller à chaque push / PR	**Le standard** : réaction immédiate, aucune charge inutile
Poll SCM	Jenkins interroge le dépôt selon une expression cron ; build si changement	Quand le SCM ne peut pas joindre Jenkins (réseau isolé)
Périodique (cron)	Build à heure fixe, qu'il y ait ou non des changements	Nightly build, tests longs, analyses de sécurité, nettoyage
Upstream (build après un autre projet)	Déclenché à la fin d'un job amont (succès, instable, échec)	Chaînage de jobs ; préférer un seul pipeline
Trigger distant (token)	URL `/job/x/build?token=…` appelée par un script externe	Intégration avec un outil tiers
Multibranch scan	Scan périodique ou par webhook ; découverte de nouvelles branches/PR	Avec Multibranch Pipeline
Generic Webhook (plugin)	Reçoit tout JSON et en extrait des variables	Événements non-SCM (Docker Hub, monitoring…)

Poll SCM vs webhook

Le polling à `* * * * *` sur 200 jobs génère 200 requêtes Git par minute et un délai de feedback d'une minute. Le webhook est instantané et sans charge.

Déclencheurs : syntaxe et expressions cron

Dans un Jenkinsfile déclaratif

```
pipeline {
  agent any
  triggers {
    // Du lundi au vendredi, une fois par nuit, à une heure "hashée" entre 2h00 et 4h59
    cron('H H(2-4) * * 1-5')

    // Polling du SCM toutes les 15 min (si pas de webhook)
    pollSCM('H/15 * * * *')

    // Après le succès du job amont
    upstream(upstreamProjects: 'library-core/main',
             threshold: hudson.model.Result.SUCCESS)
  }
  stages { /* ... */ }
}
```

Attention

Les `triggers` du Jenkinsfile ne sont enregistrés qu'**après** une première exécution du pipeline : lancer un build manuel au départ.

Syntaxe cron Jenkins

```
MINUTE HEURE JOUR_DU_MOIS MOIS JOUR_DE_SEMAINE
  0-59  0-23     1-31      1-12    0-7 (0 et 7 = dimanche)
```

Expression	Signification
`H * * * *`	Une fois par heure, minute répartie
`H/15 * * * *`	Toutes les 15 minutes
`H 2 * * *`	Chaque jour vers 2h
`H H * * 0`	Une fois par semaine, le dimanche
`@daily` · `@weekly` · `@midnight`	`H H * * *` · `H H * * H` · `H H(0-2) * * *` (entre 0h00 et 2h59)
`TZ=Europe/Paris` `H 6 * * 1-5`	Fuseau explicite : `TZ=` seul sur la 1re ligne

Le symbole `H`

Valeur « hachée » à partir du nom du job : stable pour un job donné, mais répartie entre les jobs pour éviter que tout démarre à la même seconde. Toujours préférer `H` à `0`.

Déclencheurs : configurer un webhook Git

Côté Jenkins

1. Installer le plugin **GitHub** (ou **GitLab**)
2. Jenkins doit être joignable depuis le serveur Git (URL publique ou reverse proxy ; en local : tunnel `ngrok` / `smee.io`)
3. Dans le job : cocher *GitHub hook trigger for GITScm polling* ou, pour GitLab, *Build when a change is pushed to GitLab* (note : le plugin fournit un *secret token*)
4. Pour un Multibranch Pipeline : aucun réglage par job, le webhook du dépôt (ou de l'organisation pour un Organization Folder) déclenche le scan

Côté dépôt

- **GitHub** : *Settings* → *Webhooks* → *Add*
  Payload URL : `https://jenkins.example.com/github-webhook/`
  Content type : `application/json`, événements : *push*, *pull request*
- **GitLab** : *Settings* → *Webhooks*
  URL : `https://jenkins.example.com/project/library-api`
  Secret token, événements : *Push*, *Merge request*

Vérification

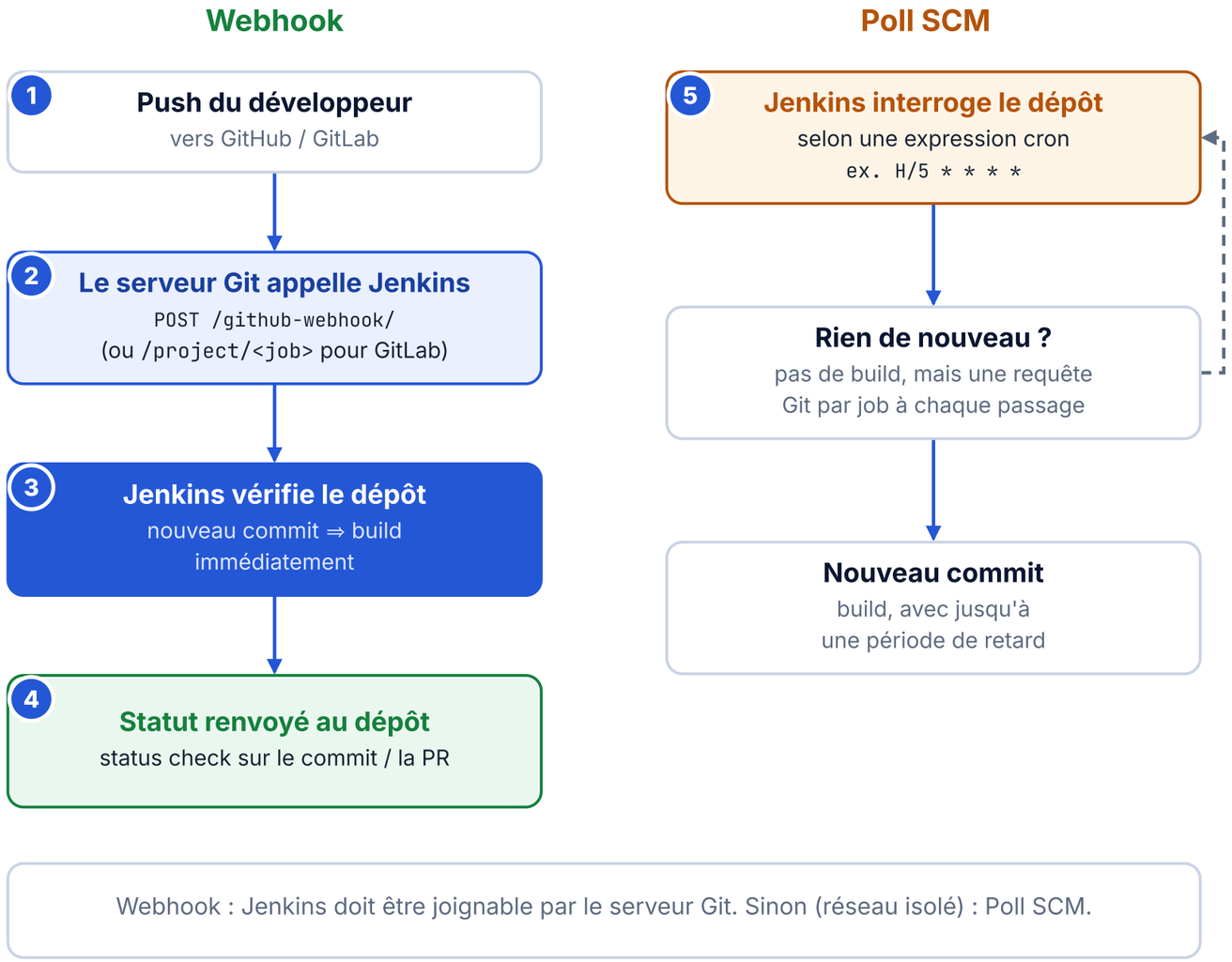
- GitHub/GitLab affichent la dernière livraison du webhook et le code HTTP (200 attendu)
- Dans Jenkins : *Manage Jenkins* → *System Log*, logger `org.jenkinsci.plugins.github` ou `com.dabsquared.gitlabjenkins`
- Un push doit faire apparaître un build dans la seconde

Statut retourné au dépôt

- Les plugins GitHub Branch Source / GitLab Branch Source publient le résultat du build comme **status check** sur le commit / la PR
- Combiné à une règle de protection de branche : impossible de merger une PR rouge

```
// Jenkinsfile : statut explicite (plugin GitLab)
post {
  success { updateGitlabCommitStatus name: 'jenkins', state: 'success' }
  failure { updateGitlabCommitStatus name: 'jenkins', state: 'failed' }
}
```

Schéma : webhook ou polling, qui prévient qui ?



Webhook : Jenkins doit être joignable par le serveur Git. Sinon (réseau isolé) : Poll SCM.

Comment le lire

1. Le développeur pousse un commit.
2. Avec un **webhook**, c'est le serveur Git qui appelle Jenkins, à l'URL fournie par le plugin (`/github-webhook/` , `/project/<job>` pour GitLab).
3. Jenkins vérifie le dépôt des jobs concernés et lance le build tout de suite (pour un Multibranch : un événement sur la branche concernée).
4. Avec un Branch Source (Multibranch GitHub / GitLab), le résultat est renvoyé au dépôt : c'est ce statut que la règle de protection de branche exige.
5. Avec **Poll SCM**, c'est Jenkins qui interroge chaque dépôt selon le cron : une requête par job à chaque passage, et un retard jusqu'à une période.

À retenir
Le webhook est la règle ; le polling est le repli quand le serveur Git ne peut pas joindre Jenkins.

Résultat du build : les statuts

Statut	Signification	Icône
SUCCESS	Toutes les étapes ont réussi, tous les tests passent	Verte, avec une coche (l'API garde le nom historique `color: "blue"` ; le plugin Green Balls est déprécié)
UNSTABLE	Le build s'est terminé mais un indicateur est dégradé : tests en échec collectés par `junit`, seuil de qualité dépassé	Orange (« yellow » dans l'API)
FAILURE	Une étape a échoué (code de retour ≠ 0, exception) : le build s'arrête	Rouge
ABORTED	Interrompu manuellement ou par un `timeout`	Grise
NOT_BUILT	Annulé par un build plus récent (`milestone` , `disableConcurrentBuilds(abortPrevious: true)`), ou module non construit. Un stage sauté par `when` apparaît « skipped » sans changer le résultat du build	Gris clair

UNSTABLE vs FAILURE

`sh 'mvn test'` seul ⇒ un test rouge fait échouer Maven ⇒ **FAILURE**. Avec `mvn -Dmaven.test.failure.ignore=true test` + step `junit` ⇒ **UNSTABLE**, et les rapports sont publiés.

La météo du job

- Icône « weather » sur le dashboard : agrège les **5 derniers builds**
- ☀️ 100 % de succès → ⛅ → 🌧️ → ⛈️ 0 % de succès
- Lecture instantanée de la santé d'un projet

Changer le statut depuis le pipeline

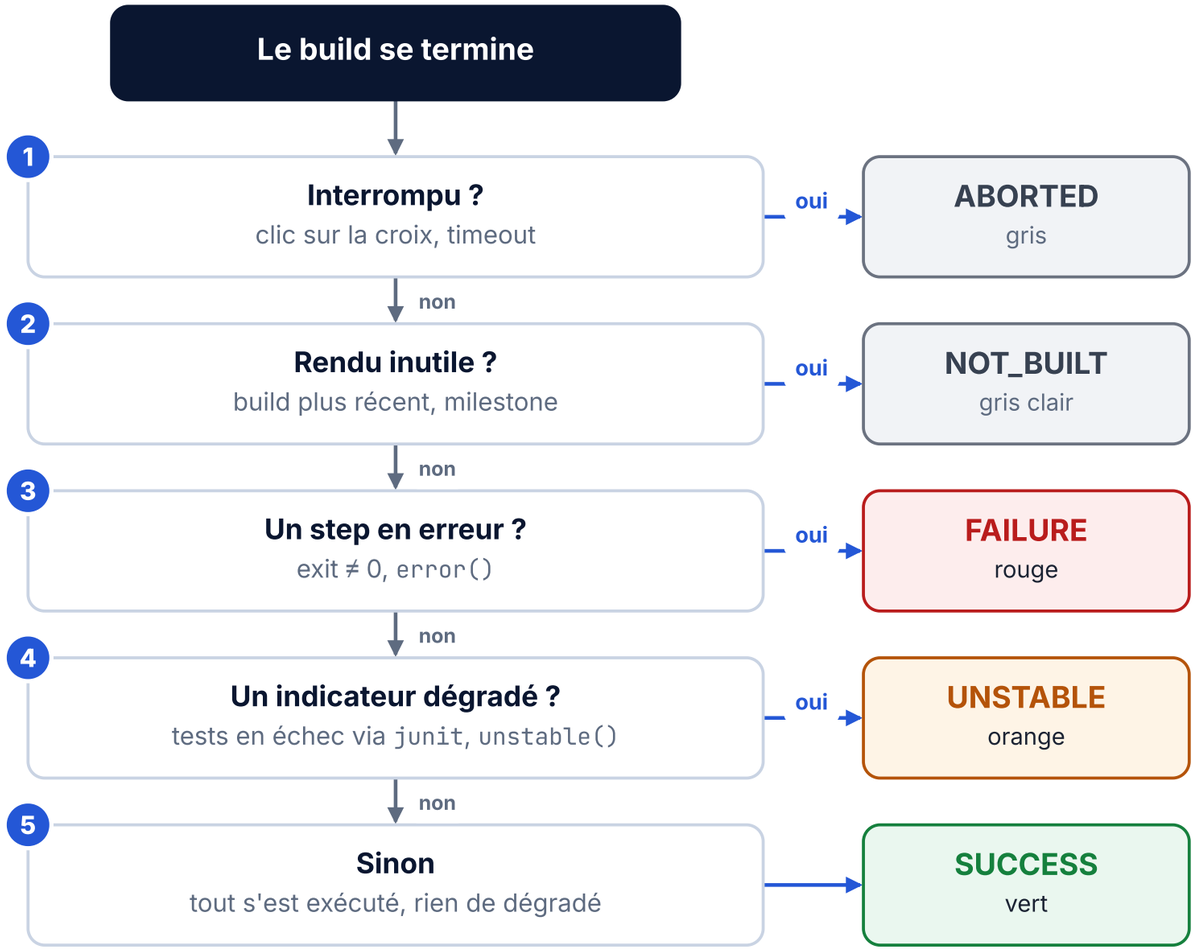
```
// Marquer instable sans arrêter
unstable(message: 'Couverture sous le seuil')

// Échouer explicitement
error('Image Docker non signée')

// Ignorer un échec de step
catchError(buildResult: 'UNSTABLE', stageResult: 'FAILURE') {
  sh 'mvn dependency-check:check'
}

// Lire le résultat courant
echo "Résultat : ${currentBuild.currentResult}"
```

Schéma : comment lire le statut d'un build



On lit les questions de haut en bas : la première réponse « oui » donne le statut.

Comment le lire

1. **ABORTED** : quelqu'un a cliqué sur la croix, ou un `timeout` a interrompu le build.
2. **NOT_BUILT** : un build plus récent l'a rendu inutile (`milestone`, `disableConcurrentBuilds(abortPrevious: true)`).
3. **FAILURE** : un step a échoué (code de retour non nul, `error()`) ; les stages suivants ne s'exécutent pas.
4. **UNSTABLE** : tout s'est exécuté, mais un indicateur est dégradé : tests en échec publiés par `junit`, seuil de qualité, `unstable()`.
5. **SUCCESS** : aucune des situations précédentes.

À retenir

Pour obtenir UNSTABLE plutôt que FAILURE sur des tests rouges, Maven ne doit pas échouer (`-Dmaven.test.failure.ignore=true`) : c'est `junit` qui juge.

Résultat du build : console, historique, artefacts

La page d'un build (`/job/x/42/`)

- **Status** : résultat, durée, déclencheur (« Started by user… », « Started by GitHub push… »), changements (commits inclus)
- **Console Output** : log complet et en temps réel de toutes les commandes ; `timestamps` ajoute l'heure sur chaque ligne
- **Changes** : liste des commits depuis le build précédent (auteur, message, fichiers)
- **Test Result** : résumé JUnit, tests échoués avec stack trace, durée
- **Artifacts** : fichiers archivés téléchargeables
- **Stages** (plugin Pipeline Graph View, proposé par l'assistant d'installation ; « Pipeline Overview » dans les versions plus anciennes) : durée de chaque stage, log par step ; **Stage View** si ce plugin-là est installé
- **Replay** : relancer en modifiant le Jenkinsfile à la volée (debug)
- **Restart from Stage** : reprendre un pipeline à un stage donné

Historique et rétention

- Chaque build conserve son log, ses résultats et ses artefacts dans `$JENKINS_HOME/jobs/<job>/builds/<n>/`
- Sans politique de rétention, le disque se remplit : toujours définir un `buildDiscarder`
- « Keep this build forever » protège un build de référence

```
options {
  buildDiscarder(logRotator(
    numToKeepStr: '30',           // au plus 30 builds
    daysToKeepStr: '60',          // et aucun de plus de 60 jours
    artifactNumToKeepStr: '10'))  // artefacts : 10 derniers seulement
}
```

Artefacts et fingerprints

```
archiveArtifacts artifacts: 'target/*.jar, target/site/**',
                 fingerprint: true, onlyIfSuccessful: true
```

Le **fingerprint** (MD5) permet de tracer un artefact d'un job à l'autre : quel build de `library-api` a été déployé par quel build de `deploy-prod` ?

Le workspace

Définition

- Répertoire sur l'**agent** dans lequel un build s'exécute : checkout des sources, compilation, fichiers temporaires
- Emplacement par défaut : `<agent root>/workspace/<nom du job>`
  ex. `/home/jenkins/workspace/library-api_main`
- Accessible dans le pipeline via `env.WORKSPACE` et `pwd()`
- Consultable dans l'UI : Freestyle : *Job* → *Workspace* ; Pipeline : *Build* → *Workspaces* (un par bloc `node`)

Comportement

- Le workspace est **réutilisé** d'un build à l'autre sur le même agent (le checkout Git fait un `fetch` incrémental)
- Des builds concurrents du même job obtiennent des workspaces suffixés `@2`, `@3`
- Un workspace peut contenir des restes du build précédent : fichiers générés, `target/`, dépendances
- Un agent éphémère (Docker, pod) démarre toujours avec un workspace vide

Nettoyer ou pas ?

Pour la reproductibilité

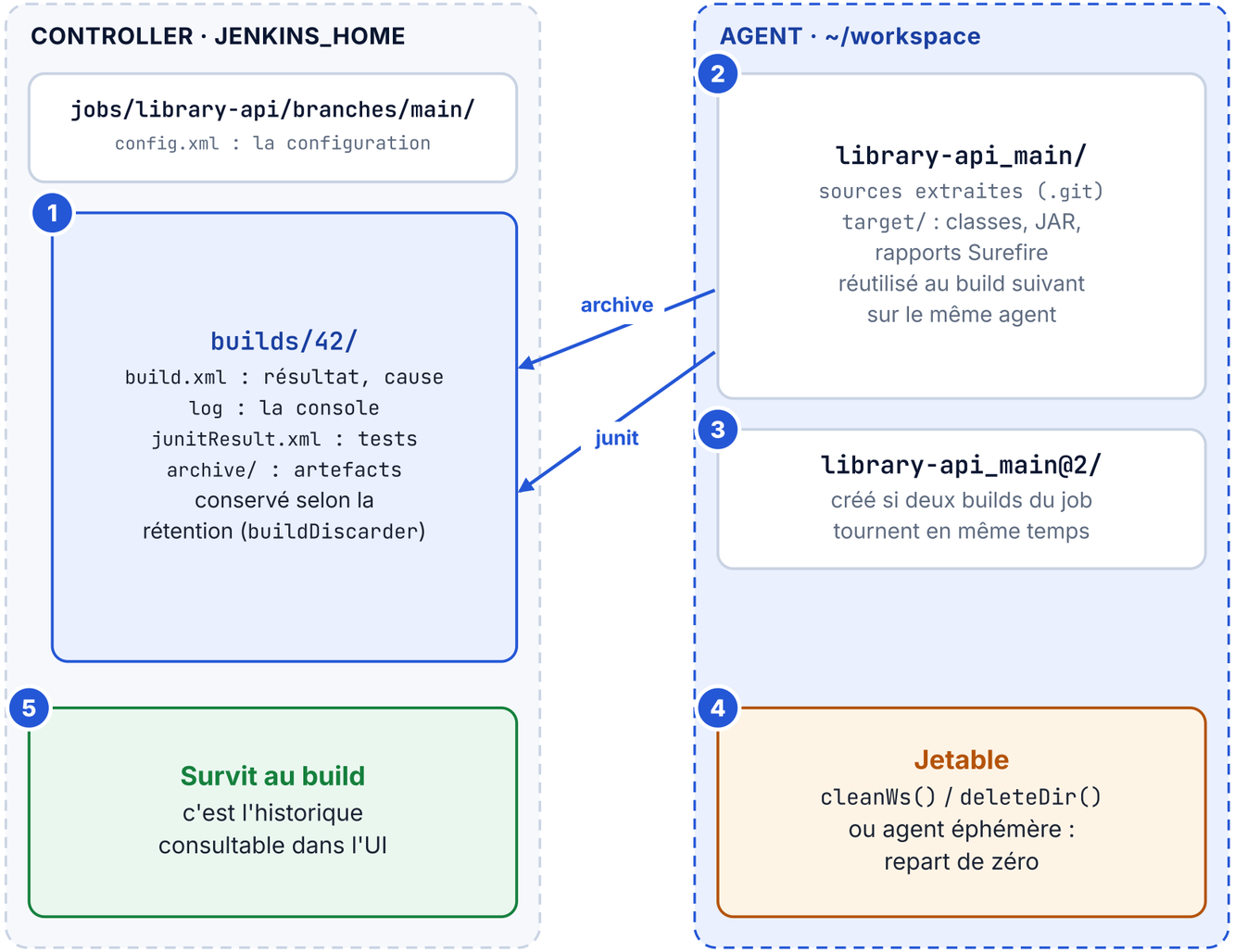
Nettoyer en fin de build (`cleanWs()` , plugin Workspace Cleanup) ou au début (`deleteDir()`). Garantit qu'un build ne dépend pas d'un fichier oublié.

Pour la performance

Conserver les caches de dépendances **hors** du workspace (`~/.m2` sur l'agent ou volume Docker) plutôt que de garder un workspace sale.

```
pipeline {
  agent { label 'linux' }
  options { skipDefaultCheckout() }      // ne pas extraire automatiquement
  stages {
    stage('Checkout') {
      steps {
        deleteDir()                      // workspace vierge
        checkout scm
      }
    }
    stage('Build') {
      steps {
        dir('backend') {                 // sous-répertoire du workspace
          sh 'mvn -B package'
        }
      }
    }
  }
  post { cleanup { cleanWs() } }         // toujours exécuté en dernier
}
```

Schéma : workspace sur l'agent, historique sur le controller

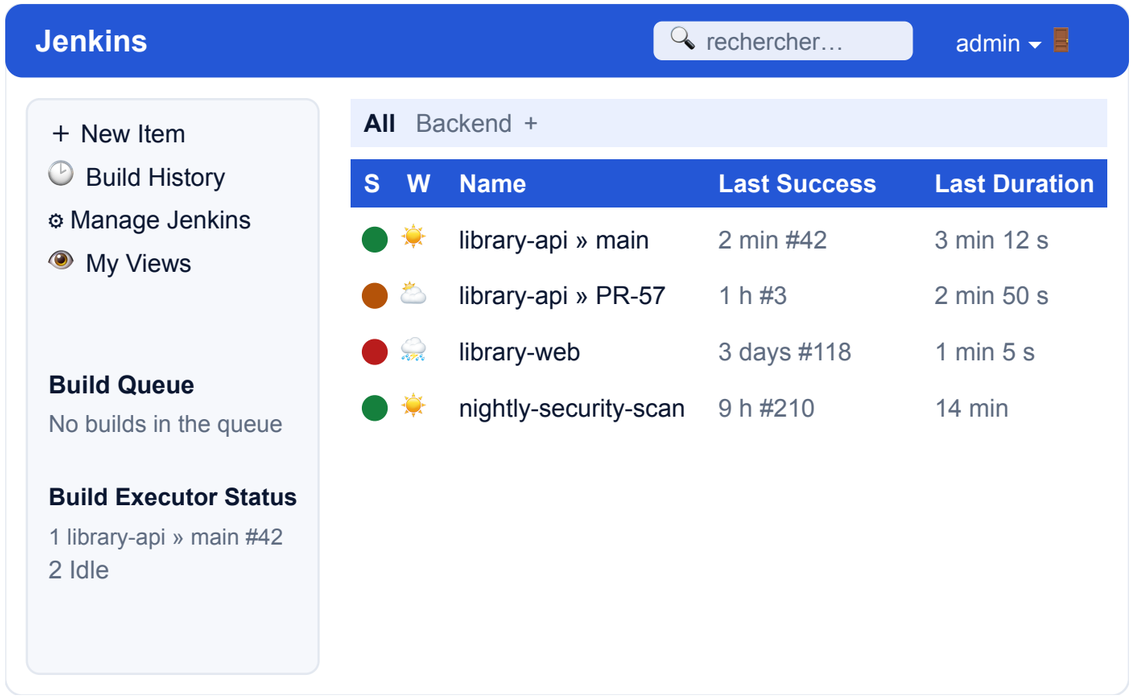


Comment le lire

1. Le **controller** garde, pour chaque build, le résultat, la console, les rapports de tests et les artefacts archivés, dans `JENKINS_HOME/jobs/…/builds/<n>/` (Multibranch : `jobs/library-api/branches/main/builds/42/`).
2. Le **workspace** est sur l'agent : sources extraites et fichiers produits (`target/`). Il est réutilisé par le build suivant du même job sur cet agent.
3. Deux builds simultanés du même job ne partagent pas leur workspace : le second travaille dans `…@2`.
4. Le workspace est **jetable** : on le nettoie (`cleanWs()`) ou on utilise un agent éphémère.
5. Ce qui doit survivre au build passe donc du workspace au controller : `archiveArtifacts`, `junit`, `recordCoverage`.

À retenir
Un fichier laissé dans le workspace n'est pas « sauvegardé » : il disparaît au prochain nettoyage ou sur un autre agent.

Visite guidée de l'interface (1/3) : le dashboard



Éléments de la page d'accueil

- **Barre supérieure** : recherche globale (jobs, builds, nodes), utilisateur connecté, notifications administrateur (mises à jour, alertes de sécurité)
- **Menu latéral** : *New Item*, *Build History*, *Manage Jenkins*, *My Views* (*People* a quitté le core en 2.452 : plugin People View)
- **Build Queue** : builds en attente d'un executor (et pourquoi : label indisponible, agent hors ligne)
- **Build Executor Status** : ce qui tourne maintenant, sur quel node
- **Vues (onglets)** : *All* + vues personnalisées (filtres par regex, par nom) ; une *Dashboard View* ou une *Pipeline View* peuvent être ajoutées par plugin
- **Liste des jobs** : colonne **S** (statut du dernier build), **W** (météo), dernier succès/échec, durée, bouton ▶ pour lancer

Depuis les versions récentes, l'UI est modernisée (thème sombre, nouvelle navigation) mais la structure reste identique.

Visite guidée de l'interface (2/3) : la page d'un job

Menu du job

- **Status** : description, derniers builds, artefacts du dernier build (lien *Workspace* en Freestyle uniquement)
- **Changes** : commits agrégés build par build
- **Build Now** / **Build with Parameters** : lancement manuel
- **Configure** : toute la configuration (Freestyle) ou l'emplacement du Jenkinsfile (Pipeline)
- **Delete**, **Rename**, **Move** (vers un Folder)
- **Stages** (Pipeline Graph View) ou **Full Stage View** (plugin Stage View) : stages × builds avec durée et statut
- **Pipeline Syntax** : générateur de snippets
- **Build History** (colonne de gauche) : builds numérotés, tri, filtre, icône de statut
- **Trend** : graphique de durée et de résultat des builds

Page d'un build

- En-tête : **#42**, résultat, date, durée, déclencheur
- **Console Output** (+ *View as plain text*)
- **Edit Build Information** : nom d'affichage et description (ex. « v1.4.2 déployé en recette »)
- **Delete build**, **Keep this build forever**
- **Test Result**, **Coverage**, **Warnings** : onglets ajoutés par les plugins de rapport
- **Pipeline Overview / Steps** : vue graphique des stages, logs par step
- **Replay**, **Restart from Stage**, **Rebuild** (plugin)

Multibranch Pipeline

- Onglets **Branches** / **Pull Requests** ; **Scan Repository Now** ; **Scan Repository Log** pour diagnostiquer la découverte des branches

Visite guidée de l'interface (3/3) : Manage Jenkins

System Configuration

- **System** : URL Jenkins, admin e-mail, executors du built-in node, variables globales, SMTP, intégrations (GitHub, SonarQube…)
- **Tools** : JDK, Maven, Git, Gradle, NodeJS, SonarScanner
- **Plugins** : installés, disponibles, mises à jour, avancé (proxy, upload .hpi)
- **Nodes** (et **Clouds**) : agents, labels, executors, état

Security

- **Security** : security realm (comment on s'authentifie), authorization strategy (qui a le droit de quoi), CSRF, agent protocols, Markup Formatter
- **Credentials** et **Credential Providers**
- **Users** : base interne d'utilisateurs, tokens API
- **In-process Script Approval** : scripts Groovy hors sandbox

Status & Tools

- **System Information** : version, JVM, variables d'environnement
- **System Log** : loggers configurables par package Java
- **Load Statistics**, **Disk Usage**
- **Script Console** : Groovy exécuté sur le controller – contrôle total de Jenkins (secrets compris) et accès système avec les droits du processus Jenkins ; réservé à *Overall/Administer*
- **Prepare for Shutdown**, **Reload Configuration from Disk**
- **Jenkins CLI**, **About Jenkins**

Administration Monitors

Les bandeaux en haut de page (*Manage Jenkins*) signalent : plugins avec vulnérabilités connues, nouvelle LTS disponible, configuration dépréciée, builds sur le controller. À traiter, pas à masquer.

Jenkins dans l'IDE

IntelliJ IDEA

- **Jenkins Control Plugin** : fenêtre d'outil listant jobs et builds, lancement d'un build, affichage du log, notifications de résultat, navigation vers l'UI
- Configuration : URL du controller + utilisateur + **API token** (jamais le mot de passe)
- **Support Jenkinsfile** : le fichier est reconnu comme Groovy ; complétion du DSL en important le GDSL du controller (*Pipeline Syntax → IntelliJ IDEA GDSL*) ; validation avec le plugin *Jenkins Pipeline Linter*

Visual Studio Code

- **Jenkins Jack** : exécuter un Jenkinsfile local sur le serveur sans commit, consulter les logs, scripts Groovy
- **Jenkins Pipeline Linter Connector** : valide la syntaxe déclarative via l'API du controller
- **Jenkins Status** : état des jobs dans la barre d'état
- Extension *Groovy* pour la coloration syntaxique

Valider un Jenkinsfile sans IDE

```
# Via l'API REST du controller (plugin Pipeline Model Definition)
JENKINS=https://jenkins.example.com
curl -s -u user:apitoken -X POST \
  -F "jenkinsfile=<Jenkinsfile" \
  $JENKINS/pipeline-model-converter/validate

# Réponse attendue :
# Jenkinsfile successfully validated.

# Via la CLI Jenkins
java -jar jenkins-cli.jar -s $JENKINS -auth user:apitoken \
  declarative-linter < Jenkinsfile
```

Pourquoi c'est utile

- Erreur de syntaxe détectée **avant** le commit, sans attendre un build rouge
- Boucle de développement du pipeline raccourcie (avec *Replay* côté serveur)

API token

Utilisateur → Security → API Token → Add new token. Révocable, limité à l'API, à utiliser pour tout outil externe.

Installation : prérequis et options

Prérequis matériels et logiciels

- **Java** : **21 ou 25**. Java 17 n'est plus supporté (dernière LTS compatible : 2.541.3, mars 2026) ; un controller lancé avec une version non supportée ne démarre pas.
- **Mémoire** : 256 Mo minimum selon la doc officielle, 4 Go+ recommandés pour une équipe ; agents dimensionnés selon les builds
- **Disque** : 50 Go+ pour `JENKINS_HOME` (historique, artefacts, plugins) – sur un volume dédié et sauvegardé
- **OS** : Linux (recommandé), Windows, macOS, ou conteneur
- **Réseau** : accès à l'Update Center (`updates.jenkins.io`) ou miroir interne ; ports 8080 (HTTP) et 50000 (agents inbound)

JENKINS_HOME

Répertoire qui contient **tout** l'état de Jenkins : `config.xml`, `jobs/`, `plugins/`, `secrets/`, `users/`, `nodes/`. Le sauvegarder = sauvegarder Jenkins.

Modes d'installation

Mode	Pour
Docker (`jenkins/jenkins:lts-jdk21`)	Formation, dev, et production conteneurisée
Paquet apt / dnf / zypper	Serveur Linux classique (service systemd)
WAR (`java -jar jenkins.war`)	Tout OS avec Java ; test rapide
Helm chart (`jenkinsci/jenkins`)	Kubernetes, agents en pods
Installeur Windows (.msi) / macOS (Homebrew)	Postes de travail

Choisir sa version

- **LTS** en production : stabilité, correctifs de sécurité
- **Weekly** : nouveautés, pour tester
- Suivre les *Security Advisories* (`jenkins.io/security`)

Installation : Docker (environnement de la formation)

Lancement simple

```
# Volume nommé pour persister JENKINS_HOME
docker volume create jenkins_home

docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  --restart unless-stopped \
  jenkins/jenkins:lts-jdk21

# Mot de passe initial
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

# Logs
docker logs -f jenkins
```

- Image officielle, utilisateur `jenkins` (uid 1000), JDK 21 inclus
- Port 8080 : UI ; port 50000 : agents inbound
- Sans volume, toute la configuration est perdue à la suppression du conteneur

Compose : Jenkins + SonarQube + Nexus

```
# docker-compose.yml
services:
  jenkins:
    image: jenkins/jenkins:lts-jdk21
    ports: ["8080:8080", "50000:50000"]
    volumes:
      - jenkins_home:/var/jenkins_home
    environment:
      JAVA_OPTS: "-Djenkins.install.runSetupWizard=true -Xmx2g"
    restart: unless-stopped

  sonarqube:
    image: sonarqube:community
    ports: ["9000:9000"]
    environment: { SONAR_ES_BOOTSTRAP_CHECKS_DISABLE: "true" }

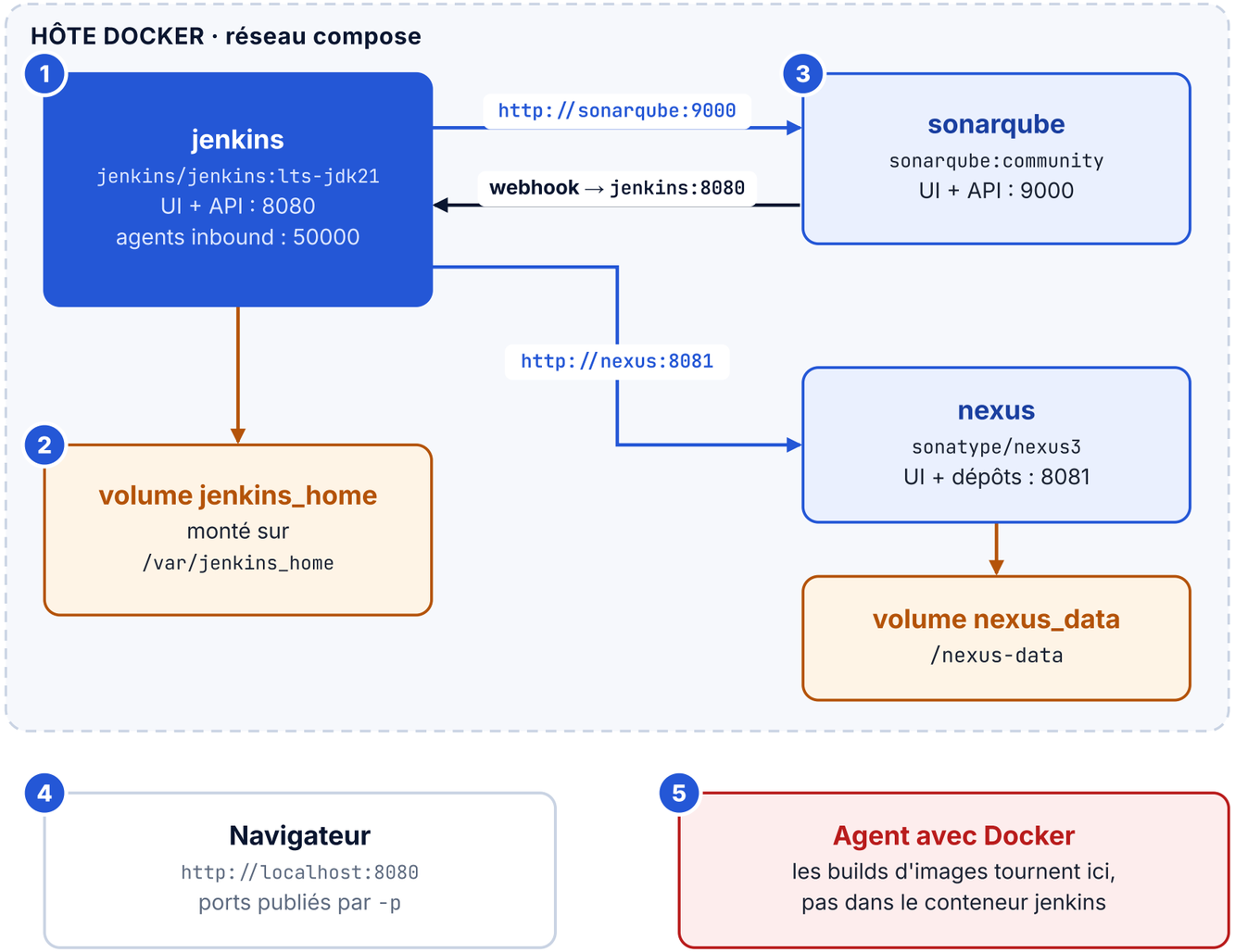
  nexus:
    image: sonatype/nexus3
    ports: ["8081:8081"]
    volumes: [ nexus_data:/nexus-data ]

volumes: { jenkins_home: {}, nexus_data: {} }
```

Builds Docker

Ils tournent sur un agent qui dispose de Docker, pas dans ce conteneur : l'image `jenkins/jenkins` n'a pas la CLI `docker`, et monter `/var/run/docker.sock` donnerait en plus un accès root à l'hôte.

Schéma : l'environnement Docker de la formation



Comment le lire

1. Le conteneur **jenkins** publie 8080 (interface et API) et 50000 (agents inbound en TCP).
2. Toute la configuration vit dans le **volume** `jenkins_home` : supprimer le conteneur ne perd rien, supprimer le volume perd tout.
3. Dans le réseau compose, les services se joignent **par leur nom** : Jenkins appelle `http://sonarqube:9000` et `http://nexus:8081` ; SonarQube rappelle `http://jenkins:8080/sonarqube-webhook/`.
4. Depuis le poste, on passe par les ports publiés : `localhost:8080`, `:9000`, `:8081`.
5. Les builds Docker tournent sur un **agent** qui a Docker : l'image `jenkins/jenkins` n'a pas la CLI docker, et monter la socket donnerait un accès root à l'hôte.

À retenir

`localhost` dans un conteneur désigne **ce conteneur** : entre services, on utilise le nom du service.

Installation : Linux (paquet) et WAR

Debian / Ubuntu

```
sudo apt-get update && sudo apt-get install -y fontconfig openjdk-21-jre

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key | \
  sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt-get update && sudo apt-get install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins

sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

RHEL / Rocky / Fedora

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/rpm-stable/jenkins.repo
sudo dnf install -y fontconfig java-21-openjdk jenkins
sudo systemctl enable --now jenkins
```

Fichiers importants (paquet)

Élément	Emplacement
JENKINS_HOME	`/var/lib/jenkins`
Logs	`journalctl -u jenkins`
Options JVM, port, utilisateur	`systemctl edit jenkins` (override `Environment="JAVA_OPTS=…"`, `JENKINS_PORT`)
Utilisateur système	`jenkins`

WAR autonome

```
wget https://get.jenkins.io/war-stable/latest/jenkins.war
java -jar jenkins.war --httpPort=8080
# JENKINS_HOME = ~/.jenkins par défaut
JENKINS_HOME=/srv/jenkins java -jar jenkins.war
```

Reverse proxy

En production : Nginx/Traefik en frontal avec TLS, et *Jenkins URL* configurée en `https://…` pour que les liens et webhooks soient corrects.

Premier démarrage : l'assistant de configuration

1. Unlock
Coller le mot de passe de `initialAdminPassword`

2. Plugins
Install suggested plugins ou sélection manuelle

3. Admin
Créer le premier utilisateur administrateur

4. URL
Définir la *Jenkins URL* (utilisée dans les mails, webhooks)

5. Prêt
Accès au dashboard

Les plugins suggérés installent notamment

- Folders, OWASP Markup Formatter, Build Timeout, Credentials Binding, Timestamper, Workspace Cleanup
- **Pipeline** (et toute sa suite), Pipeline: GitHub Groovy Libraries, Pipeline Graph View
- **Git**, GitHub Branch Source
- SSH Build Agents, Matrix Authorization Strategy, PAM/LDAP Authentication
- **Email Extension**, Mailer

Un bon point de départ ; les plugins Maven, JUnit, Coverage, SonarQube seront ajoutés dans la partie 3.

Vérifications après installation

- *Manage Jenkins* → *System Information* : version Jenkins, version Java, `JENKINS_HOME`
- *Manage Jenkins* : aucun bandeau d'alerte de sécurité
- *Manage Jenkins* → *Security* : *Logged-in users can do anything* est le réglage par défaut – à durcir (partie 3, Habilitations)
- Créer un premier job Freestyle « hello » avec un step shell `echo "Hello $BUILD_NUMBER"` et vérifier la console

Désactiver l'assistant (automatisation)

`-Djenkins.install.runSetupWizard=false` + **Configuration as Code** (plugin JCasC) pour un Jenkins entièrement provisionné par fichier YAML.

Configuration générale : System

Manage Jenkins → System

Paramètre	Valeur recommandée
Home directory	Lecture seule (JENKINS_HOME)
System Message	Bandeau d'information (maintenance, contact)
# of executors (built-in node)	**0** en production ; 2 en formation faute d'agents
Labels (built-in)	Vide ou `controller`
Usage	*Only build jobs with label expressions matching this node*
Quiet period	5 s (regroupe les pushes rapprochés)
SCM checkout retry count	2–3
Jenkins URL	URL publique exacte, avec `/` final
System Admin e-mail address	Expéditeur des mails : `jenkins@example.com`
Global properties → Environment variables	Variables partagées par tous les builds (proxy, `MAVEN_OPTS`…)

Sections ajoutées par les plugins

- **GitHub** / **GitLab** : serveurs et credentials d'API
- **SonarQube servers** : URL + token
- **E-mail Notification** et **Extended E-mail Notification** : SMTP
- **Global Pipeline Libraries** : Shared Libraries
- **Timestamper**, **Pipeline Graph View**…

Exemple JCasC équivalent

```
# jenkins.yaml (plugin Configuration as Code)
jenkins:
  systemMessage: "Jenkins Utopios – formation"
  numExecutors: 0
  mode: EXCLUSIVE
  quietPeriod: 5
  scmCheckoutRetryCount: 2
unclassified:
  location:
    url: "https://jenkins.example.com/"
    adminAddress: "jenkins@example.com"
  globalLibraries: { libraries: [] }
```

Configuration générale : Tools

Manage Jenkins → Tools

Déclare les outils utilisables par les jobs, avec un **nom** référencé dans les pipelines (`tools { }`) ou les jobs Freestyle.

Outil	Installation	Nom (convention)
JDK	Chemin déjà présent sur l'agent (`JAVA_HOME`) ou installeur automatique (adoptium)	`jdk21`
Maven	Install automatically from Apache (version 3.9.x) ou chemin	`maven-3.9`
Git	Exécutable `git` du PATH (par défaut)	`Default`
SonarQube Scanner	Installeur automatique	`sonar-scanner`
NodeJS, Gradle, Ant, Docker	Selon plugins	—

Installateur automatique

Jenkins télécharge et installe l'outil **sur l'agent** au premier usage, dans `<agent root>/tools/` . Pratique en formation ; en production, préférer des images d'agent préconstruites.

Utilisation dans un pipeline

```
pipeline {
  agent any
  tools {
    jdk   'jdk21'
    maven 'maven-3.9'
  }
  stages {
    stage('Versions') {
      steps {
        sh 'java -version'
        sh 'mvn -v'
      }
    }
  }
}
```

Les outils déclarés sont ajoutés au `PATH` et `JAVA_HOME` / `MAVEN_HOME` sont positionnés.

Alternative : un agent Docker qui embarque les outils

```
agent {
  docker {
    image 'maven:3.9-eclipse-temurin-21'
    args  '-v m2cache:/root/.m2'   // cache des dépendances
  }
}
```

Configuration générale : Credentials

Principe

- Les secrets sont stockés **chiffrés** dans `JENKINS_HOME/credentials.xml` (clé dans `secrets/`)
- Identifiés par un **ID** stable (ex. `git-ssh-key`, `nexus-deploy`) utilisé dans les jobs
- Jamais affichés en clair dans l'UI ni dans les logs (masquage automatique dans la console)

Types courants

Type	Usage
Username with password	Nexus, registry Docker, SMTP, Git HTTPS
SSH Username with private key	Git SSH, agents SSH, déploiement SSH
Secret text	Token API (GitHub, GitLab, SonarQube, Slack)
Secret file	Keystore, kubeconfig, `settings.xml`
Certificate	Client TLS (PKCS#12)

Scope et emplacement

- **Scope Global** : utilisable par les jobs ; **scope System** : réservé au controller (connexion aux agents, SMTP)
- **Emplacement** (*store*) : Jenkins (global), **Folder** (visible des seuls items du dossier – isolation entre équipes), User

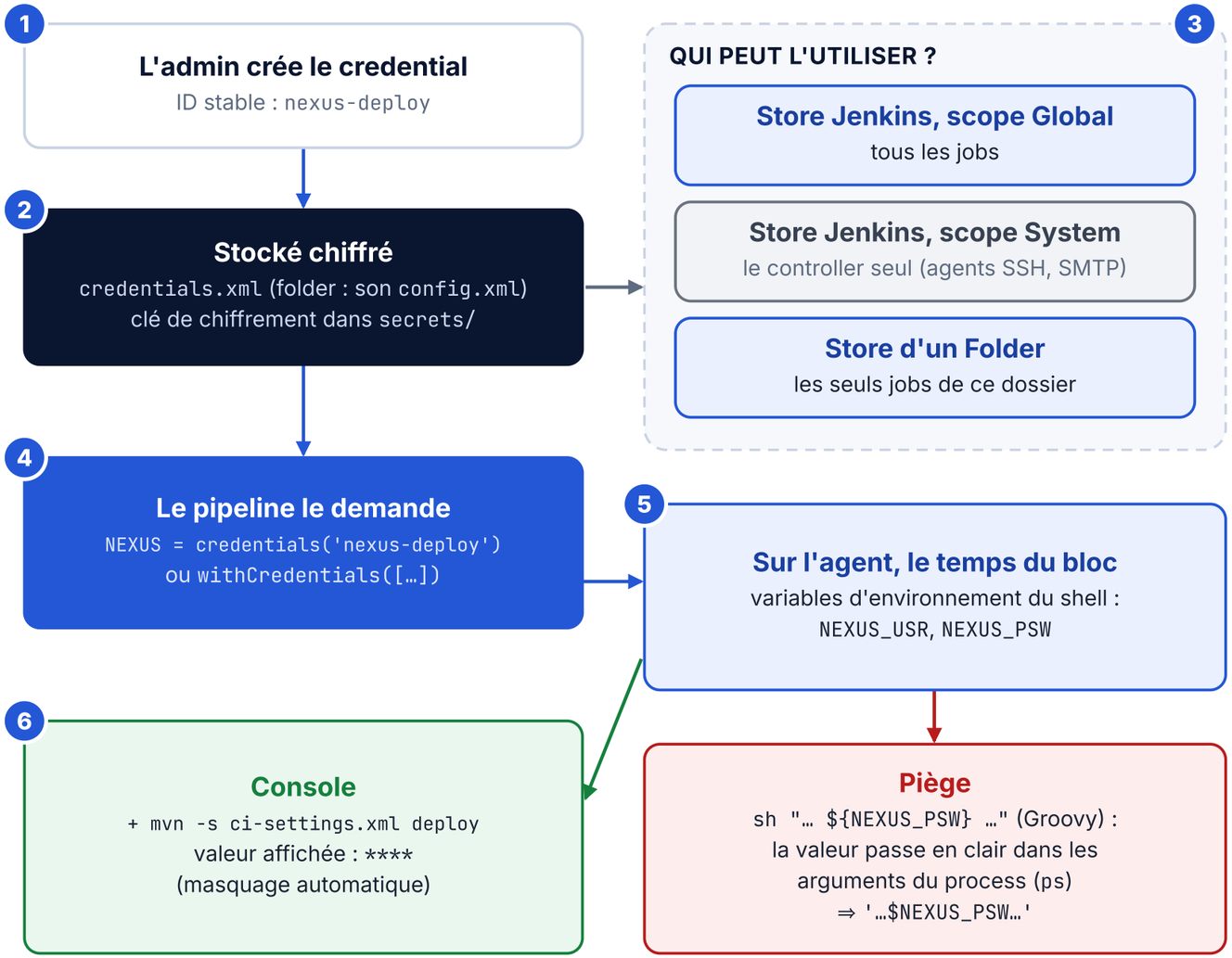
Utilisation dans un pipeline

```
pipeline {
  agent any
  environment {
    // Username/password → NEXUS_USR et NEXUS_PSW
    NEXUS = credentials('nexus-deploy')
    SONAR_TOKEN = credentials('sonar-token')   // secret text
  }
  stages {
    stage('Deploy') {
      steps {
        // settings.xml : <server><username>${env.NEXUS_USR}</username>…
        sh 'mvn -B -s ci-settings.xml deploy'
      }
    }
    stage('Docker push') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'docker-registry',
                         usernameVariable: 'REG_USER',
                         passwordVariable: 'REG_PASS')]) {
          sh 'echo "$REG_PASS" | docker login -u "$REG_USER" --password-stdin registry.example.com'
        }
      }
    }
  }
}
```

Guillemets simples

Utiliser `'…$VAR…'` (interpolation par le shell), pas `"…${VAR}…"` (interpolation Groovy) : sinon la valeur est insérée dans la ligne de commande du process (visible par `ps` sur l'agent) ; Jenkins le signale par un avertissement dans la console.

Schéma : le parcours d'un credential



Comment le lire

1. L'administrateur enregistre le secret sous un **ID** stable ; les jobs ne connaissent que cet ID.
2. Jenkins le stocke **chiffré** (`credentials.xml` , ou le `config.xml` du folder) ; la clé est dans `secrets/` (d'où l'importance de sauvegarder les deux ensemble).
3. Le **store** et le **scope** décident qui peut l'utiliser : tous les jobs, le controller seul, ou les jobs d'un dossier.
4. Le pipeline le demande par son ID (`credentials()` , `withCredentials`).
5. Jenkins l'expose au shell de l'agent comme **variables d'environnement**, uniquement pendant le bloc concerné.
6. Si la valeur apparaît dans la console, elle est **masquée** ; mais une interpolation Groovy (`"${…}"`) met la valeur dans la ligne de commande, visible par `ps` sur l'agent : guillemets simples.

À retenir
Un secret ne sort jamais du store en clair dans l'UI ; il ne fuit que si le pipeline le recopie lui-même (interpolation Groovy, écriture dans un fichier ou un artefact).

Configuration générale : Nodes et agents

Ajouter un agent SSH (*Manage Jenkins → Nodes → New Node*)

1. Sur la machine agent : installer Java 21 et Git, créer un utilisateur `jenkins`, un répertoire `/home/jenkins/agent`, déposer la clé publique du controller dans `authorized_keys`
2. Dans Jenkins : *Name* `linux-1`, *Remote root directory* `/home/jenkins/agent`, *Labels* `linux java21 maven docker`, *Usage* « Only build jobs with label expressions », *Launch method* « Launch agents via SSH », host + credential SSH
3. Save → Jenkins se connecte, copie `remoting.jar`, l'agent passe *online* (voir le log du node)

Agent inbound (Windows, derrière NAT)

```
# Récupéré depuis la page du node
curl -sO https://jenkins.example.com/jnlpJars/agent.jar
java -jar agent.jar -url https://jenkins.example.com/ \
  -secret @secret-file -name windows-1 -webSocket \
  -workDir "C:\jenkins"
```

Agents cloud (à la demande)

- **Docker plugin** : un conteneur par build, à partir d'une image (ex. `jenkins/agent:jdk21`) ; label `docker`
- **Kubernetes plugin** : un pod par build, définition en YAML dans le Jenkinsfile
- **EC2 / Azure VM** : VM créée au besoin, détruite après inactivité

```
agent {
  kubernetes {
    yaml '''
      apiVersion: v1
      kind: Pod
      spec:
        containers:
        - name: maven
          image: maven:3.9-eclipse-temurin-21
          command: ["sleep"]
          args: ["infinity"]
    '''
    defaultContainer 'maven'
  }
}
```

Règle

Controller à 0 executor + agents éphémères = sécurité (le build ne touche pas le controller), reproductibilité (image figée) et élasticité.

Installation des plugins

Manage Jenkins → Plugins

- **Updates** : mises à jour disponibles, avec indication des vulnérabilités corrigées
- **Available plugins** : recherche dans l'Update Center ; *Install* (sans redémarrage si possible) ou *Download now and install after restart*
- **Installed plugins** : activer/désactiver, désinstaller, voir les dépendances
- **Advanced settings** : proxy HTTP, **Deploy Plugin** (upload manuel d'un `.hpi`), URL de l'Update Center

Mécanique

- Un plugin = un fichier `.hpi` / `.jpi` dans `JENKINS_HOME/plugins/`
- Les plugins ont des **dépendances** entre eux (installées automatiquement) et une **version minimale de Jenkins**
- Beaucoup de fonctionnalités « de base » (Git, Pipeline, Credentials) sont des plugins
- Un redémarrage est parfois nécessaire : `<JENKINS_URL>/safeRestart` attend la fin des builds en cours

En ligne de commande

```
# Via la CLI
java -jar jenkins-cli.jar -s https://jenkins.example.com/ -auth user:token \
  install-plugin maven-plugin junit coverage sonar -restart

# Dans une image Docker personnalisée
# plugins.txt :
#   git:<version>
#   workflow-aggregator:<version>
#   pipeline-maven:<version>
#   junit:<version>
#   coverage:<version>
#   sonar:<version>
#   email-ext:<version>
#   role-strategy:<version>
#   configuration-as-code:<version>
FROM jenkins/jenkins:lts-jdk21
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt
```

Réseau isolé (air-gapped)

- Télécharger les `.hpi` et leurs dépendances depuis `updates.jenkins.io/download/plugins/` ou avec `jenkins-plugin-cli` sur un poste connecté
- Uploader via *Advanced settings* → *Deploy Plugin*, ou copier dans `plugins/` puis redémarrer
- Alternative : miroir interne de l'Update Center (`update-center.json`)

Plugins essentiels pour une chaîne Java/Maven

Plugin	Apporte	Utilisé dans la partie
Pipeline (workflow-aggregator)	Jobs Pipeline, Jenkinsfile, stages, Multibranch	2, 3
Git + **GitHub** / **GitLab** (+ Branch Source)	Checkout Git, webhooks, découverte des branches/PR, status checks	3 – Accès aux sources
Credentials + **Credentials Binding** + **SSH Agent**	Stockage et injection des secrets	2, 3
Maven Integration (maven-plugin)	Type de job « Maven project », invocation Maven en Freestyle	3 – Création d'un job
Pipeline Maven Integration (pipeline-maven)	Step `withMaven` : publication automatique des rapports JUnit, de couverture, des artefacts	3 – Paramétrage de Maven
JUnit	Step `junit`, onglet Test Result, tendances	3 – Rapports de tests
Coverage (remplace les plugins JaCoCo et Code Coverage API, dépréciés)	Rapports de couverture, seuils, tendance	3 – Rapports de tests
Email Extension (email-ext)	Mails riches, templates, destinataires dynamiques	3 – Notifications
SonarQube Scanner (sonar)	`withSonarQubeEnv`, `waitForQualityGate`	3 – Qualité
Warnings Next Generation	Agrégation d'alertes (javac, Checkstyle, SpotBugs, PMD)	3 – Qualité
Nexus Artifact Uploader / **Artifactory**	Publication d'artefacts	3 – Déploiement
Publish Over SSH, **Docker Pipeline**, **Kubernetes**	Déploiement et agents	3 – Déploiement
Matrix Authorization, **Role-based Authorization Strategy**, **LDAP**	Habilitations	3 – Habilitations
Configuration as Code (JCasC), **Job DSL**	Jenkins et jobs décrits en YAML/Groovy	Industrialisation

Plugins : bonnes pratiques d'administration

Installer peu, installer utile

Chaque plugin est du code tiers chargé dans le controller : surface d'attaque, mémoire, risques d'incompatibilité. Vérifier la popularité, la date de dernière release et le nombre de mainteneurs sur `plugins.jenkins.io`.

Mettre à jour régulièrement

Des Security Advisories Jenkins paraissent plusieurs fois par mois, sans calendrier fixe : s'abonner au flux. Un bandeau rouge dans *Manage Jenkins* signale les plugins vulnérables installés.

Tester avant de mettre à jour

Une instance de pré-production (ou une copie de `JENKINS_HOME` démarrée en Docker) pour valider une montée de version LTS + plugins.

Figer les versions

`plugins.txt` avec versions explicites + image Docker reconstruite = Jenkins reproductible. Pas de « Install » cliqué en production sans trace.

Sauvegarder JENKINS_HOME

Avant toute mise à jour : snapshot du volume ou `tar` de `JENKINS_HOME` (sans `workspace/` ni `builds/*/archive` si volumineux).

Documenter

Pourquoi chaque plugin est installé et par qui il est utilisé. Désinstaller ce qui ne sert plus.

Attention aux plugins « UI »

Certains anciens plugins (Blue Ocean, anciens thèmes) ne sont plus maintenus. Préférer **Pipeline Graph View** pour la visualisation des pipelines.

03

Construire un projet Java avec Maven

Rappels Maven · Création d'un job · Accès aux sources · Paramétrage de Maven · Rapports de tests unitaires · Envoi de mails · Déploiement automatique · Rapports d'analyse qualité · Habilitations

Rappels Maven : le POM et les coordonnées

Project Object Model

- `pom.xml` à la racine : décrit **quoi** construire, pas comment (convention over configuration)
- **Coordonnées** (GAV) : `groupId` : `artifactId` : `version` (+ `packaging`)
- Les versions `-SNAPSHOT` sont des versions de développement (écrasables) ; les versions sans suffixe sont des **releases** immuables
- **Héritage** : `<parent>` (ex. `spring-boot-starter-parent`) ; **agrégation** : `<modules>` pour les projets multi-modules

Arborescence standard

```
library-api/
├── pom.xml
├── mvnw, mvnw.cmd, .mvn/        # Maven Wrapper
├── Jenkinsfile
└── src/
    ├── main/java/…              # code
    ├── main/resources/          # application.yml
    ├── test/java/…              # *Test.java (unitaires), *IT.java (intégration)
    └── test/resources/
```

`pom.xml` du fil rouge (extrait)

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
  </parent>

  <groupId>fr.utopios</groupId>
  <artifactId>library-api</artifactId>
  <version>${revision}</version>   <!— fixée par la CI —>
  <packaging>jar</packaging>

  <properties>
    <revision>1.4.0-SNAPSHOT</revision>
    <java.version>21</java.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build><plugins>
    <!— tests *IT : Failsafe n'est lié au cycle que s'il est déclaré —>
    <plugin><artifactId>maven-failsafe-plugin</artifactId></plugin>
    <!— JAR exécutable (repackage), pour le java -jar du Dockerfile —>
    <plugin><groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId></plugin>
    <!— POM publié avec ${revision} résolu (version + executions obligatoires) —>
    <plugin><groupId>org.codehaus.mojo</groupId>
      <artifactId>flatten-maven-plugin</artifactId><version>1.8.0</version>
      <configuration><updatePomFile>true</updatePomFile>
        <flattenMode>resolveCiFriendliesOnly</flattenMode></configuration>
      <executions>
        <execution><id>flatten</id><phase>process-resources</phase>
          <goals><goal>flatten</goal></goals></execution>
        <execution><id>flatten-clean</id><phase>clean</phase>
          <goals><goal>clean</goal></goals></execution>
      </executions></plugin>
  </plugins></build>
</project>
```

Rappels Maven : le cycle de vie et les phases



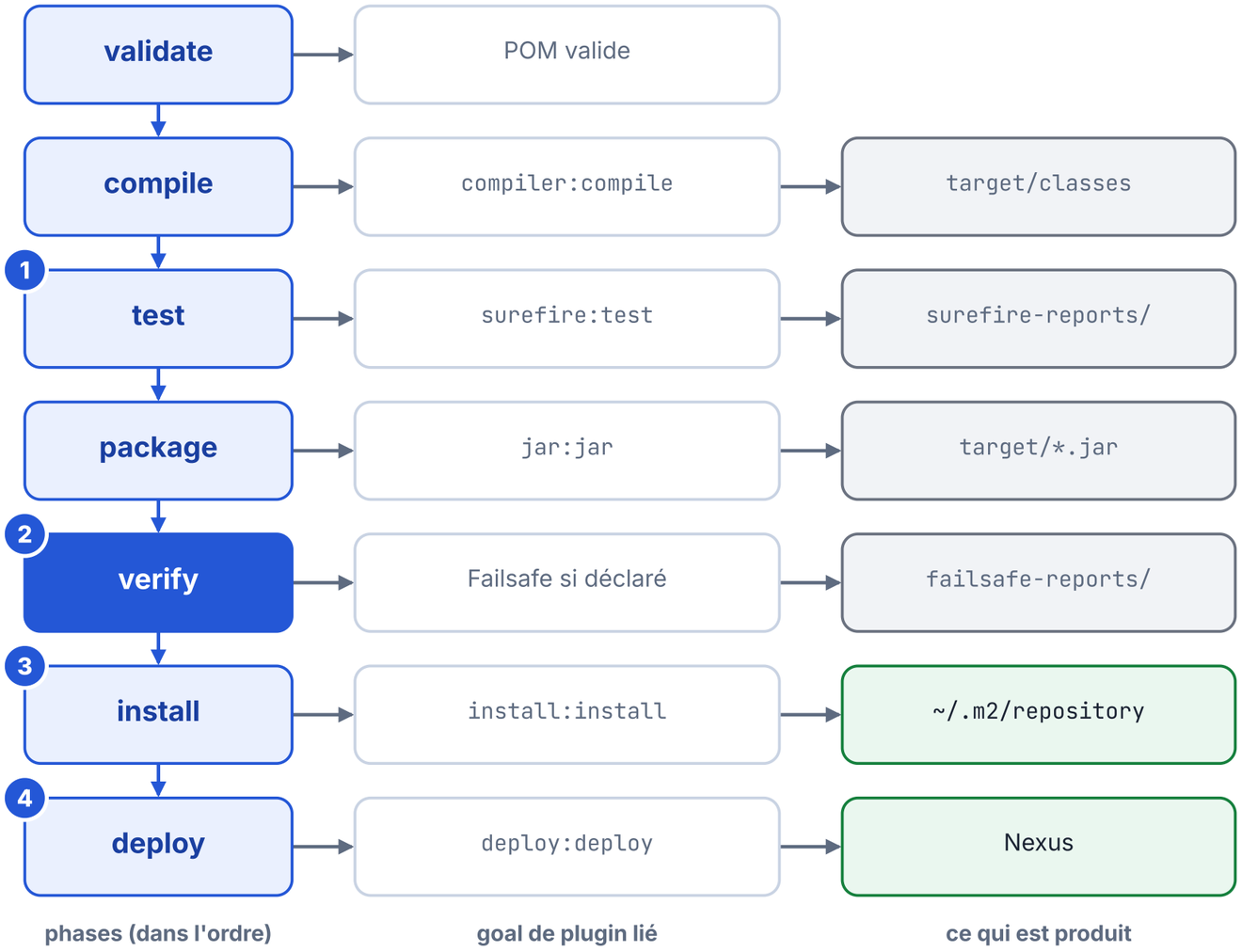
Règles

- Invoquer une phase exécute **toutes les phases précédentes** : `mvn package` = validate + compile + test + package
- `clean` appartient à un cycle séparé : `mvn clean verify` est la commande de CI standard
- Chaque phase est liée à des **goals de plugins** : `compiler:compile`, `surefire:test`, `jar:jar`, `deploy:deploy` ; Failsafe (`failsafe:integration-test`, `failsafe:verify`) seulement s'il est déclaré dans le POM
- Un goal peut être appelé directement : `mvn jacoco:report`, `mvn sonar:sonar`, `mvn dependency:tree`

Options utiles en CI

Option	Effet
`-B` / `--batch-mode`	Pas d'interaction, pas de couleurs ni de barres de progression (logs propres)
`-U`	Force la mise à jour des SNAPSHOT distants
`-DskipTests`	Compile les tests mais ne les exécute pas (préférer à `-Dmaven.test.skip`)
`-Dmaven.test.failure.ignore=true`	Continue malgré des tests rouges ⇒ Jenkins marque UNSTABLE via `junit`
`-s settings.xml`	Fichier de configuration (dépôts, credentials, proxy)
`-P profil`	Active un profil (ex. `-P ci`, `-P integration`)
`-T 1C`	Build parallèle (1 thread par cœur) sur multi-modules
`-ntp`	Pas de log de progression des téléchargements

Schéma : ce que chaque phase Maven exécute et produit



Comment le lire

1. `test` : Surefire exécute les tests unitaires et écrit les rapports JUnit XML que Jenkins publiera.
2. `verify` : demander une phase exécute toutes les précédentes ; `mvn clean verify` compile, teste, package et lance les tests d'intégration (Failsafe, s'il est déclaré).
3. `install` copie l'artefact dans le dépôt local `~/.m2` de la machine : utile entre modules, inutile pour publier.
4. `deploy` publie dans le dépôt distant (Nexus) : c'est la phase qui rend l'artefact disponible aux autres.

À retenir

En CI : `clean verify` sur chaque commit ; `deploy` seulement sur la branche ou le tag qui publie.

Rappels Maven : dépendances, dépôts, settings.xml

Résolution des dépendances

- Recherche dans le **dépôt local** `~/.m2/repository`, sinon téléchargement depuis les **dépôts distants** (Maven Central par défaut)
- **Scopes** : `compile` (défaut), `provided`, `runtime`, `test`, `import` (BOM)
- Dépendances transitives résolues automatiquement ; `<dependencyManagement>` fixe les versions
- `mvn dependency:tree` pour comprendre d'où vient une version

En entreprise : un dépôt proxy

- **Nexus** / **Artifactory** servent de **proxy cache** de Maven Central (vitesse, résilience, audit) et d'**hébergement** des artefacts internes (`maven-releases`, `maven-snapshots`)
- Le `settings.xml` redirige tout vers ce dépôt (`<mirror>`) et porte les credentials de publication (`<server>`)

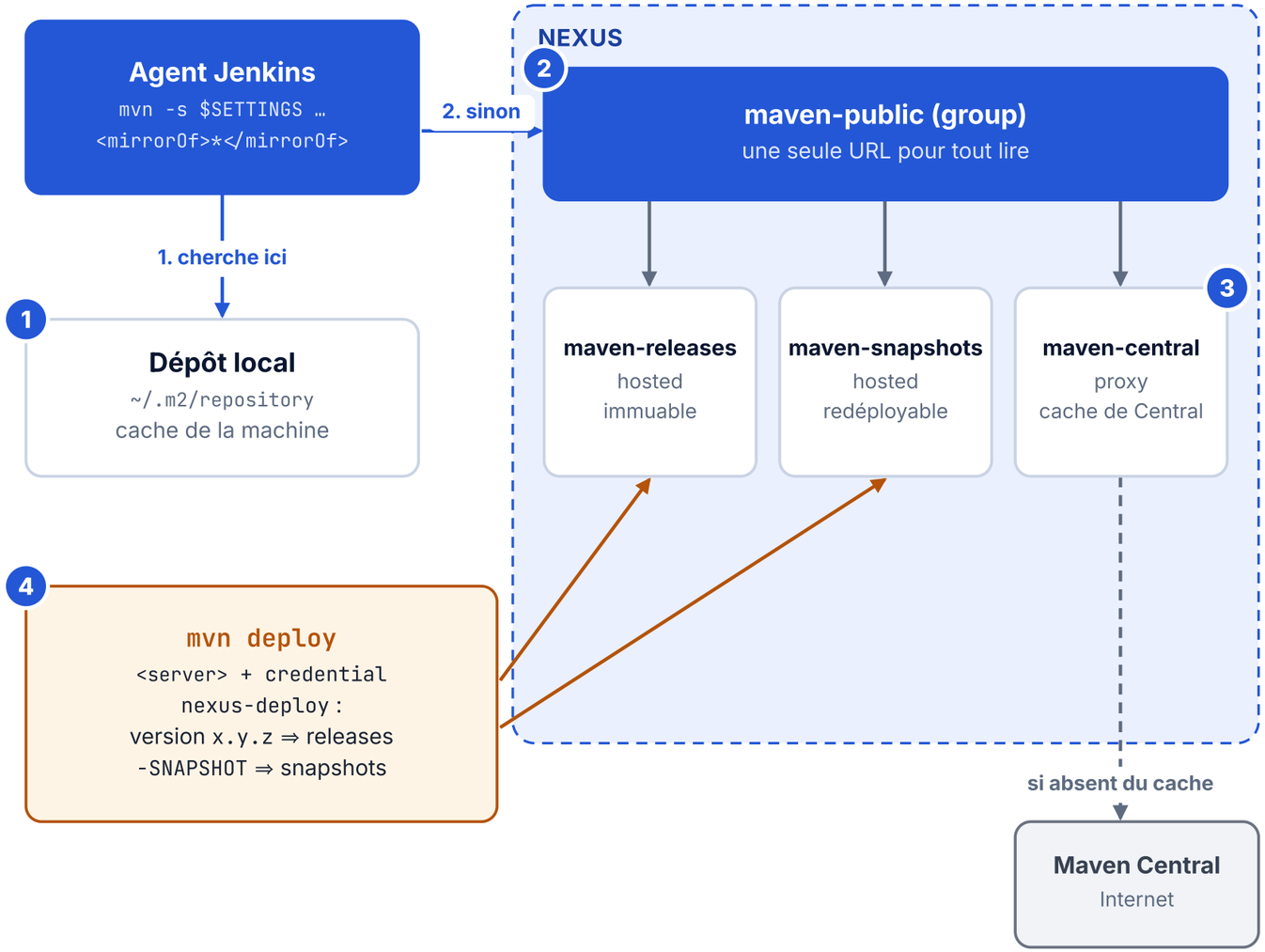
`settings.xml` pour les agents Jenkins

```
<settings>
  <mirrors>
    <mirror>
      <id>nexus</id>
      <mirrorOf>*</mirrorOf>
      <url>http://nexus:8081/repository/maven-public/</url>
    </mirror>
  </mirrors>
  <servers>
    <server>
      <id>nexus-releases</id>
      <username>${env.NEXUS_USR}</username>
      <password>${env.NEXUS_PSW}</password>
    </server>
    <server>
      <id>nexus-snapshots</id>
      <username>${env.NEXUS_USR}</username>
      <password>${env.NEXUS_PSW}</password>
    </server>
  </servers>
</settings>
```

Où le mettre ?

Dans Jenkins : *Manage Jenkins → Managed files* (plugin Config File Provider) ou credential « Secret file », puis `mvn -s $SETTINGS …`. Les mots de passe viennent de variables injectées par `credentials()`, jamais écrits en dur.

Schéma : d'où viennent les dépendances, où vont les artefacts



Comment le lire

1. Maven cherche d'abord dans le **dépôt local** de la machine (`~/.m2/repository`).
2. Sinon, le `<mirror>` du `settings.xml` envoie **toutes** les requêtes vers le group `maven-public` de Nexus.
3. Le group lit les dépôts internes et le **proxy** de Maven Central, qui ne télécharge sur Internet que ce qu'il n'a pas encore en cache.
4. `mvn deploy` publie dans `maven-releases` ou `maven-snapshots` selon la version, avec les identifiants du `<server>` injectés par le credential.

À retenir
Les agents ne parlent qu'à Nexus : builds plus rapides, résistants aux pannes de Central, et chaque dépendance utilisée est tracée.

Création d'un job (1/2) : Freestyle project

Prérequis

- Plugins **Git**, **Maven Integration**, **JUnit** installés
- Tool `jdk21` et `maven-3.9` déclarés (*Tools*)
- Credential d'accès au dépôt créé

Étapes (*New Item*)

1. **Nom** : `library-api-freestyle`, type *Freestyle project*, OK
2. **General** : description ; *Discard old builds* (20 builds) ; *GitHub project* (URL)
3. **Source Code Management** : Git, URL, credential, branche `*/main`
4. **Build Triggers** : *GitHub hook trigger* (ou *Poll SCM* `H/5 * * * *` en formation)
5. **Build Environment** : *Delete workspace before build starts*, *Add timestamps*
6. **Build Steps** : *Invoke top-level Maven targets* → Maven `maven-3.9`, Goals `-B clean verify`, JVM Options / Properties au besoin
7. **Post-build Actions** : *Publish JUnit test result report* → `target/surefire-reports/*.xml` ; *Archive the artifacts* → `target/*.jar` ; *E-mail Notification*
8. Save → **Build Now** → Console Output

Variante : type « Maven project »

- Fourni par le plugin Maven Integration : Jenkins **parse le POM**, publie automatiquement les rapports Surefire, détecte les modules, archive les artefacts
- Moins de configuration, mais plus lent (Maven embarqué dans un processus Jenkins) et considéré comme **legacy**

Ce qu'on observe après le premier build

- Console : clone Git, résolution des dépendances (lent la 1re fois, puis cache `~/.m2`), `BUILD SUCCESS`
- Page du build : **Test Result** (n tests, 0 échec), **Build Artifacts** (`library-api-1.4.0-SNAPSHOT.jar`)
- Job : météo ☀️, lien **Workspace** consultable

> **Pédagogie**
> Le Freestyle montre physiquement chaque brique (SCM, trigger, step, post-build). Le pipeline suivant reprend exactement les mêmes briques en code.

Création d'un job (2/2) : Pipeline et Multibranch Pipeline

Job Pipeline (Jenkinsfile dans le dépôt)

1. *New Item* → nom `library-api-pipeline` → *Pipeline*
2. **Pipeline → Definition** : *Pipeline script from SCM*
3. SCM : Git, URL, credential, branche `*/main`
4. **Script Path** : `Jenkinsfile` (ou `ci/Jenkinsfile`)
5. *Lightweight checkout* coché : Jenkins lit uniquement le Jenkinsfile pour démarrer
6. Save → Build Now

Pour apprendre : *Pipeline script* (éditeur intégré) permet d'itérer sans commit ; migrer ensuite vers le SCM.

Jenkinsfile minimal du fil rouge

```
pipeline {
  agent { label 'linux' }
  tools { jdk 'jdk21'; maven 'maven-3.9' }
  options { timestamps(); buildDiscarder(logRotator(numToKeepStr: '20')) }
  stages {
    stage('Build & Test') {
      steps { sh 'mvn -B clean verify' }
    }
  }
  post {
    always { junit 'target/surefire-reports/*.xml' }
    success { archiveArtifacts 'target/*.jar' }
  }
}
```

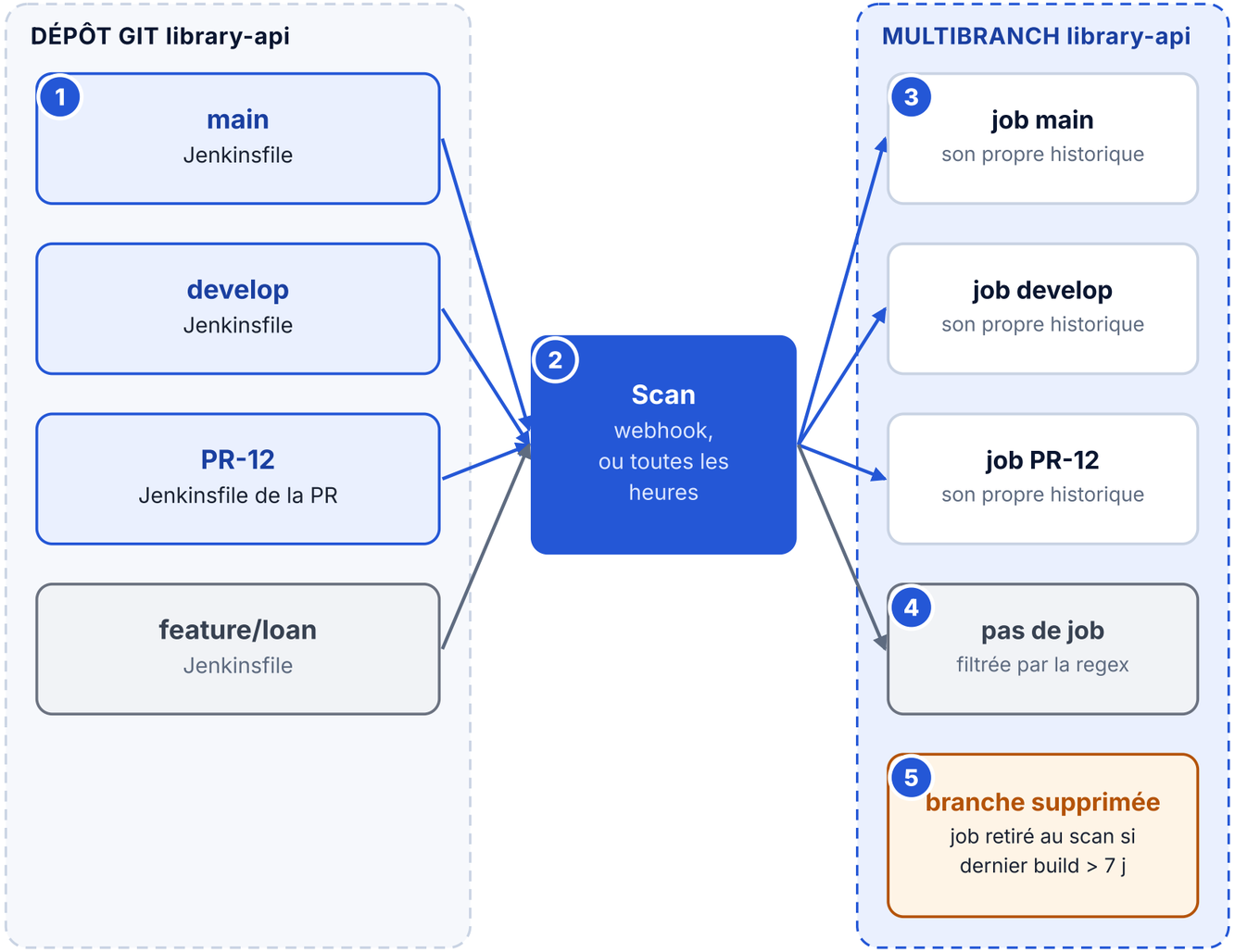
Multibranch Pipeline (recommandé)

1. *New Item* → nom `library-api` → *Multibranch Pipeline*
2. **Branch Sources** → *GitHub* (ou *GitLab*, *Git*) : credential, owner/repo
3. **Behaviours** : *Discover branches*, *Discover pull requests from origin*, filtre par regex (`main|develop|release/.*|PR-.*`)
4. **Build Configuration** : *by Jenkinsfile*, Script Path `Jenkinsfile`
5. **Scan Multibranch Pipeline Triggers** : *Periodically if not otherwise run* (1 h) en complément du webhook
6. **Orphaned Item Strategy** : supprimer, au scan suivant, les jobs des branches disparues dont le dernier build a plus de 7 jours
7. Save → *Scan Repository Now* → un job par branche apparaît et se construit

Ce qu'on obtient

Chaque push sur n'importe quelle branche et chaque PR déclenche le même pipeline ; `env.BRANCH_NAME` et `env.CHANGE_ID` (PR) permettent d'adapter le comportement (`when { branch 'main' }`).

Schéma : un Multibranch Pipeline, un job par branche



Comment le lire

1. Chaque branche et chaque Pull Request porte **son propre Jenkinsfile** : une PR peut modifier le pipeline qu'elle déclenche.
2. Le **scan** (déclenché par le webhook, ou périodiquement) découvre branches et PR.
3. Pour chacune, Jenkins crée un **job** qui a son historique et lance un build. `env.BRANCH_NAME` et `env.CHANGE_ID` disent où l'on est.
4. Les branches qui ne passent pas le filtre (ici `main|develop|release/.*|PR-.*`) n'ont pas de job.
5. Une branche supprimée laisse un job orphelin ; l'*Orphaned Item Strategy* le retire au scan suivant si son dernier build a plus de 7 jours.

À retenir

On configure **une fois** le Multibranch ; les jobs de branches apparaissent et disparaissent seuls.

Accès aux sources (1/2) : configuration Git

Protocoles

Protocole	Credential	Remarques
HTTPS	Username + **Personal Access Token** (pas le mot de passe)	Simple, traverse les proxys ; token à scope `repo` / `read_repository`
SSH	SSH Username with private key (clé dédiée à Jenkins, sans passphrase ou avec)	Clé publique déposée comme *Deploy key* du dépôt ; vérifier `known_hosts` (*Git Host Key Verification Configuration*)
GitHub App	App ID + clé privée	Recommandé pour GitHub : tokens courts, quotas API élevés, status checks

Options du SCM Git (Freestyle et Pipeline)

- **Branches to build** : `*/main`, `refs/tags/v*`, `${BRANCH}` (paramètre)
- **Additional Behaviours** : *Clean before checkout*, *Shallow clone* (depth 1 : plus rapide), *Checkout to sub-directory*, *Sparse checkout*, *Advanced submodule behaviours*, *Polling ignores commits in certain paths* (ex. `docs/.*`)

Dans le Jenkinsfile

```
// 1) Le SCM configuré dans le job : checkout implicite avant le
//    1er stage ; checkout scm explicite seulement avec
//    options { skipDefaultCheckout() }
stage('Checkout') {
  steps { checkout scm }
}

// 2) Checkout explicite, dépôt différent ou options
stage('Checkout') {
  steps {
    checkout scmGit(
      branches: [[name: 'refs/heads/main']],
      userRemoteConfigs: [[
        url: 'git@github.com:utopios/library-api.git',
        credentialsId: 'git-ssh-key']],
      extensions: [
        cloneOption(shallow: true, depth: 1, noTags: false),
        cleanBeforeCheckout()
      ])
  }
}

// 3) Récupérer des informations Git
stage('Info') {
  steps {
    script {
      env.GIT_SHORT = sh(script: 'git rev-parse --short HEAD',
                         returnStdout: true).trim()
      currentBuild.displayName = "#${BUILD_NUMBER} ${env.GIT_SHORT}"
    }
  }
}
```

Dans tout pipeline déclaratif chargé depuis le SCM (*Pipeline script from SCM*, Multibranch), le checkout est automatique avant le premier stage (avec `agent none` : dans chaque stage qui déclare un agent ; sauf `skipDefaultCheckout()`).

Accès aux sources (2/2) : branches, tags, PR et changements

Variables disponibles

Variable	Contenu
`GIT_COMMIT`	SHA complet
`GIT_BRANCH`	`origin/main` (job Pipeline simple)
`BRANCH_NAME`	`main`, `feature/x`, `PR-57` (Multibranch)
`TAG_NAME`	Nom du tag si build de tag
`CHANGE_ID`, `CHANGE_TARGET`, `CHANGE_AUTHOR`	Numéro, branche cible et auteur de la PR
`GIT_PREVIOUS_SUCCESSFUL_COMMIT`	Pour calculer un diff

Changements détectés

```
script {
  for (cs in currentBuild.changeSets) {
    for (e in cs.items) {
      echo "${e.author} - ${e.msg} (${e.affectedFiles.size()} fichiers)"
    }
  }
}
```

Adapter le pipeline à la branche

```
stage('Publish SNAPSHOT') {
  when { branch 'develop' }
  steps { sh 'mvn -B deploy -DskipTests' }
}
stage('Release') {
  when { tag pattern: 'v\\d+\\.\\d+\\.\\d+', comparator: 'REGEXP' }
  steps { sh 'mvn -B deploy -DskipTests -Drevision=${TAG_NAME#v}' }  // v1.4.0 → 1.4.0
}
stage('PR checks') {
  when { changeRequest target: 'main' }
  steps { sh 'mvn -B verify -P quick' }
}
stage('Deploy recette') {
  when {
    allOf { branch 'main'; not { changeRequest() } }
  }
  steps { sh './deploy.sh recette' }
}
```

Déclencher sur des chemins

Pour ignorer les commits touchant uniquement `docs/` : option *Polling ignores commits in certain paths* du SCM, ou `when { changeset "src/**" }` sur un stage.

Paramétrage de Maven (1/2) : goals, options, environnement

Freestyle : step « Invoke top-level Maven targets »

Champ	Exemple
Maven Version	`maven-3.9`
Goals	`-B clean verify`
POM	`pom.xml` ou `backend/pom.xml`
Properties	`skipITs=true` `revision=1.4.${BUILD_NUMBER}`
JVM Options	`-Xmx1g`
Settings file	*Settings file in filesystem* ou *provided Config File*
Use private Maven repository	Dépôt local par job/workspace (isolation, mais pas de cache partagé)

Variables Maven depuis Jenkins

- Toute variable Jenkins est accessible : `${BUILD_NUMBER}`, `${GIT_COMMIT}`, paramètres du job
- Côté POM, une propriété `${revision}` (CI-friendly versions) permet d'injecter la version depuis Jenkins

Pipeline : équivalent

```
pipeline {
  agent { label 'linux' }
  tools { jdk 'jdk21'; maven 'maven-3.9' }
  environment {
    MAVEN_OPTS = '-Xmx1g -Dmaven.repo.local=/var/cache/m2'
    MAVEN_ARGS = '-B -ntp -s /etc/maven/settings-nexus.xml'  // lu par Maven ≥ 3.9
    APP_VERSION = "1.4.${BUILD_NUMBER}"
  }
  stages {
    stage('Build') {
      steps {
        sh 'mvn -Drevision=$APP_VERSION clean verify'
      }
    }
  }
}
```

Versions CI-friendly dans le POM

```
<version>${revision}</version>
<properties>
  <revision>1.4.0-SNAPSHOT</revision>   <!— valeur locale —>
</properties>
<!— + flatten-maven-plugin pour publier un POM résolu —>
```

Le numéro de build Jenkins devient le numéro de version de l'artefact : traçabilité totale commit → build → artefact → déploiement.

Paramétrage de Maven (2/2) : withMaven et cache des dépendances

Plugin Pipeline Maven Integration

Le step `withMaven` configure Maven et **publie automatiquement** après l'exécution :

- Rapports JUnit (Surefire et Failsafe)
- Rapports JaCoCo
- Artefacts générés (JAR/WAR) + fingerprints
- Dépendances (pour déclencher les jobs aval qui en dépendent)
- Alertes Warnings NG (javac, Checkstyle, SpotBugs…)

```
stage('Build') {
  steps {
    withMaven(
        maven: 'maven-3.9',
        jdk: 'jdk21',
        mavenSettingsConfig: 'nexus-settings',   // Config File Provider
        mavenLocalRepo: '.repository',          // ou laissé par défaut
        options: [
          junitPublisher(disabled: false),
          jacocoPublisher(),
          artifactsPublisher(disabled: false)
        ]) {
      sh 'mvn clean verify'      // -B ajouté automatiquement
    }
  }
}
```

Accélérer : cache des dépendances

Contexte	Solution
Agent persistant (VM)	`~/.m2/repository` partagé par tous les builds de l'agent ; nettoyage périodique
Agent Docker	Volume nommé monté : `args '-v m2cache:/root/.m2'`
Agent Kubernetes	PVC monté dans le pod, ou dépôt proxy Nexus très proche (réseau rapide)
Tous	Dépôt proxy Nexus/Artifactory : téléchargement interne, pas d'accès Internet depuis les agents

Autres optimisations

- **Maven Wrapper** (`./mvnw`) : version Maven versionnée avec le projet, pas de dépendance à *Tools*
- `-T 1C` sur les projets multi-modules
- Séparer `mvn test` (rapide, chaque commit) et `mvn verify -P integration` (plus long, sur `main` ou la nuit)
- `-o` (offline) une fois le cache chaud pour éviter les vérifications réseau

Build paramétré

Pourquoi

- Choisir l'environnement cible, la version à déployer, activer un profil Maven, sauter les tests d'intégration…
- Le bouton *Build Now* devient **Build with Parameters** ; les valeurs sont disponibles comme variables d'environnement et dans `params.*`

Types de paramètres

Type	Exemple
`string`	Version, nom de branche
`text`	Multi-lignes (notes de release)
`booleanParam`	Sauter les tests d'intégration
`choice`	Environnement : dev / recette / prod
`password`	Saisie masquée (préférer les credentials)
`credentials`	Choix d'un credential existant
Git Parameter (plugin)	Liste des branches/tags du dépôt

```
pipeline {
  agent { label 'linux' }
  parameters {
    choice(name: 'TARGET_ENV', choices: ['dev', 'recette', 'prod'],
           description: 'Environnement de déploiement')
    string(name: 'ARTIFACT_VERSION', defaultValue: '1.4.0',
           description: 'Version à déployer depuis Nexus')
    booleanParam(name: 'RUN_IT', defaultValue: true,
           description: 'Exécuter les tests d\'intégration')
  }
  stages {
    stage('Tests d\'intégration') {
      when { expression { params.RUN_IT } }
      steps { sh 'mvn -B verify -P integration' }
    }
    stage('Deploy') {
      steps {
        echo "Déploiement de ${params.ARTIFACT_VERSION} sur ${params.TARGET_ENV}"
        sh "./deploy.sh ${params.TARGET_ENV} ${params.ARTIFACT_VERSION}"
      }
    }
  }
}
```

Première exécution

Comme pour `triggers`, les `parameters` déclarés dans le Jenkinsfile ne sont pris en compte dans l'UI qu'après un premier build (qui utilise les valeurs par défaut).

Lancement par API : `curl -X POST -u user:token "$JENKINS/job/deploy/buildWithParameters?TARGET_ENV=recette&ARTIFACT_VERSION=1.4.0"`

Rapports de tests unitaires (1/3) : Surefire et le format JUnit

Côté Maven

- **maven-surefire-plugin** exécute les tests unitaires en phase `test` : classes `*Test`, `Test*`, `*Tests`, `*TestCase`
- Produit `target/surefire-reports/TEST-*.xml` (format **JUnit XML**, standard de fait lu par tous les outils CI) et des `.txt`
- **maven-failsafe-plugin**, une fois déclaré dans le POM, exécute les tests d'intégration (`*IT`) en phases `integration-test` / `verify` et produit `target/failsafe-reports/*.xml`

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <!— tests JUnit 5 détectés via le starter Spring Boot —>
    <rerunFailingTestsCount>0</rerunFailingTestsCount>
    <trimStackTrace>false</trimStackTrace>
  </configuration>
</plugin>
```

Exemple de test du fil rouge

```
// src/test/java/fr/utopios/library/BookServiceTest.java
@ExtendWith(MockitoExtension.class)
class BookServiceTest {

  @Mock BookRepository repository;
  @InjectMocks BookService service;

  @Test
  void shouldRejectLoanWhenNoCopyAvailable() {
    when(repository.findById(42L))
        .thenReturn(Optional.of(new Book(42L, "Clean Code", 0)));

    assertThrows(NoCopyAvailableException.class,
        () -> service.loan(42L, "mohamed"));
  }

  @Test
  void shouldDecrementCopiesOnLoan() {
    when(repository.findById(1L))
        .thenReturn(Optional.of(new Book(1L, "DDD", 3)));

    service.loan(1L, "alice");

    verify(repository).save(argThat(b -> b.copies() == 2));
  }
}
```

Le rapport XML contient, par classe : nombre de tests, échecs, erreurs, ignorés, durée, et pour chaque échec le message et la stack trace.

Rapports de tests unitaires (2/3) : publication dans Jenkins

Le step `junit` (plugin JUnit)

```
stage('Test') {
  steps {
    // Ne pas faire échouer Maven : c'est Jenkins qui jugera
    sh 'mvn -B test -Dmaven.test.failure.ignore=true'
  }
  post {
    always {
      junit testResults: 'target/surefire-reports/*.xml',
            allowEmptyResults: false,   // pas de rapport = FAILURE
            skipPublishingChecks: false // publie aussi un check GitHub
    }
  }
}
stage('Integration tests') {
  // goals Failsafe seuls : les tests unitaires ne sont pas rejoués
  steps { sh 'mvn -B failsafe:integration-test failsafe:verify -Dmaven.test.failure.ignore=true' }
  post { always { junit 'target/failsafe-reports/*.xml' } }
}
```

- Placé dans `post { always }` : les rapports sont publiés même si l'étape échoue
- Un test en échec ⇒ build **UNSTABLE** (pas FAILURE) : la différence entre « ça ne compile pas » et « ça ne passe pas »
- Freestyle : *Post-build Actions → Publish JUnit test result report*

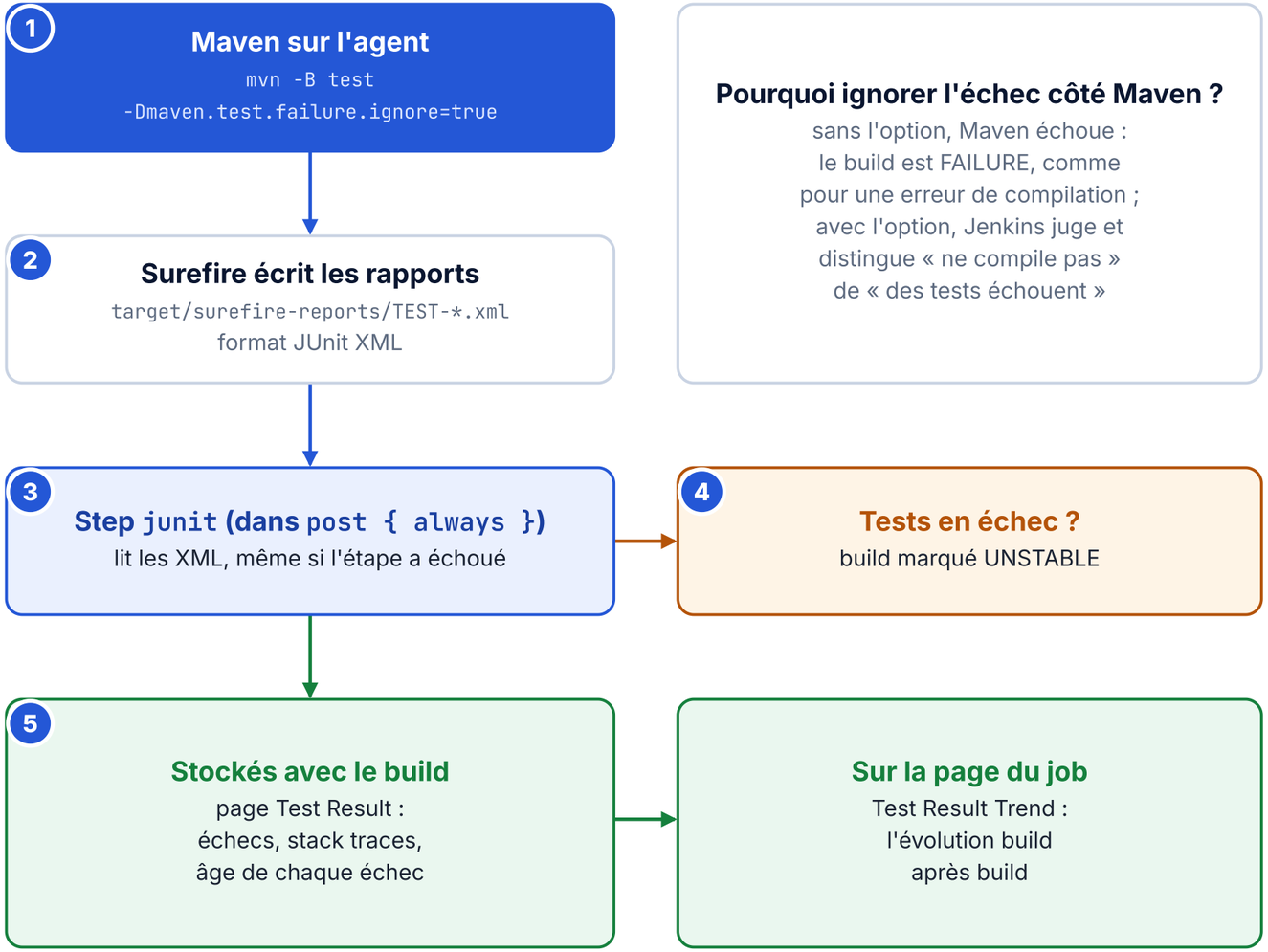
Ce que Jenkins affiche

- **Page du build → Test Result** : total, échecs, ignorés, durée ; liste des tests échoués avec message, stack trace, *stdout/stderr* capturés
- Navigation par package → classe → méthode ; **âge** d'un échec (depuis combien de builds il échoue)
- **Page du job → Test Result Trend** : graphique de l'évolution (succès / échecs / ignorés) build après build
- Résumé dans la console et dans les mails : « 2 tests failed »

Options utiles

Option	Effet
`healthScaleFactor: 2.0`	Poids des échecs dans la météo du job
`stdioRetention: 'ALL'`	Conserver toute la sortie des tests (`'FAILED'` : sortie complète seulement pour les tests en échec ; remplace `keepLongStdio`, déprécié)
`testDataPublishers`	Pièces jointes, captures (plugins)
Claim (plugin)	Un développeur « réclame » un test cassé pour le corriger

Schéma : des tests Maven au rapport Jenkins



Comment le lire

1. Maven exécute les tests sans faire échouer le build sur un test rouge : c'est Jenkins qui décidera.
2. Surefire écrit un fichier JUnit XML par classe de test, dans le workspace.
3. Le step `junit`, placé dans `post { always }`, lit ces fichiers et les rapatrie sur le controller.
4. S'il trouve des tests en échec, le build devient **UNSTABLE** (orange) au lieu de FAILURE.
5. Les résultats sont conservés avec le build et alimentent la tendance du job.

À retenir

Pas de `junit`, pas de rapport : le build n'est que vert ou rouge, sans savoir quels tests ont échoué.

Rapports de tests unitaires (3/3) : couverture de code avec JaCoCo

Côté Maven

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.13</version>
  <executions>
    <execution>              <!-- instrumente la JVM des tests -->
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>              <!-- génère target/site/jacoco -->
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

Côté Jenkins (plugin Coverage)

```
post {
  always {
    recordCoverage(
      tools: [[parser: 'JACOCO',
               pattern: 'target/site/jacoco/jacoco.xml']],
      sourceCodeRetention: 'LAST_BUILD',
      qualityGates: [
        [threshold: 80.0, metric: 'LINE',   baseline: 'PROJECT', criticality: 'UNSTABLE'],
        [threshold: 70.0, metric: 'BRANCH', baseline: 'PROJECT', criticality: 'UNSTABLE']
      ])
  }
}
```

- Onglet **Coverage** sur le build : lignes, branches, par package/fichier, code source coloré
- **Tendance** sur la page du job ; différentiel par rapport au build de référence (couverture du **nouveau code** sur une PR)
- Ici la porte est côté Jenkins : le *quality gate* marque UNSTABLE. L'alternative est une exécution `jacoco:check` dans le POM, qui fait échouer Maven (FAILURE) : choisir l'une ou l'autre, pas les deux

Lecture

La couverture mesure ce qui est *exécuté* par les tests, pas ce qui est *vérifié*. 80 % est un garde-fou, pas un objectif en soi.

Envoi de mails de notification (1/3) : configuration SMTP

Deux plugins

- **Mailer** (de base) : post-build « E-mail Notification », step `mail`. Simple, peu configurable.
- **Email Extension** (email-ext) : contenu HTML, templates, destinataires dynamiques (committers, culprits), pièces jointes, déclencheurs fins. **Recommandé.**

Manage Jenkins → System

Section	Champ	Valeur
Extended E-mail Notification	SMTP server	`smtp.example.com`
	SMTP Port	587 (STARTTLS) ou 465 (SSL)
	Credentials	Username/password SMTP
	Default Recipients / Reply-To	`dev-team@example.com`
E-mail Notification	SMTP server, port, auth	Idem ; bouton *Test configuration*
Jenkins Location	System Admin e-mail address	`jenkins@example.com` (expéditeur)

En formation : un serveur SMTP de test

```
# Mailpit : capture tous les mails, UI web sur :8025
docker run -d --name mailpit -p 1025:1025 -p 8025:8025 axllent/mailpit

# Dans Jenkins : SMTP server = mailpit (ou host.docker.internal)
#                SMTP port   = 1025, sans authentification ni TLS
```

Vérifier

- *E-mail Notification → Test configuration by sending test e-mail*
- Lire le mail dans Mailpit (`http://localhost:8025`)
- En cas d'échec : *System Log*, logger `hudson.plugins.emailext`

Adresses des committers

Email-ext peut résoudre l'auteur d'un commit en adresse mail : via l'adresse Git du commit, ou via l'annuaire (LDAP) si les utilisateurs Jenkins y sont liés.

Envoi de mails de notification (2/3) : dans le pipeline

Step `emailext` dans `post { }`

```
post {
  failure {
    emailext(
      subject: "[Jenkins] ÉCHEC ${env.JOB_NAME} #${env.BUILD_NUMBER}",
      body: '''${SCRIPT, template="groovy-html.template"}''',
      mimeType: 'text/html',
      recipientProviders: [culprits(), developers(), requestor()],
      to: 'dev-team@example.com',
      attachLog: true, compressLog: true)
  }
  unstable {
    emailext(
      subject: "[Jenkins] INSTABLE ${env.JOB_NAME} #${env.BUILD_NUMBER} - ${currentBuild.description ?: ''}",
      body: """<p>Tests en échec : voir <a href="${env.BUILD_URL}testReport/">le rapport</a>.</p>
               <pre>${'$'}{FAILED_TESTS}</pre>""",
      mimeType: 'text/html',
      recipientProviders: [culprits()])
  }
  fixed {      // le build repasse au vert
    emailext(subject: "[Jenkins] RÉPARÉ ${env.JOB_NAME}", body: env.BUILD_URL,
             recipientProviders: [developers()])
  }
}
```

Destinataires dynamiques (`recipientProviders`)

Provider	Qui
`developers()`	Auteurs des commits inclus dans ce build
`culprits()`	Auteurs des commits depuis le dernier build réussi (les suspects)
`requestor()`	Utilisateur qui a lancé le build manuellement
`brokenBuildSuspects()`	Committers du build qui a cassé
`upstreamDevelopers()`	Committers du job amont

Tokens disponibles dans sujet et corps

- `${BUILD_STATUS}`, `${PROJECT_NAME}`, `${BUILD_URL}`, `${CAUSE}`
- `${CHANGES}` : liste des commits ; `${FAILED_TESTS}` ; `${TEST_COUNTS}`
- `${BUILD_LOG, maxLines=50}` : fin de la console
- `${SCRIPT, template="…"}` : templates Groovy (`groovy-html.template` fourni)
- Liste : *Manage Jenkins → System → Extended E-mail → Content Token Reference*

Freestyle : *Post-build Actions → Editable Email Notification → Advanced Settings → Triggers* (Failure, Unstable, Fixed…).

Envoi de mails de notification (3/3) : stratégie et alternatives

Quoi notifier, à qui

Événement	Destinataires	Canal
Build **cassé** (FAILURE)	Culprits + équipe	Mail + Slack/Teams
Build **instable** (tests)	Culprits	Mail
Build **réparé** (fixed)	Équipe	Slack/Teams
Chaque build vert	Personne	Dashboard, badge, status check
Déploiement en production	Équipe + exploitation	Slack/Teams + mail de traçabilité
Approbation requise (`input`)	Responsable	Mail avec lien

Fatigue de notification

Un mail à chaque build vert finit filtré ; les vrais échecs passent inaperçus. Notifier les **changements d'état** (`changed` , `fixed` , `regression`), pas les états.

Slack

```
// plugin Slack Notification ; token dans un credential
post {
  failure {
    slackSend channel: '#ci-library',
              color: 'danger',
              message: "ÉCHEC ${env.JOB_NAME} #${env.BUILD_NUMBER} (<${env.BUILD_URL}|ouvrir>)"
  }
  fixed {
    slackSend channel: '#ci-library', color: 'good',
              message: "Réparé : ${env.JOB_NAME} #${env.BUILD_NUMBER}"
  }
}
```

Microsoft Teams

```
// Webhook « Workflows » de Teams (les Office 365 Connectors sont
// arrêtés depuis mai 2026) ; plugin HTTP Request ; URL en credential
post {
  always {
    withCredentials([string(credentialsId: 'teams-webhook', variable: 'URL')]) {
      httpRequest url: URL, httpMode: 'POST', contentType: 'APPLICATION_JSON',
        requestBody: """{"type":"message","attachments":[{
          "contentType":"application/vnd.microsoft.card.adaptive",
          "content":{"type":"AdaptiveCard","version":"1.4","body":[{"type":"TextBlock",
          "text":"Build ${currentBuild.currentResult} : ${env.BUILD_URL}"}]}}]}"""
    }
  }
}
```

Statut dans le dépôt

Les plugins GitHub/GitLab Branch Source publient automatiquement le résultat comme **status check** sur le commit : c'est la notification la plus visible pour le développeur, directement dans sa PR.

Déploiement automatique (1/4) : stratégies

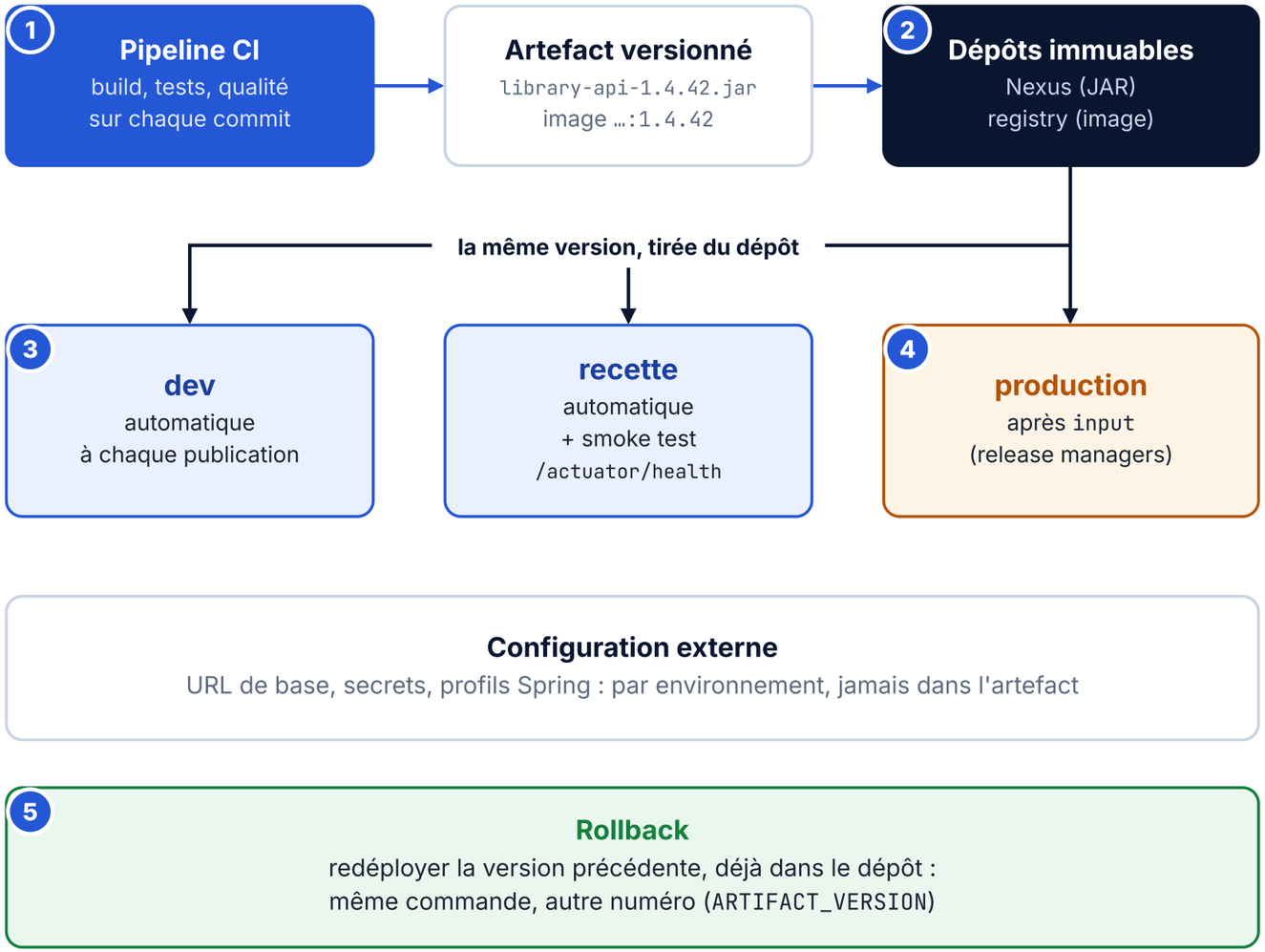
Que déploie-t-on ?

Artefact	Cible	Mécanisme
JAR exécutable Spring Boot	VM / serveur Linux	SCP + systemd (plugin Publish Over SSH, `sshagent`)
WAR	Tomcat / serveur d'applications	Manager Tomcat (plugin Deploy to container), copie dans `webapps/`
Image Docker	Hôte Docker, Swarm	`docker build/push` puis `docker compose up -d` à distance
Image Docker + chart Helm	Kubernetes	`helm upgrade --install`, `kubectl set image`, ou GitOps (Argo CD)
JAR / POM	Nexus / Artifactory	`mvn deploy` (publication, pas exécution)
Infrastructure	Cloud	Terraform, Ansible, AWS CLI depuis le pipeline

Principes

- **Publier avant de déployer** : l'artefact est d'abord stocké dans un dépôt immuable, puis déployé *depuis* ce dépôt. On déploie une version, pas un fichier du workspace.
- **Même artefact pour tous les environnements** : la configuration (URL BDD, secrets) est externalisée (variables d'environnement, profils Spring, ConfigMaps)
- **Déploiement idempotent et scripté** : `deploy.sh <env> <version>` ou playbook Ansible ; le pipeline ne fait que l'appeler
- **Promotion** : dev automatique → recette automatique → production après `input` (livraison continue) ou automatique (déploiement continu)
- **Rollback** : redéployer la version précédente doit être aussi simple qu'un déploiement (paramètre `ARTIFACT_VERSION`)
- **Smoke test** après déploiement : appel `/actuator/health`, échec du pipeline si KO

Schéma : publier une fois, déployer partout



Comment le lire

1. La CI produit un artefact **versionné** (JAR, image) à partir d'un commit.
2. Il est **publié** dans un dépôt immuable avant tout déploiement : on déploie une version, pas un fichier du workspace.
3. Chaque environnement reçoit **la même version**, tirée du dépôt : dev et recette automatiquement, avec un smoke test.
4. La production attend une approbation (`input`) en livraison continue ; elle est automatique en déploiement continu.
5. Revenir en arrière, c'est redéployer la version précédente : elle est encore dans le dépôt.

À retenir

Ce qui change d'un environnement à l'autre, c'est la **configuration**, pas le binaire.

Déploiement automatique (2/4) : publier dans Nexus

POM : `distributionManagement`

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>http://nexus:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>http://nexus:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

Les `id` correspondent aux `<server>` du `settings.xml` (slide Rappels Maven) qui portent les credentials.

Côté Nexus

- Dépôts hosted `maven-releases` (Deployment policy : *Disable redeploy*) et `maven-snapshots` (*Allow redeploy*)
- Group `maven-public` = releases + snapshots + proxy central
- Utilisateur technique `jenkins` avec un rôle regroupant les privileges *nx-repository-view-maven2-*-add*, *-edit* et *-read* (Maven met aussi à jour `maven-metadata.xml`)

Pipeline

```
stage('Publish to Nexus') {
  when { anyOf { branch 'main'; branch 'develop' } }
  environment {
    NEXUS = credentials('nexus-deploy')   // → NEXUS_USR / NEXUS_PSW
    // main : release 1.4.<build> ; develop : SNAPSHOT
    REVISION = "${env.BRANCH_NAME == 'main' ? env.APP_VERSION : '1.4.0-SNAPSHOT'}"
  }
  steps {
    configFileProvider([configFile(fileId: 'nexus-settings',
                                   variable: 'SETTINGS')]) {
      sh 'mvn -B -s $SETTINGS -Drevision=$REVISION -DskipTests deploy'
    }
  }
}
```

Alternative sans Maven : plugin Nexus Artifact Uploader

```
nexusArtifactUploader(
  nexusVersion: 'nexus3', protocol: 'http', nexusUrl: 'nexus:8081',
  groupId: 'fr.utopios', version: env.APP_VERSION,
  repository: 'maven-releases', credentialsId: 'nexus-deploy',
  artifacts: [[artifactId: 'library-api', classifier: '',
               file: "target/library-api-${env.APP_VERSION}.jar", type: 'jar']])
```

Résultat

L'artefact `fr.utopios:library-api:1.4.42` est stocké, versionné, téléchargeable par URL : `…/maven-releases/fr/utopios/library-api/1.4.42/library-api-1.4.42.jar`. Les jobs de déploiement le récupèrent par sa version.

Déploiement automatique (3/4) : vers un serveur (SSH) et Docker

Déploiement SSH d'un JAR Spring Boot

```
stage('Deploy recette') {
  when { branch 'main' }
  environment { HOST = 'app@recette.example.com' }
  steps {
    sshagent(credentials: ['deploy-ssh-key']) {
      sh '''
        set -e
        JAR=target/library-api-${APP_VERSION}.jar
        # host key de recette déjà dans ~/.ssh/known_hosts de l'agent
(empreinte vérifiée)
        scp $JAR $HOST:/opt/library-api/releases/
        ssh $HOST "ln -sfn /opt/library-api/releases/library-
api-${APP_VERSION}.jar \\
                           /opt/library-api/current.jar \\
                  && sudo systemctl restart library-api"
      '''
    }
  }
}
stage('Smoke test') {
  steps {
    sh 'for i in $(seq 1 30); do curl -sf
https://recette.example.com/actuator/health && exit 0; sleep 2; done;
exit 1'
  }
}
```

Sur le serveur : unité systemd `library-api.service` pointant sur `current.jar` ; l'utilisateur `app` a le droit sudo limité à `systemctl restart library-api`. Rollback = changer le lien symbolique.

Build et push d'une image Docker

```
// REGISTRY et APP_VERSION : dans l'environment global du pipeline
stage('Docker image') {
  when { branch 'main' }
  steps {
    script {
      def image = docker.build("${REGISTRY}/library-api:${APP_VERSION}",
                               "--build-arg JAR=target/library-api-${APP_VERSION}.jar .")
      docker.withRegistry("https://${REGISTRY}", 'docker-registry') {
        image.push()
        image.push('latest')
      }
    }
  }
}
stage('Deploy Kubernetes') {
  steps {
    withKubeConfig([credentialsId: 'kubeconfig-recette']) {
      sh "kubectl -n library set image deployment/library-api api=${REGISTRY}/library-api:${APP_VERSION}"
      sh 'kubectl -n library rollout status deployment/library-api --timeout=120s'
    }
  }
}
```

Dockerfile du fil rouge

```
FROM eclipse-temurin:21-jre-alpine
ARG JAR
COPY ${JAR} /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Déploiement automatique (4/4) : environnements et approbation

Approbation manuelle avec `input`

```
stage('Approve production') {
  when { branch 'main' }
  options { timeout(time: 24, unit: 'HOURS') }   // sinon le build attend indéfiniment
  agent none          // libère l'executor seulement si le pipeline est en agent none
  steps {
    script {
      def r = input(
        message: "Déployer ${APP_VERSION} en production ?",
        ok: 'Déployer',
        submitter: 'release-managers',        // groupe/utilisateurs autorisés
        submitterParameter: 'APPROVER',
        parameters: [string(name: 'TICKET', description: 'Référence du change')])
      env.APPROVER = r.APPROVER     // input renvoie une Map : APPROVER + TICKET
      env.TICKET = r.TICKET
    }
  }
}
stage('Deploy production') {
  when { branch 'main' }
  steps {
    echo "Approuvé par ${env.APPROVER} (${env.TICKET})"
    sh "./deploy.sh prod ${APP_VERSION}"
  }
}
```

- Le build se met en pause ; un lien *Proceed / Abort* apparaît dans l'UI. `input` n'envoie aucun mail : envoyer soi-même le lien `${BUILD_URL}input` avant
- `agent none` au niveau pipeline + agents par stage : l'attente ne consomme pas d'executor

Verrous et ordonnancement

```
// plugin Lockable Resources : un seul déploiement à la fois sur un env
stage('Deploy recette') {
  options { lock(resource: 'env-recette') }
  steps { sh './deploy.sh recette ${APP_VERSION}' }
}
// plugin Pipeline Milestone : abandonne les builds plus anciens
stage('Deploy production') {
  steps {
    milestone 100
    sh './deploy.sh prod ${APP_VERSION}'
  }
}
```

Organisation recommandée

- **Un pipeline CI** par dépôt : build, tests, qualité, publication de l'artefact
- **Un pipeline CD** (paramétré : `ARTIFACT_VERSION`, `TARGET_ENV`) qui déploie depuis Nexus/registry – déclenché par le pipeline CI (`build job: 'deploy', parameters: […]`) ou manuellement
- Les droits de lancer le pipeline CD en `prod` sont restreints (Habilitations). Attention : un build lancé par `build job:` tourne par défaut en SYSTEM ; pour que ces droits s'appliquent réellement, plugin **Authorize Project** et `input` dans le job CD lui-même
- Chaque déploiement est tracé : build Jenkins, version, approbateur, ticket

Rapports d'analyse qualité (1/3) : SonarQube

Ce que SonarQube apporte

- Analyse statique multi-langages : **bugs**, **vulnérabilités**, **security hotspots**, **code smells**, duplication, complexité
- Agrège la **couverture** (JaCoCo) et les résultats de tests
- **Quality Profile** : jeu de règles ; **Quality Gate** : conditions de réussite (ex. *Sonar way* : aucune nouvelle issue, tous les nouveaux Security Hotspots revus, couverture du nouveau code ≥ 80 %, duplication du nouveau code ≤ 3 %)
- Vision « **Clean as You Code** » : les seuils portent sur le *nouveau code*, pas sur la dette historique
- Décoration des PR (édition Developer+) : commentaires directement dans GitHub/GitLab

Installation (formation)

```
docker run -d --name sonarqube -p 9000:9000 \
  -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true sonarqube:community
# http://localhost:9000 – admin / admin (à changer)
# My Account → Security → Generate Token (type : Global analysis)
```

Configuration côté Jenkins

1. Installer le plugin **SonarQube Scanner for Jenkins**
2. Credential *Secret text* `sonar-token` avec le token généré
3. *Manage Jenkins* → *System* → *SonarQube servers* : name `sonarqube`, URL `http://sonarqube:9000`, token = `sonar-token` ; cocher *Environment variables*
4. Côté SonarQube : *Administration* → *Configuration* → *Webhooks* → URL `http://jenkins:8080/sonarqube-webhook/` (nécessaire pour `waitForQualityGate`)

Côté Maven

- Aucun plugin à déclarer obligatoirement : `mvn sonar:sonar` résout `sonar-maven-plugin` (idéalement épinglé dans `<pluginManagement>`)
- Propriétés : `sonar.projectKey`, `sonar.coverage.jacoco.xmlReportPaths`, `sonar.java.source`

Nommage

Le `projectKey` suit le GAV : `fr.utopios:library-api`. Une branche/PR analysée séparément nécessite l'édition Developer (ou le plugin communautaire *sonarqube-community-branch-plugin*).

Rapports d'analyse qualité (2/3) : analyse et Quality Gate dans le pipeline

```
stage('SonarQube analysis') {
  steps {
    withSonarQubeEnv('sonarqube') {       // injecte SONAR_HOST_URL et le token
      sh '''
        mvn -B sonar:sonar \
          -Dsonar.projectKey=fr.utopios:library-api \
          -Dsonar.projectName=library-api \
          -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml \
          -Dsonar.junit.reportPaths=target/surefire-reports
      '''
    }
  }
}

stage('Quality Gate') {
  steps {
    timeout(time: 10, unit: 'MINUTES') {
      // attend le webhook SonarQube ; abortPipeline: true ⇒ FAILURE si gate KO
      waitForQualityGate abortPipeline: true
    }
  }
}
```

Ordre des stages

L'analyse Sonar doit s'exécuter **après** les tests et le rapport JaCoCo (sinon couverture à 0 %). Chaîne typique : `mvn verify` → `mvn sonar:sonar` → `waitForQualityGate`.

Ce que l'on obtient

- Dans la console : lien vers le tableau de bord du projet SonarQube
- Icône SonarQube sur la page du build (statut du Quality Gate)
- Build FAILURE (ou UNSTABLE si `abortPipeline: false` + test de `qg.status`) quand le gate est rouge
- Tableau de bord SonarQube : nouveaux problèmes par sévérité, couverture nouveau code, dette estimée, historique

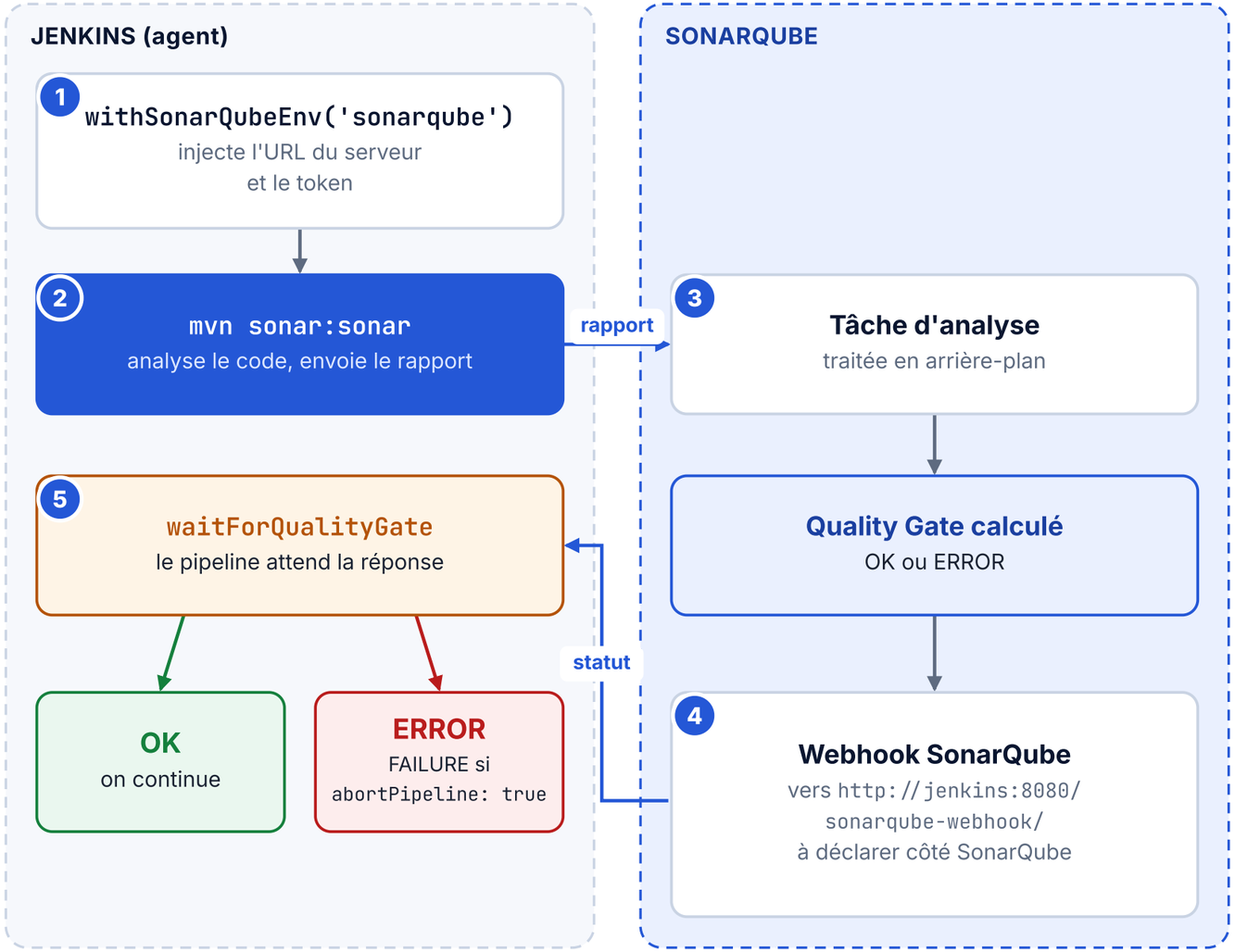
Variante avec gestion fine

```
stage('Quality Gate') {
  steps {
    script {
      def qg = waitForQualityGate()
      if (qg.status ≠ 'OK') {
        unstable("Quality Gate : ${qg.status}")
      }
    }
  }
}
```

Freestyle

Build Environment → *Prepare SonarQube Scanner environment*, puis step Maven `sonar:sonar`, puis *Post-build* → *Quality Gates* (plugin additionnel) ou step suivant.

Schéma : l'aller-retour Jenkins ↔ SonarQube



Comment le lire

1. `withSonarQubeEnv` fournit à Maven l'URL du serveur et le token déclarés dans Jenkins.
2. `mvn sonar:sonar` analyse le code (après les tests et JaCoCo, pour avoir la couverture) et envoie le rapport.
3. SonarQube traite le rapport **en arrière-plan**, puis calcule le Quality Gate.
4. Il prévient Jenkins par son **webhook** : sans ce webhook, l'attente ne se termine que si l'analyse était déjà finie au démarrage du step (d'où le `timeout`).
5. `waitForQualityGate` reçoit le statut : OK, le pipeline continue ; ERROR, il échoue avec `abortPipeline: true`.

À retenir

Si le webhook ne revient jamais, `waitForQualityGate` attend indéfiniment : toujours l'entourer d'un `timeout` (10 min dans le fil rouge).

Rapports d'analyse qualité (3/3) : Warnings NG, sécurité des dépendances

Warnings Next Generation

- Agrège dans Jenkins les alertes de dizaines d'outils : **javac**, **Checkstyle**, **PMD**, **SpotBugs**, **ESLint**, **Trivy**, **OWASP Dependency-Check**, **Maven console**…
- Onglet par outil sur le build, tendance, **nouvelles alertes vs. alertes corrigées**, seuils (quality gates)

```
stage('Static analysis') {
  steps {
    // spotbugs-maven-plugin (com.github.spotbugs) doit être déclaré dans le POM
    sh 'mvn -B checkstyle:checkstyle pmd:pmd spotbugs:spotbugs'
  }
  post {
    always {
      recordIssues(
        tools: [java(), checkStyle(), pmdParser(), spotBugs(useRankAsPriority: true)],
        qualityGates: [[threshold: 1, type: 'NEW_HIGH', criticality: 'UNSTABLE']],
        enabledForFailure: true)
    }
  }
}
```

Dépendances vulnérables : OWASP Dependency-Check

```
<!— pom.xml —>
<plugin>
  <groupId>org.owasp</groupId>
  <artifactId>dependency-check-maven</artifactId>
  <version>12.1.3</version>
  <configuration>
    <failBuildOnCVSS>7</failBuildOnCVSS>
    <formats><format>HTML</format><format>XML</format></formats>
    <nvdApiKey>${env.NVD_API_KEY}</nvdApiKey>
  </configuration>
</plugin>
```

```
stage('Dependency check') {
  environment { NVD_API_KEY = credentials('nvd-api-key') }
  steps { sh 'mvn -B dependency-check:check' }
  post {
    always {
      dependencyCheckPublisher pattern: 'target/dependency-check-report.xml'
      publishHTML target: [reportDir: 'target', reportFiles: 'dependency-check-report.html',
                           reportName: 'Dependency-Check', keepAll: true]
    }
  }
}
```

- Base NVD volumineuse : la mettre en cache (`dataDirectory` partagé) ; exécuter en nightly plutôt qu'à chaque commit
- Alternative image : `trivy image registry.example.com/utopios/library-api:1.4.42` avant le déploiement

Habilitations (1/4) : authentification et autorisation

Deux questions distinctes (*Manage Jenkins → Security*)

Security Realm – qui êtes-vous ?

- **Jenkins' own user database** : comptes locaux ; décocher *Allow users to sign up*
- **LDAP** / **Active Directory** : annuaire d'entreprise, groupes réutilisables dans les autorisations
- **SSO** : OpenID Connect (Keycloak, Entra ID), SAML, GitHub/GitLab OAuth – plugins dédiés
- **Unix user/group (PAM)**

Authorization Strategy – que pouvez-vous faire ?

- **Anyone can do anything** : jamais
- **Logged-in users can do anything** : défaut après l'assistant d'installation, insuffisant ; **Legacy mode** : héritage, à ne pas utiliser
- **Matrix-based security** : permissions globales par utilisateur/groupe
- **Project-based Matrix Authorization** : matrice globale + matrices par job/folder
- **Role-Based Strategy** (plugin) : rôles globaux, par item (regex), par agent

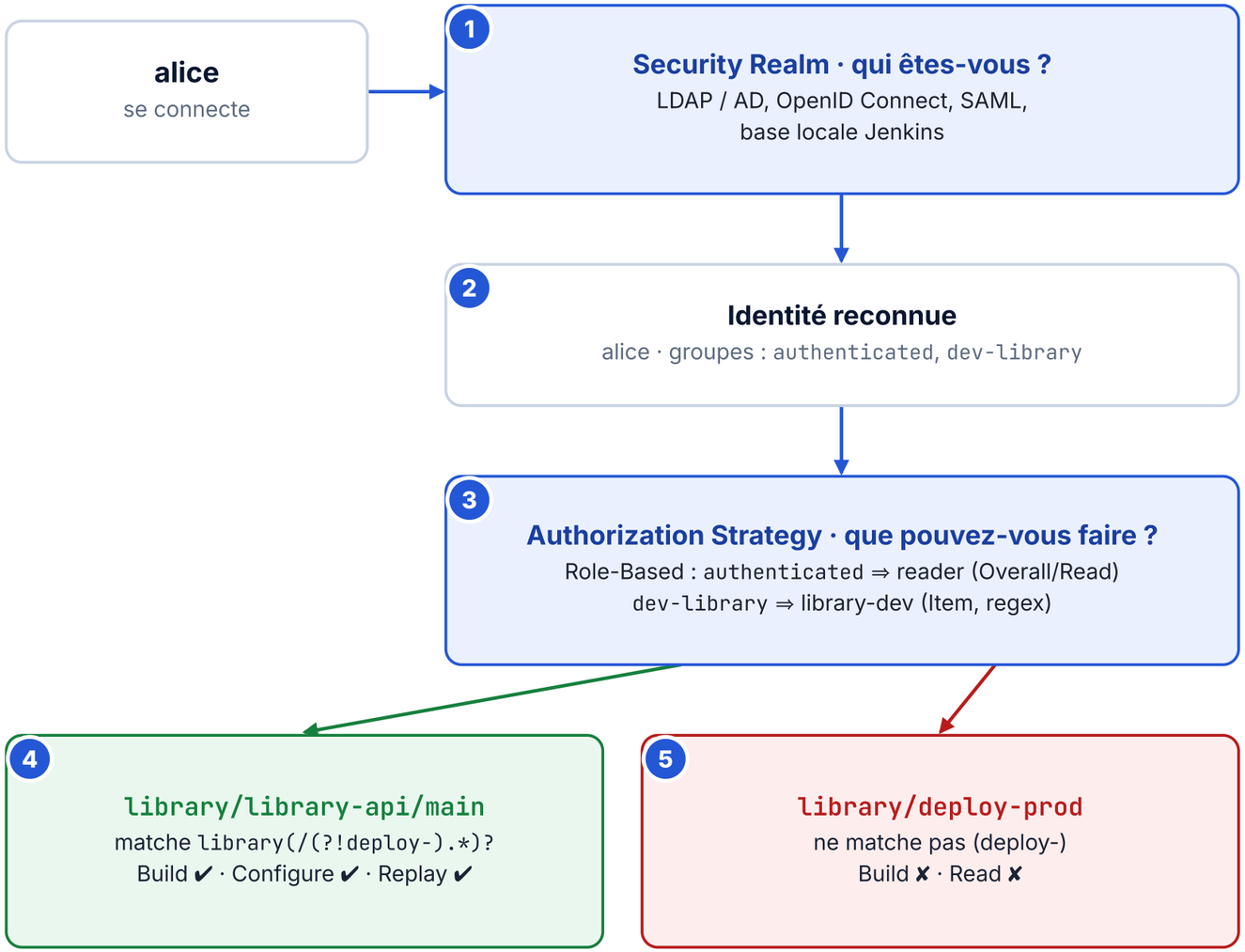
Autres réglages de sécurité

- **CSRF Protection** : activé (crumb) ; les appels API utilisent un token d'API
- **Agent → Controller** : protection toujours active depuis 2.326 (ce n'est plus un réglage) : un agent compromis ne peut pas lire le controller
- **Markup Formatter** : *Safe HTML* (évite le XSS dans les descriptions)
- **API tokens** par utilisateur, révocables ; jamais de mot de passe dans les scripts
- **Script Security** : les pipelines tournent dans une **sandbox Groovy** ; les appels non autorisés passent par *In-process Script Approval* (administrateur)
- **Controller sans executor** : un build ne doit jamais s'exécuter sur le controller

Groupes types

Groupe	Besoins
`jenkins-admins`	Tout, y compris plugins, nodes, Script Console
`dev-library`	Lire, lancer, configurer les jobs de leur folder
`release-managers`	Approuver `input`, lancer le CD prod
`viewers` / stagiaires	Lecture seule (`Overall/Read` + `Job/Read`)

Schéma : authentification, puis autorisation



Comment le lire

1. Le **Security Realm** vérifie l'identité (annuaire, SSO, base locale).
2. Il renvoie l'utilisateur et ses **groupes** : ce sont eux qu'on utilise ensuite, pas les noms de personnes.
3. L'**Authorization Strategy** transforme groupes en permissions : ici, un rôle global de lecture et un rôle d'item limité par une regex.
4. Sur les jobs dont le nom matche la regex, alice peut lancer, configurer, rejouer.
5. Sur `deploy-prod`, exclu par la regex, elle n'a rien : elle ne voit même pas le job.

À retenir
Deux réglages indépendants : changer d'annuaire ne change pas les droits, et inversement.

Habilitations (2/4) : Matrix-based security

Principales permissions

Catégorie	Permissions
Overall	`Administer` (tout), `Read` (accès à l'UI), `Manage` (par défaut depuis 2.542) ; `SystemRead` (à activer : `-Djenkins.security.SystemReadPermission=true` ou plugin Extended Read Permission)
Credentials	Create, Delete, ManageDomains, Update, View
Agent	Build, Configure, Connect, Create, Delete, Disconnect
Job	`Build`, `Cancel`, `Configure`, `Create`, `Delete`, `Discover`, `Move`, `Read`, `Workspace`
Run	Delete, Replay, Update (description)
View	Configure, Create, Delete, Read
SCM	Tag

- `Overall/Read` est le minimum pour se connecter
- `Job/Discover` : un anonyme qui ouvre l'URL d'un job est redirigé vers le login au lieu d'un 404 ; ne liste pas les jobs (il faut `Job/Read`)
- `Job/Workspace` et `Run/Replay` donnent accès à des données sensibles (fichiers, secrets) : à restreindre

Exemple de matrice globale

Utilisateur / groupe	Overall	Job	Credentials	Agent
`jenkins-admins`	Administer	—	—	—
`authenticated`	Read	Discover	—	—
`dev-library`	Read	Build, Cancel, Read, Workspace	View	—
`release-managers`	Read	Build, Cancel, Read	—	—
`anonymous`	—	—	—	—

Project-based Matrix : affiner par job / folder

- Dans la configuration d'un **Folder** : *Properties* → *Enable project-based security* → matrice locale
- **Héritage** : *Inherit permissions from parent* (par défaut), *Inherit globally defined permissions only*, ou *Do not inherit*
- Permet : `dev-library` a `Job/Configure` dans le folder `library` seulement ; `dev-mobile` ne voit pas ce folder

Ne jamais se verrouiller

Avant de sauvegarder une matrice, vérifier qu'un administrateur garde `Overall/Administer`. Sinon : désactiver la sécurité dans `config.xml` (`<useSecurity>false</useSecurity>`) et redémarrer.

Habilitations (3/4) : Role-Based Authorization Strategy

Principe

- Plugin **Role-based Authorization Strategy** : on définit des **rôles** (ensembles de permissions) puis on **assigne** des utilisateurs/groupes aux rôles
- Trois niveaux : **Global roles**, **Item roles** (appliqués aux items dont le nom complet matche une **regex** ; le folder lui-même doit matcher, sinon ses jobs sont inaccessibles), **Agent roles**
- Plus lisible que la matrice quand il y a beaucoup d'équipes ; s'appuie sur une convention de nommage (`library/…`, `mobile/…`)

Manage Jenkins → Manage and Assign Roles

Rôle	Type / Pattern	Permissions
`admin`	Global	Overall/Administer
`reader`	Global	Overall/Read (se connecter)
`viewer`	Global	Overall/Read, Job/Read
`library-dev`	Item – `library(/(?!deploy-).*)?`	Job/Build, Cancel, Read, Configure, Workspace ; Run/Replay
`library-read`	Item – `library(/.*)?`	Job/Read
`library-release`	Item – `library/deploy-.*`	Job/Build, Read
`mobile-dev`	Item – `mobile(/.*)?`	Job/Build, Cancel, Read, Configure
`linux-agents`	Agent – `linux-.*`	Agent/Build, Connect, Disconnect

Assignation

Groupe (LDAP) / utilisateur	Rôles
`jenkins-admins`	admin
`authenticated`	reader
`viewers`	viewer
`dev-library`	library-dev
`release-managers`	library-read, library-release
`dev-mobile`	mobile-dev

En JCasC (extrait : admin, reader, library-dev)

```
jenkins:
  authorizationStrategy:
    roleBased:
      roles:
        global:
          - name: admin
            permissions: ["Overall/Administer"]
            entries: [{ group: jenkins-admins }]
          - name: reader
            permissions: ["Overall/Read"]
            entries: [{ group: authenticated }]
        items:
          - name: library-dev
            pattern: "library(/(?!deploy-).*)?"
            permissions: ["Job/Build","Job/Cancel","Job/Read","Job/Configure",
                          "Job/Workspace","Run/Replay"]
            entries: [{ group: dev-library }]
```

Une assignation sur un *item role* ne donne pas `Overall/Read` : toujours combiner avec un rôle global de lecture.

Habilitations (4/4) : ce qu'un rôle peut ou ne peut pas faire

Action	Viewer (`viewer`)	Développeur (`library-dev`)	Release manager (`library-read` + `library-release`)	Administrateur
Voir le dashboard et les jobs `library/*`	✔ (Job/Read)	✔ sauf `deploy-*`	✔	✔
Lire la console et les rapports de tests	✔	✔ sauf `deploy-*`	✔	✔
Lancer un build `library/main`	✘	✔	✘	✔
Lancer `library/deploy-prod`	✘	✘	✔	✔
Approuver un `input` (`submitter: 'release-managers'`)	✘	✘	✔	✔
Modifier la configuration d'un job / le Jenkinsfile dans l'UI	✘	✔ (Job/Configure)	✘	✔
Consulter le workspace / Replay avec un script modifié	✘	✔ (à restreindre si secrets)	✘	✔
Voir les credentials (valeurs)	✘	✘ (jamais en clair, même avec Credentials/View)	✘	✘ en UI ; possible via Script Console
Créer un job dans `library/`	✘	✘ sans Job/Create	✘	✔
Installer un plugin, ajouter un node, Script Console	✘	✘	✘	✔
Voir les jobs `mobile/*`	✔ (Job/Read global)	✘	✘	✔

Limites à connaître

`Job/Configure` permet d'éditer un pipeline inline qui s'exécute avec les credentials du job : quiconque configure un job peut en extraire les secrets accessibles. Les credentials sensibles (prod) vont dans un folder séparé auquel les développeurs n'ont pas `Configure`, et le Jenkinsfile vient du SCM (revu en PR), pas de l'UI. En Multibranch, toute branche poussée s'exécute avec son propre Jenkinsfile, non relu : les credentials prod ne doivent pas être accessibles aux jobs de branche. Et un `build job: 'library/deploy-prod'` lancé depuis un pipeline tourne en SYSTEM, quelles que soient ces permissions : plugin **Authorize Project**.

Pipeline complet du fil rouge `library-api` — le code

```groovy
pipeline {
  agent none              // chaque stage prend son agent : l'attente de l'input n'en bloque aucun
  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '30', artifactNumToKeepStr: '10'))
    skipStagesAfterUnstable()           // tests rouges (UNSTABLE) : ni publication ni déploiement
  }
  environment {
    APP_VERSION = "1.4.${BUILD_NUMBER}"
    REGISTRY    = 'registry.example.com/utopios'
  }
  stages {
    stage('CI') {
      agent { label 'linux' }           // un seul agent, un seul workspace pour les stages imbriqués
      tools { jdk 'jdk21'; maven 'maven-3.9' }
      options { timeout(time: 30, unit: 'MINUTES') }
      stages {
        stage('Info') { steps {
          script { currentBuild.displayName = "#${BUILD_NUMBER} ${env.GIT_COMMIT.take(7)}" } } }
        stage('Build & tests') {
          steps { sh 'mvn -B -ntp -Drevision=$APP_VERSION clean verify -Dmaven.test.failure.ignore=true' }
          post { always {
            junit 'target/surefire-reports/*.xml, target/failsafe-reports/*.xml'
            recordCoverage tools: [[parser: 'JACOCO', pattern: 'target/site/jacoco/jacoco.xml']]
          } }
        }
        stage('Quality') {
          when { not { changeRequest() } }
          steps {
            withSonarQubeEnv('sonarqube') {
              sh 'mvn -B -ntp sonar:sonar -Dsonar.projectKey=fr.utopios:library-api'
            }
            timeout(time: 10, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
          }
        }
        stage('Publish') {
          when { branch 'main' }
          environment { NEXUS = credentials('nexus-deploy') }   // lu par le settings.xml
          steps {
            configFileProvider([configFile(fileId: 'nexus-settings', variable: 'SETTINGS')]) {
              sh 'mvn -B -ntp -s $SETTINGS -Drevision=$APP_VERSION -DskipTests deploy'
            }
            archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            script {
              def img = docker.build("${REGISTRY}/library-api:${APP_VERSION}",
                                     "--build-arg JAR=target/library-api-${APP_VERSION}.jar .")
              docker.withRegistry("https://${REGISTRY}", 'docker-registry') { img.push() }
            }
          }
        }
        stage('Deploy recette') {
          when { branch 'main' }
          options { lock('env-recette') }
          steps {
            withKubeConfig([credentialsId: 'kubeconfig-recette']) {
              sh "kubectl -n library set image deployment/library-api api=${REGISTRY}/library-api:${APP_VERSION}"
              sh 'kubectl -n library rollout status deployment/library-api --timeout=120s'
            }
            sh 'curl -sf https://recette.example.com/actuator/health'
          }
        }
      }
      post { cleanup { cleanWs() } }
    }
    stage('Production') {               // sans agent : l'attente ne consomme aucun executor
      when { branch 'main' }
      steps {
        timeout(time: 24, unit: 'HOURS') {
          input message: "Déployer ${APP_VERSION} en production ?", ok: 'Déployer',
                submitter: 'release-managers'
        }
        milestone 100                   // une version plus ancienne ne passe plus après une récente
        build job: 'library/deploy-prod',
              parameters: [string(name: 'ARTIFACT_VERSION', value: env.APP_VERSION)]
      }
    }
  }
  post {
    failure  { emailext subject: "[Jenkins] ÉCHEC ${JOB_NAME} #${BUILD_NUMBER}",
                        body: '${SCRIPT, template="groovy-html.template"}', mimeType: 'text/html',
                        recipientProviders: [culprits(), requestor()], to: 'dev-team@example.com' }
    unstable { emailext subject: "[Jenkins] INSTABLE ${JOB_NAME} #${BUILD_NUMBER}",
                        body: '${FAILED_TESTS}', recipientProviders: [culprits()] }
    fixed    { slackSend channel: '#ci-library', color: 'good', message: "Réparé : ${JOB_NAME}" }
  }
}
```

Jenkinsfile complet (~90 lignes), présenté sur deux colonnes : lire la colonne de gauche puis celle de droite.

Pipeline complet du fil rouge `library-api` — lecture

Lecture du pipeline

1. **Structure** : `agent none` ; le stage **CI** porte l'agent et un seul workspace pour ses stages imbriqués ; **Production** n'a pas d'agent
2. **Info** : checkout implicite (SCM du job), nom de build lisible (SHA court)
3. **Build & tests** : un seul `mvn verify` (unitaires + `*IT`) ; tests publiés même en échec ⇒ UNSTABLE, et `skipStagesAfterUnstable()` arrête là ; couverture JaCoCo via Coverage
4. **Quality** : Sonar après les tests, Quality Gate bloquant ; sauté sur les PR (leur analyse demande SonarQube Developer ou plus)
5. **Publish** (`main` uniquement) : JAR versionné `1.4.<build>` dans Nexus, image Docker dans le registry, fingerprint
6. **Deploy recette** : verrou sur l'environnement, rollout Kubernetes, smoke test
7. **Production** : approbation par un *release manager* (timeout 24 h propre au stage), milestone, délégation à un job CD paramétré
8. **post** : mails ciblés sur échec/instabilité, Slack quand ça repasse au vert ; nettoyage du workspace dans le stage CI

Prérequis à avoir configurés

- Tools `jdk21`, `maven-3.9` ; agent label `linux` avec Docker et kubectl
- Credentials : `nexus-deploy`, `docker-registry`, `kubeconfig-recette`, `sonar-token`
- Config file `nexus-settings` ; serveur SonarQube `sonarqube` + webhook ; SMTP ; Slack
- Plugins : Pipeline, Git/GitHub, JUnit, Coverage, SonarQube Scanner, Config File Provider, Docker Pipeline, Kubernetes CLI, Lockable Resources, Pipeline Milestone Step, Email Ext, Slack, Workspace Cleanup
- Groupe `release-managers` défini dans le security realm, avec `Job/Read` sur ce job (pour approuver l'input)

Schéma : le pipeline du fil rouge en un coup d'œil



Comment le lire

1. **Info** : le build porte le SHA du commit dans son nom.
2. **Build & tests** : un seul `mvn verify` ; tests et couverture publiés même en échec. Un test rouge rend le build UNSTABLE et arrête là.
3. **Quality** : analyse Sonar et Quality Gate bloquant (pas sur les PR).
4. **Publish** (main) : JAR dans Nexus, image dans le registry, avec la même version.
5. **Deploy recette** (main) : un seul déploiement à la fois grâce au verrou, puis smoke test.
6. **Production** : sans agent, le pipeline attend l'approbation, passe le `milestone` et délègue au job CD.

À retenir

Chaque stage ne s'exécute que si le précédent a réussi : la production ne voit que des versions testées, analysées et publiées.

Synthèse

1 – L'intégration continue

- Une **pratique** : intégrer souvent, vérifier automatiquement, réparer immédiatement
- Repose sur : SCM unique, build scripté et reproductible, tests automatisés, analyse qualité
- Chaîne de fabrication : checkout → build → tests → analyse → package → publish → deploy → feedback
- CI ⊂ Continuous Delivery ⊂ Continuous Deployment

2 – Jenkins

- Controller (UI, ordonnancement, JENKINS_HOME) + agents (exécution), 0 executor sur le controller
- Items : Freestyle pour comprendre, **Pipeline / Multibranch** pour produire
- Déclencheurs : webhook d'abord, cron avec `H`, polling en dernier recours
- Résultat : SUCCESS / UNSTABLE / FAILURE / ABORTED ; workspace sur l'agent, nettoyé
- Configuration : System, Tools, Credentials, Nodes, Plugins ; idéalement en JCasC

3 – Projet Java/Maven

- `mvn -B clean verify` + `junit` + `recordCoverage`
- Maven paramétré par `tools` / `withMaven`, settings.xml géré, cache `~/.m2`
- Notifications ciblées (`emailext` , culprits, fixed), Slack/Teams, status checks
- Publier dans Nexus/registry, déployer depuis le dépôt, `input` avant la prod
- SonarQube + Quality Gate bloquant ; Warnings NG ; Dependency-Check
- Habilitations : realm + stratégie (Matrix / Role-Based), folders, credentials cloisonnés

Le fil conducteur

Tout ce qui est fait dans l'UI pour apprendre doit finir **en code versionné** : Jenkinsfile dans le dépôt, Jenkins décrit en JCasC, plugins figés dans une image. C'est ce qui rend la chaîne reproductible, auditable et transmissible.

Ressources et pour aller plus loin

Documentation officielle

- `jenkins.io/doc` – guide utilisateur, Pipeline, administration
- `jenkins.io/doc/book/pipeline/syntax` – référence du Jenkinsfile déclaratif
- `<JENKINS_URL>/pipeline-syntax` – Snippet Generator (dans votre instance)
- `plugins.jenkins.io` – catalogue des plugins, documentation, santé des projets
- `jenkins.io/security/advisories` – bulletins de sécurité
- `maven.apache.org/guides` – Maven ; `docs.sonarsource.com` – SonarQube

Lectures

- Martin Fowler – *Continuous Integration* (article, 2024)
- Jez Humble, David Farley – *Continuous Delivery*
- Nicole Forsgren, Jez Humble, Gene Kim – *Accelerate* (métriques DORA)

Sujets d'approfondissement

- **Shared Libraries** : factoriser les pipelines de toute l'organisation (`vars/`, `src/`)
- **Configuration as Code** (JCasC) + **Job DSL** : Jenkins et ses jobs 100 % en fichiers
- **Agents Kubernetes** : pods éphémères, templates de pods, montée en charge
- **Haute disponibilité et sauvegarde** du controller ; mise à jour LTS
- **Sécurité de la supply chain** : signature des artefacts, SBOM (CycloneDX), attestations
- **GitOps** : Jenkins pour la CI, Argo CD / Flux pour le CD

Pour valider ses acquis

- Certification **Certified Jenkins Engineer** (CJE) – CloudBees
- Mettre en place de bout en bout la chaîne du fil rouge sur un projet personnel

Utopios – MA-IT Consulting · Jenkins – Intégration continue · Support v1.6